

الجمهورية الشعبية الديمقراطية الجزائرية
People's Democratic Republic of Algeria
وزارة التعليم العالي والبحث العلمي
Ministry of Higher Education and Scientific Research
المدرسة العليا للإعلام الآلي 8 ماي 1945 - سيدي بلعباس
Higher School of Computer Science
8 Mai 1945 - Sidi Bel Abbès



Graduation's Thesis

To obtain the diploma of State Engineer Degree

Field of Study: Computer Science

Specialization: Intelligence Artificielle et Science de Données (IASD)

Theme

Design and Implementation of a Multi-Role Consultation Platform with Integrated AI Services

Presented by
BELMANA Soufyane
Defended on: **28 June, 2026**
In front of the jury composed of:

Dr. BEZZAOUCHA Fatima Souad
Dr. CHIKH Asma
Pr. AMAR BENSABER Djamel
Dr. ABDELHAK Soumia

President
Examiner
Supervisor
Incubator Representative

Academic Year: 2025/2026

Acknowledgement

In the name of Allah, the Most Gracious, the Most Merciful.

First, I thank Allah for giving me the strength, patience, and determination to complete this work.

I would like to express my respect and gratitude to my supervisor, Djamel Amar Bensaber, for his supervision and guidance throughout this project.

I also thank the teachers and staff of Higher School of Computer Science of Sidi Bel Abbès for the knowledge and education they provided during my studies.

Finally, I am deeply grateful to my family and friends for their continuous support, encouragement, and prayers during this academic journey.

Abstract

The digital transformation of professional services has created a growing demand for platforms that efficiently connect clients with verified experts. This thesis presents the design, development, and implementation of Istachir, a comprehensive consultation marketplace platform that bridges the gap between clients seeking expert advice and qualified professionals offering consulting services. The platform encompasses two primary interfaces: a web-based administrative dashboard built with React, TypeScript, and Vite, and a cross-platform mobile application developed with React Native and Expo.

The system addresses key challenges in the consultation marketplace domain, including secure payment processing through an escrow-based mechanism, real-time video consultations via WebRTC technology, AI-powered session analysis and transcription using Google Gemini and AssemblyAI, and a robust multi-language interface supporting both English and Arabic. The backend infrastructure leverages Supabase for authentication, database management, real-time capabilities, and storage, creating a scalable and maintainable architecture.

Key features include expert discovery and profiling, interactive booking and scheduling, secure escrow payments integrated with the Chargily payment gateway, real-time messaging, and comprehensive administrative controls for platform management. The platform incorporates AI-driven features such as automated session summaries, expert profile Q&A, and post-session transcript analysis, enhancing the overall user experience and providing value-added services.

This work demonstrates the successful integration of modern web and mobile technologies to create a production-ready consultation marketplace, contributing to the growing ecosystem of digital service platforms in the Algerian market.

Keywords— Consultation Marketplace, Retrieval-Augmented Generation (RAG), AssemblyAI, Automated Session Summaries, WebRTC, Escrow Payments, AI Integration, Supabase

Résumé

La transformation numérique des services professionnels a créé une demande croissante pour des plateformes qui connectent efficacement les clients avec des experts vérifiés. Ce mémoire présente la conception, le développement et la mise en œuvre d’Istachir, une plateforme complète de marché de consultation qui comble le fossé entre les clients à la recherche de conseils d’experts et les professionnels qualifiés proposant des services de conseil. La plateforme comprend deux interfaces principales : un tableau de bord administratif en ligne développé avec React, TypeScript et Vite, et une application mobile multiplateforme développée avec React Native et Expo.

Le système répond aux défis majeurs du domaine du marché de la consultation, notamment le traitement sécurisé des paiements via un mécanisme d’entiercement (escrow), les consultations vidéo en temps réel grâce à la technologie WebRTC, l’analyse et la transcription de session assistées par IA à l’aide de Google Gemini et AssemblyAI, et une interface multilingue robuste prenant en charge l’anglais et l’arabe. L’infrastructure backend s’appuie sur Supabase pour l’authentification, la gestion de base de données, les fonctionnalités en temps réel et le stockage, créant ainsi une architecture évolutive et maintenable.

Les fonctionnalités clés incluent la découverte et le profilage d’experts, la réservation et la planification interactives, les paiements sécurisés par entiercement intégrés à la passerelle de paiement Chargily, la messagerie en temps réel et des contrôles administratifs complets pour la gestion de la plateforme. La plateforme intègre des fonctionnalités basées sur l’IA telles que les résumés de session automatisés, une FAQ pour les profils d’experts et l’analyse des transcriptions après session, améliorant ainsi l’expérience utilisateur globale et fournissant des services à valeur ajoutée.

Ce travail démontre l’intégration réussie des technologies web et mobiles modernes pour créer un marché de consultation prêt pour la production, contribuant ainsi à l’écosystème en plein essor des plateformes de services numériques sur le marché algérien.

Mots-clés— Marché de Consultation, Génération augmentée par récupération (RAG), AssemblyAI, Résumés de session automatisés, WebRTC, Paiements par Entiercement, Intégration de l’IA, Supabase

ملخص

أدّى التحول الرقمي في مجال الخدمات المهنية إلى تزايد الحاجة إلى منصّات رقمية تربط العملاء بالخبراء المعتمدين بطريقة فعّالة وآمنة. تقدّم هذه المذكرة تصميم وتطوير وتنفيذ منصّة إستشير (Istachir)، وهي سوق استشارات رقمي متكامل يهدف إلى سدّ الفجوة بين العملاء الباحثين عن استشارات متخصصة والخبراء المؤهلين الذين يقدّمون خدماتهم المهنية. تتكوّن المنصّة من واجهتين أساسيتين: لوحة تحكم إدارية مبنية على تقنيات الويب باستخدام React و TypeScript و Vite، وتطبيق هاتف محمول متعدّد المنصّات مطوّر بإطار عمل React Native ومنظومة Expo.

يعالج النظام عدداً من التحديات الجوهرية في مجال الاستشارات الرقمية، تشمل: المعالجة الآمنة للدفوعات عبر آلية الضمان المالي (Escrow)، والاستشارات المرئية الفورية عبر بروتوكول WebRTC، والتحليل الذكي لجلسات الاستشارة بما في ذلك التفرغ الصوتي التلقائي باستخدام AssemblyAI وتوليد الملخصات المنظّمة بواسطة نموذج Google Gemini، فضلاً عن واجهة مستخدم ثنائية اللغة تدعم العربية والإنجليزية مع دعم كامل للاتجاه من اليمين إلى اليسار (RTL). تعتمد البنية الخلفية للمنصّة على Supabase كمنصّة خدمات خلفية متكاملة (BaaS)، توفر المصادقة، وقاعدة بيانات PostgreSQL، والاتصال الفوري عبر WebSocket، وخدمات التخزين السحابي.

من أبرز وظائف المنصّة: استعراض واكتشاف الخبراء وفقاً لتخصّصاتهم وتقييماتهم وشهاداتهم المهنية، والحجز التفاعلي للجلسات، والدفع الآمن عبر بوابة Chargily بالدينار الجزائري (بطاقات CIB و Edahabia)، والمراسلة الفورية، وأدوات الإدارة الشاملة للمشرفين. كما تضمّن المنصّة ميزات ذكاء اصطناعي أصيلة تتجاوز الاستهلاك البسيط لواجهات برمجة التطبيقات، وتشمل: روبوت محادثة ذكي يعمل بنمط RAG يجيب عن أسئلة العملاء حول الخبراء اعتماداً على بيانات ملفّاتهم الشخصية، وخطّ أنابيب آلي لتوليد ملخصات الجلسات يمرّ بمراحل التسجيل الصوتي ثمّ التفرغ النصّي ثمّ هندسة الأوامر (Prompt Engineering) وصولاً إلى الملخص المنظّم. يُثبت هذا العمل نجاح دمج تقنيات الويب والهاتف المحمول الحديثة في بناء سوق استشارات رقمي جاهز للتشغيل الفعلي، ويسهم في إثراء المنظومة الرقمية المتنامية لمنصّات الخدمات المهنية في السوق الجزائرية.

الكلمات المفتاحية--- سوق الاستشارات، توليد الاسترجاع المعزز، (AssemblyAI (RAG، ملخصات الجلسات المؤتمتة، WebRTC، مدفوعات الضمان المالي، تكامل الذكاء الاصطناعي، Supabase

I	Introduction	1
1	General Introduction	2
1.1	Context and Motivation	2
1.2	Problem Statement and Challenges	2
1.2.1	Technical Infrastructure Limitations	3
1.2.2	Scalability and Resource Management	3
1.2.3	Security and Isolation Concerns	3
1.2.4	Multi-Platform Consistency	3
1.2.5	Payment Integration and Financial Infrastructure	3
1.3	Objectives and Scope	3
1.3.1	Primary Objectives	3
1.3.2	Secondary Objectives	4
1.3.3	Scope and Limitations	4
1.4	Thesis Organization	4
II	Background and Technical Foundation	6
2	Background and Technical Foundation	7
2.1	Introduction	7
2.2	Consultation Marketplace Business Model	7
2.2.1	Business Model Canvas	8
2.2.2	Project Roadmap and Development Strategy	10
2.2.2.1	Key Development Phases and Milestones	10
2.2.2.2	Necessary Resources and Financing	10
2.2.2.3	Five-Year Financial Projections	11
2.3	Comparative Study of Existing Consultation Platforms	11
2.3.1	Analysis of Existing Platforms	11
2.3.2	The Istachir Approach	12
2.4	Frontend and User Interfaces	13
2.4.1	React.js for Administrative Interfaces	13
2.4.2	React Native and Expo for Cross-Platform Mobile Development	14
2.4.3	Performance Optimization for Mobile Commerce	14

2.5	Backend Development	15
2.5.1	Supabase Backend Platform	15
2.5.1.1	PostgreSQL Database	15
2.5.1.2	Authentication System	15
2.5.2	RESTful API Design	16
2.5.3	Justification of Technical Stack Choices	16
2.5.3.1	Supabase versus Firebase	16
2.5.3.2	React Native vs. Native Development (Swift/Kotlin)	17
2.5.3.3	React.js for the Web Admin Dashboard	17
2.6	Real-Time Capabilities	17
2.6.1	WebRTC for Video Consultations	17
2.6.1.1	WebRTC Architecture	17
2.6.1.2	Multi-Platform WebRTC	18
2.6.2	Real-Time Messaging	18
2.7	Security Frameworks and Authentication	18
2.7.1	Multi-Layer Security Architecture	18
2.7.2	Authentication and Authorization Strategies	19
2.8	Artificial Intelligence Integration	19
2.8.1	Google Gemini AI	19
2.8.2	AssemblyAI Platform	20
2.8.2.1	The Principle of AssemblyAI	20
2.8.2.2	Core Product Offerings	20
2.8.2.3	Key Features for Summarization	20
2.8.2.4	How It Works in Practice	20
2.8.2.5	Pricing and Limitations	21
2.8.3	Automated Session Summaries	21
2.8.3.1	Key Phases of the Summarization Process	21
2.8.3.2	Technical Viability and Context Windows	22
2.8.3.3	Strategic Advantages	22
2.8.3.4	Limitations and Precautions	22
2.9	System Architecture Overview	23
2.10	Conclusion	23

III Requirements Analysis 24

3	Requirements Analysis	25
3.1	Introduction	25
3.2	Functional Requirements Specification	25
3.2.1	User Management and Authentication	25
3.2.2	Expert and Service Management	26
3.2.3	Booking and Consultation Requirements	26
3.2.4	Payment and Escrow Requirements	27
3.2.5	Content and Communication Requirements	27
3.3	Non-Functional Requirements Specification	28
3.3.1	Usability Requirements	28
3.3.2	Security Requirements	28
3.3.3	Performance Requirements	29

3.4	Identification of Actors	29
3.4.1	Administrators	29
3.4.2	Experts	29
3.4.3	Clients	30
3.5	Use Case Diagrams	30
3.5.1	User Management Use Cases	32
3.5.2	Consultation Booking Use Cases	32
3.5.3	Payment Management Use Cases	33
3.5.4	Content and Communication Use Cases	33
3.5.5	Detailed Use Case Descriptions	34
3.6	Activity Diagrams	46
3.6.1	Consultation Booking Process	46
3.6.2	Escrow Release Process	46
3.6.3	Video Call Connection Process	47
3.7	Sequence Diagrams	49
3.7.1	User Authentication Sequence	49
3.7.2	Booking Creation Sequence	50
3.7.3	Messaging Sequence	50
3.8	Conclusion	54

IV System Design 55

4	System Design	56
4.1	Introduction	56
4.1.1	Profile Management Domain	58
4.1.2	Consultation and Service Domain	58
4.1.3	Communication Domain	58
4.1.4	Social Content Domain	58
4.1.5	Credentials and Verification Domain	59
4.2	Relationship Patterns and Data Integrity	59
4.3	Technical Architecture Implementation	59
4.3.1	Presentation Layer	59
4.3.2	Application Layer	60
4.3.3	Data Layer	60
4.3.4	Integration Layer	60
4.4	Deployment Architecture	61
4.4.1	Client-Side Components	61
4.4.2	Server Infrastructure	61
4.4.3	Third-Party Services	61
4.5	AI Workflow Architecture	61
4.5.1	Retrieval-Augmented Generation (RAG) for Expert Profile Q&A	61
4.5.2	Session Summary Generation Pipeline	63
4.6	Conclusion	64

V Implementation and Technical Realization 65

5	Platform Implementation	66
5.1	Introduction	66
5.2	Development Environment and Technology Stack	66
5.2.1	Development Hardware Configuration	66
5.2.2	Software Environment	66
5.2.3	Version Control and Collaboration	67
5.3	Web Admin Dashboard Implementation	67
5.3.1	User Management Systems	69
5.3.2	Expert and Service Management	71
5.3.3	Booking and Schedule Management	71
5.3.4	Payment and Financial Management	73
5.3.5	Real-Time Communication	75
5.4	Customer Mobile Application Implementation	76
5.4.1	Authentication and Onboarding	76
5.4.2	Expert Discovery and Profiles	77
5.4.3	Booking and Scheduling	79
5.4.4	Video Consultation (WebRTC)	81
5.4.5	AI-Powered Session Features	82
5.4.6	Messaging System	83
5.4.7	Financial Dashboard	84
5.5	Backend and Database Implementation	85
5.5.1	Database Schema	85
5.5.2	API Implementation	89
5.5.3	Row-Level Security Policies	90
5.6	Internationalization Implementation	90
5.7	System Testing, Evaluation, and Validation	90
5.7.1	Functional Testing	90
5.7.2	Performance Evaluation	91
5.7.2.1	WebRTC Connection Latency	91
5.7.2.2	Database Query Optimization	92
5.7.2.3	AI Pipeline Processing Latency	92
5.7.3	Security Policy Validation	92
5.8	Conclusion	93

VI Conclusion and Perspectives 94

6	Conclusion and Perspectives	95
6.1	General Conclusion	95
6.2	Key Achievements and Contributions	95
6.2.1	Technical Innovation	95
6.2.2	Practical Contributions	96
6.3	Limitations and Challenges	96
6.3.1	Technical Limitations	96
6.3.2	Implementation Challenges	96
6.4	Future Perspectives	97

6.4.1	Short-Term Enhancements (6-12 Months)	97
6.4.2	Medium-Term Developments (1-2 Years)	97
6.4.3	Long-Term Vision (2-5 Years)	97
6.5	Final Remarks	97

LIST OF FIGURES

2.1	Python Implementation of AssemblyAI Summarization	21
2.2	System Layered Architecture	23
3.1	User Management Subsystem Use Case Diagram	30
3.2	Consultation & Session Subsystem Use Case Diagram	31
3.3	Payment & Escrow Subsystem Use Case Diagram	31
3.4	Content & Communication Subsystem Use Case Diagram	32
3.5	Consultation Booking Process Activity Diagram	48
3.6	Escrow Release Process Activity Diagram	49
3.7	User Authentication Sequence Diagram	51
3.8	Booking Creation Sequence Diagram	52
3.9	Session Recording and Speech-to-Text Transcription Sequence Diagram	53
4.1	Core Entity UML Class Diagram	57
4.2	Profile Q&A Chatbot RAG Workflow	62
4.3	AI Session Summary Generation Pipeline Architecture	63
5.1	Web Dashboard Landing Page	68
5.2	Web Login Page	69
5.3	Web Sign Up Page	69
5.4	Web Dashboard – Expert Browsing Interface	70
5.5	Web – Expert Posts Directory Interface	70
5.6	Web Dashboard – Expert Profile View	71
5.7	Web Dashboard – Booking Schedule Calendar	72
5.8	Web Dashboard – Administration Management Interface	72
5.9	Web Dashboard – Booking Detail View	73
5.10	Web Dashboard – Post-Session Summary View	73
5.11	Web – Chargily Payment Checkout Page	74
5.12	Web Dashboard – Payment and Webhook Confirmation	74
5.13	Web Dashboard – Messaging Interface	75
5.14	Web Dashboard – Video Consultation Room	76
5.15	Mobile App – Authentication Flow (Splash, Login, Sign Up)	77
5.16	Mobile App – Expert Discovery and Community (Home, Search, Posts)	78
5.17	Mobile App – Expert Profile and Client Dashboard	79

5.18	Mobile App – Booking Interface and Checkout Details	80
5.19	Mobile App – Completed Session and AI-Generated Summary	81
5.20	Mobile App – Video Consultation Room and Appointment Details	82
5.21	Mobile App – Post-Session AI Summary and Transcription View	83
5.22	Mobile App – Real-Time Chat Interface	84
5.23	Mobile App – Booking Details and Wallet/Finance View	85

LIST OF TABLES

2.1	Istachir Business Model Canvas	9
2.2	Istachir Five-Year Project Roadmap	10
2.3	Five-Year Financial Projections (in DZD)	11
2.4	Feature Comparison of Consultation Platforms	13
3.1	User Management Functional Requirements	26
3.2	Expert Management Functional Requirements	26
3.3	Booking Functional Requirements	27
3.4	Payment Functional Requirements	27
3.5	Content and Communication Functional Requirements	28
3.6	Detailed Description of UC01: User Registration	34
3.7	Detailed Description of UC02: User Authentication	35
3.8	Detailed Description of UC03: Profile Management	35
3.9	Detailed Description of UC04: Password Reset	36
3.10	Detailed Description of UC05: Role Management (Admin)	36
3.11	Detailed Description of UC06: Browse Experts	37
3.12	Detailed Description of UC07: View Expert Profile	37
3.13	Detailed Description of UC08: Schedule Consultation	38
3.14	Detailed Description of UC09: Process Payment	38
3.15	Detailed Description of UC10: Join Video Call	39
3.16	Detailed Description of UC11: View Session Summary	40
3.17	Detailed Description of UC12: Rate and Review	40
3.18	Detailed Description of UC13: Release Escrow	41
3.19	Detailed Description of UC14: File Dispute	41
3.20	Detailed Description of UC15: Configure Payout Account	42
3.21	Detailed Description of UC16: View Transaction History	42
3.22	Detailed Description of UC17: Manage Payouts (Admin)	43
3.23	Detailed Description of UC18: Create and Manage Posts	43
3.24	Detailed Description of UC19: Like and Comment on Posts	44
3.25	Detailed Description of UC20: Report Post	44
3.26	Detailed Description of UC21: Manage Certifications	45
3.27	Detailed Description of UC22: Send Messages	45
3.28	Detailed Description of UC23: Manage Conversations	46

5.1	Development Software Stack	67
5.2	Supabase Database Schema: profiles Table	86
5.3	Supabase Database Schema: services Table	86
5.4	Supabase Database Schema: posts Table	87
5.5	Supabase Database Schema: bookings Table	87
5.6	Supabase Database Schema: reviews Table	88
5.7	Supabase Database Schema: conversations Table	88
5.8	Supabase Database Schema: messages Table	88
5.9	Supabase Database Schema: certifications Table	88
5.10	Supabase Database Schema: post_likes Table	89
5.11	Supabase Database Schema: post_comments Table	89
5.12	Supabase Database Schema: post_reports Table	89
5.13	Functional Test Suite Results	91
5.14	WebRTC Connection Establishment Latency	91
5.15	Expert Search Query Execution Time (ms)	92
5.16	AI Summarization Pipeline Latency Breakdown	92

- API** Application Programming Interface. 17–19
- BaaS** Backend as a Service. 15
- CCP** Compte Chèque Postal. 3
- DOM** Document Object Model. 13
- DZD** Algerian Dinar. 3
- EAS** Expo Application Services. 14
- ICE** Interactive Connectivity Establishment. 18
- JWT** JSON Web Token. 19
- RIP** Relevé d’Identité Postal. 3
- RLS** Row Level Security. 3, 15, 19
- RPC** Remote Procedure Call. 16
- RTL** Right-to-Left. 4
- SRTP** Secure Real-Time Transport Protocol. 18
- STUN** Session Traversal Utilities for NAT. 18
- TURN** Traversal Using Relays around NAT. 18
- UI** User Interface. 18
- WebRTC** Web Real-Time Communication. 3

Part I

Introduction

1.1 Context and Motivation

The rapid digitization of professional services has fundamentally transformed how individuals and businesses access expertise. Traditional models of consulting typically require significant time investment for scheduling, geographical proximity, and often involve complex payment arrangements. The emergence of digital marketplace platforms has disrupted this paradigm by enabling on-demand access to a global pool of experts across various domains.

In the Algerian context, the demand for qualified professional advice spans multiple sectors including business management, legal consultation, career guidance, health and wellness, technology, and education. However, several barriers hinder seamless access to expertise: geographical dispersion of qualified professionals, lack of transparent pricing, absence of verified credential systems, and limited digital payment infrastructure adapted to the local market.

The motivation for this thesis stems from the need to create a comprehensive digital platform that addresses these challenges by providing:

- A centralized marketplace where clients can discover and connect with verified experts.
- Secure and transparent payment mechanisms adapted to the Algerian financial ecosystem, including CCP and RIP systems.
- Real-time video consultation capabilities enabling remote expert-client interactions.
- AI-powered tools to enhance the consultation experience through automated summaries, transcription, and intelligent assistance.
- A bilingual interface supporting both English and Arabic to serve a diverse user base.

1.2 Problem Statement and Challenges

The development of Istachir presents several challenges that span technical, business, and user experience domains.

1.2.1 Technical Infrastructure Limitations

Building a platform that supports real-time video consultations, live messaging, and secure payments requires a robust technical infrastructure. The system must handle concurrent video sessions, maintain low latency for real-time features, and ensure high availability. The choice of technologies must balance development velocity with production reliability, particularly given the need for real-time bidirectional communication through Web Real-Time Communication (WebRTC) and Supabase Realtime subscriptions.

1.2.2 Scalability and Resource Management

As the platform grows, it must efficiently manage increasing numbers of users, consultations, and data volumes. The architecture must support horizontal scaling of both the frontend applications and the backend services. The use of Supabase as a Backend-as-a-Service provides a strong foundation, but careful consideration of database query optimization, real-time channel management, and storage utilization is essential.

1.2.3 Security and Isolation Concerns

The platform handles sensitive user data including personal information, financial details, and consultation recordings. Implementing robust security measures including Row Level Security (RLS), secure authentication flows, encrypted data transmission, and proper access control mechanisms is paramount. The escrow-based payment system introduces additional complexity in ensuring transactional integrity and dispute resolution.

1.2.4 Multi-Platform Consistency

Delivering a consistent user experience across web and mobile platforms while maintaining feature parity presents significant development challenges. The web administrative dashboard requires comprehensive management capabilities, while the mobile application must provide an intuitive and responsive interface optimized for on-the-go usage. Ensuring seamless cross-platform behavior for features like video calls, messaging, and payments is particularly challenging.

1.2.5 Payment Integration and Financial Infrastructure

Integrating with the Algerian payment ecosystem presents unique challenges. The Chargily payment gateway must be properly configured to handle Algerian Dinar (DZD) transactions, while the escrow system requires careful state management to protect both clients and experts. The platform must support multiple payout methods including Compte Chèque Postal (CCP) and Relevé d'Identité Postal (RIP) accounts.

1.3 Objectives and Scope

1.3.1 Primary Objectives

The primary objective of this thesis is to design, develop, and deploy a comprehensive consultation marketplace platform that:

1. Provides a web-based administrative dashboard for platform management, user administration, financial oversight, and content moderation.
2. Delivers a cross-platform mobile application enabling clients to discover experts, book consultations, participate in video calls, and manage payments.
3. Implements a secure escrow-based payment system integrated with local payment gateways.
4. Incorporates AI-powered features for session analysis, transcription, and intelligent assistance.
5. Supports bilingual interfaces (English and Arabic) with full Right-to-Left (RTL) support.
6. Ensures real-time communication capabilities for video consultations and messaging.

1.3.2 Secondary Objectives

Secondary objectives include:

- Establishing a scalable and maintainable technical architecture.
- Implementing comprehensive security measures including role-based access control and data protection.
- Creating an intuitive user experience that minimizes friction in the consultation workflow.
- Providing analytics and reporting capabilities for platform administrators.
- Building a foundation for future enhancements including loyalty programs, AI-driven recommendations, and dedicated delivery partner applications.

1.3.3 Scope and Limitations

The scope of this thesis encompasses the complete software development lifecycle including requirements analysis, system design, implementation, and testing of the Istachir platform. The project delivers two primary applications: a web-based administrative dashboard (Istachir Web Admin Portal) and a cross-platform mobile application (Istachir Mobile Application), both sharing a common Supabase backend infrastructure.

Areas explicitly outside the scope include: native iOS and Android application development (the mobile app uses React Native with Expo for cross-platform deployment), the development of a dedicated delivery partner application, comprehensive machine learning model training (existing AI APIs are leveraged), and the physical infrastructure for deployment (cloud hosting configuration is referenced but not implemented).

1.4 Thesis Organization

This thesis is organized into six chapters structured across four parts:

- **Part I: Introduction** – The current chapter establishes the context, motivation, problem statement, objectives, and scope of the project.

- **Part II: Background and Technical Foundation** – Chapter 2 explores the theoretical foundations and technologies underpinning the platform, including consultation marketplace models, frontend and backend technologies, real-time communication protocols, and security frameworks.
- **Part III: Requirements Analysis** – Chapter 3 presents a comprehensive analysis of functional and non-functional requirements, identifies system actors, and provides detailed use case, activity, and sequence diagrams.
- **Part IV: System Design** – Chapter 4 details the system architecture including class diagrams, entity relationships, technical architecture layers, and deployment considerations.
- **Part V: Implementation** – Chapter 5 describes the development environment, technology stack, and detailed implementation of both the web administration dashboard and the mobile application.
- **Part VI: Conclusion and Perspectives** – Chapter 6 summarizes the achievements, discusses limitations, and presents future perspectives for platform evolution.

Part II

Background and Technical Foundation

CHAPTER 2

BACKGROUND AND TECHNICAL FOUNDATION

2.1 Introduction

This chapter provides a comprehensive overview of the theoretical foundations and technologies employed in the development of the Istachir consultation marketplace platform. We examine the consultation marketplace business model, explore the frontend and backend technologies, analyze real-time communication protocols, and review security frameworks that form the technical backbone of our system.

2.2 Consultation Marketplace Business Model

Consultation marketplaces are digital platforms that connect individuals seeking expert advice with qualified professionals offering specialized knowledge and services. These platforms operate on a two-sided market model, creating value for both clients and experts through:

- **Discovery and Matching:** Clients can search, filter, and browse expert profiles based on expertise areas, ratings, availability, and pricing. Advanced search algorithms and AI-powered recommendations enhance the matching process.
- **Trust and Verification:** Platforms implement verification mechanisms including credential validation, review systems, and rating aggregations to establish trust between parties. AI-generated profile summaries and review analyses provide additional transparency.
- **Secure Transactions:** Escrow-based payment systems protect both parties by holding funds until service delivery is confirmed. Platform commission models typically range from 10% to 30% of transaction value.
- **Communication Infrastructure:** Integrated video, audio, and text communication tools enable remote consultations, eliminating geographical barriers and reducing scheduling complexity.

The consultation marketplace model has been successfully implemented by global platforms such as JustAnswer, Clarity.fm, and Upwork, each targeting specific market segments. Istachir

differentiates itself by focusing on the Algerian market with localized payment integration, bilingual support, and AI-enhanced consultation features.

2.2.1 Business Model Canvas

To further define the operational and strategic framework of Istachir, a Business Model Canvas (BMC) is presented in Table 2.1. This canvas illustrates how the platform creates, delivers, and captures value within the Algerian consultation ecosystem.

Table 2.1: *Istachir Business Model Canvas*

Key Partners	Key Activities	Value Propositions	Customer Relationships	Customer Segments
- Experts (legal, medical, finance, tech, HR, etc.) - Algerian universities & higher education institutions - Incubators (Algeria Venture, Flat6Labs, etc.) - Payment operators: CIB, BaridiMob, Dahabiya - National professional associations Technical Stack: - Supabase - Vercel - Gemini API - AssemblyAI - Namecheap	- Web & mobile development & maintenance - Expert profile management & moderation - Secure payment processing - Continuous improvement of the AI assistant - Client & expert acquisition - Support & service quality control	For Clients: - Find the right expert easily, anywhere in Algeria - AI assistant to prepare for the consultation - Secure audio/video sessions with automatic summaries - Historical records & recordings of each session For Experts: - Nationwide visibility through enriched profiles - Revenue generation without administrative hassle - Publication of articles, projects, and certifications - Zero fixed fees — commission-based pricing only	- Integrated AI assistant (guided self-service) - Pre-booking text chat - Post-session ratings & reviews - Post-session audio recordings & AI summaries - Responsive customer support	Clients (Seekers): - Individuals seeking advice (legal, health, finance, etc.) - Entrepreneurs & project founders - Students seeking career orientation Experts (Providers): - Freelance professionals (lawyers, doctors, accountants, etc.) - Independent consultants & coaches - Engineers & technical experts - University professors & researchers
	Key Resources - Web portal (Vercel + istachir.me) - Database & auth (Supabase) - Conversational AI (Gemini API) - Voice transcription (AssemblyAI) - iOS & Android mobile applications - Tech team (2 developers) + marketing		Channels - Website (istachir.me) - iOS & Android mobile applications - Social media (Instagram, Facebook, LinkedIn) - Word-of-mouth & user recommendations - University & professional association partnerships	
Cost Structure		Revenue Streams		
Infrastructure & APIs: - Supabase (DB, auth, storage) ~\$30/month - Vercel hosting (Free/Pro ~\$20/month) - Gemini API (AI services) — pay-per-use - AssemblyAI (transcription) — pay-per-use - Namecheap (domain) ~\$15/year - WebRTC / Agora (audio-video) — pay-per-use		10% Unified Commission per session: - Client pays the expert via the platform. - Istachir automatically deducts a 10% commission fee. - Expert receives 80% net payout. - Payment methods: CIB, BaridiMob, Dahabiya. - Zero subscription fees, zero fixed costs for experts.		

2.2.2 Project Roadmap and Development Strategy

The growth and scalability of the Istachir platform rely on a structured, multi-phase roadmap spanning five years. This strategic plan coordinates technical development, commercial milestones, resource acquisition, and financial projections to establish Istachir as the leading consultation marketplace in Algeria.

2.2.2.1 Key Development Phases and Milestones

The platform’s roll-out is structured into five sequential phases, each with specific timelines, objectives, and deliverables:

Table 2.2 shows the istachir Five-Year Project Roadmap.

Table 2.2: *Istachir Five-Year Project Roadmap*

Phase	Timeline	Core Objectives & Deliverables
Phase 1: Concept & Planning	Months 1–3	Market research, feature definition, UI/UX wireframing, and legal/regulatory compliance analysis for digital payment services in Algeria.
Phase 2: Technical Development	Months 4–9	MVP engineering, database schema configuration (Supabase), real-time WebRTC room deployment, and Gemini/AssemblyAI API integration.
Phase 3: Commercial Launch	Months 10–12	Closed beta testing with 50 users and experts, public store release (iOS, Android, Web), and target user acquisition campaigns.
Phase 4: Growth & Expansion	Years 2–3	Target 5,000 active users, partner with universities and syndicates, integrate advanced escrow automation, and refine AI models.
Phase 5: Market Leadership	Years 4–5	Expand to international consulting markets (Algerian diaspora), introduce premium subscriptions, and implement sovereign cloud hosting.

2.2.2.2 Necessary Resources and Financing

To execute this roadmap successfully, Istachir requires a coordinated set of resources across three primary domains:

- **Human Resources:** A core team comprising 2 full-stack developers (managing Web and Mobile frontend and Supabase integration), 1 AI specialist (focusing on prompt engineering and transcription integration), 1 digital marketing coordinator, and 1 client support agent.
- **Technical Resources:** Production-grade cloud hosting (Supabase Pro plan for database, auth, and storage), API subscription credits (Gemini, AssemblyAI, and SMS gateways), custom domain names, App Store and Google Play developer accounts, and local payment gateway merchant keys (Chargily).

- **Financial Resources & Funding:** Initial seed funding of approximately 500,000 DZD is required to cover early server costs, developer accounts, domain names, and initial marketing campaigns. Subsequent operations will be financed through the platform’s transaction fee model.

2.2.2.3 Five-Year Financial Projections

Istachir’s financial model is built on a 10% commission fee per completed consultation session. Table 2.3 outlines the estimated total revenues, operational costs (hosting, marketing, API bills, human resources), and net profit projections for the first five years of operation (all figures are in Algerian Dinars, DZD).

Table 2.3: *Five-Year Financial Projections (in DZD)*

Financial Metric	Year 1	Year 2	Year 3	Year 4	Year 5
Total Revenue	500,000	1,200,000	2,500,000	4,500,000	7,000,000
Total Operating Cost	300,000	700,000	1,400,000	2,300,000	3,500,000
Net Profit	200,000	500,000	1,100,000	2,200,000	3,500,000

2.3 Comparative Study of Existing Consultation Platforms

To position the Istachir platform within the existing professional consulting landscape, a comprehensive comparative study was conducted. We analyze three global consulting and freelancing platforms (Clarity.fm, JustAnswer, and Upwork) and one Algerian startup platform (Moustachir).

2.3.1 Analysis of Existing Platforms

- **Clarity.fm:** Clarity.fm is a global web-based marketplace that connects clients with experts in business, entrepreneurship, and technology. The communication model is primarily audio-based, with pay-per-minute billing processed through Stripe. While effective globally, Clarity.fm lacks mobile apps, does not offer integrated AI services (such as transcription, semantic profile search, or summaries), and does not support local payment methods in Algeria (like Edahabia or CIB cards in DZD).
- **JustAnswer:** JustAnswer connects users with certified professionals in fields such as law, medicine, veterinary services, and tech support. It operates on a subscription-based recurring revenue model and is available on web and mobile. However, its communication is purely asynchronous chat-based, and it lacks live WebRTC-based video rooms or post-session AI analytics. Furthermore, its payment options are limited to international credit cards, and it does not cater to the local Algerian market context.
- **Upwork:** Upwork is a leading global freelancing platform supporting various project formats (hourly, fixed-price). While it includes audio/video calls and instant messaging, it is not specialized for immediate, on-demand consultation sessions. The platform does not provide automated session transcription, post-consultation summaries, or dedicated expert profile

chatbots. Furthermore, local payment integration for the Algerian Dinar (DZD) is absent, limiting its accessibility for Algerian clients and experts.

- **Moustachir:** Moustachir is an Algerian consulting platform. It operates exclusively as a web-based portal and does not offer a mobile application. Architecturally, it has no integrated artificial intelligence capabilities. Discovery of experts is limited, as the platform does not host expert-written posts, blogs, or verification certificates for public viewing. Expert profiles display only a brief text description. Consequently, clients are practically required to know the expert and their credentials prior to visiting the platform in order to select them. This creates a significant discovery barrier.

2.3.2 The Istachir Approach

Istachir is designed to address the shortcomings of both global platforms and local competitors. In contrast to Moustachir, Istachir implements:

- **Integrated Multi-Role Mobile Application:** Built with React Native, providing a unified mobile interface for both clients and experts.
- **Advanced Discovery Engine:** Clients can browse and discover vetted experts dynamically. Profiles display verifiable academic and professional certifications, ratings, and expert-authored posts/blogs, allowing users to assess an expert's competence before booking.
- **Localized Payments (Chargily Gateway):** Complete integration of the Chargily payment gateway, enabling clients to pay securely in DZD using local Edahabia (Algérie Poste) and CIB bank cards. An escrow mechanism holds the payment until the session is successfully completed.
- **Integrated AI Ecosystem:** Features an AI Q&A chatbot trained on the expert's profile for client inquiries, along with automated WebRTC session recording, transcription (AssemblyAI), and structured summary generation (Gemini) saved directly to the database.

A detailed feature comparison is summarized in Table 2.4.

Table 2.4: *Feature Comparison of Consultation Platforms*

Feature	Clarity.fm	JustAnswer	Upwork	Moustachir	Istachir
Target Market	Global	Global	Global	Algeria (Local)	Algeria (Local)
Platform Apps	Web Only	Web / Mobile	Web / Mobile	Web Only	Web / Mobile
Payment Base	USD (Stripe/PayPal)	USD (Intl Card)	USD (Intl Card)	DZD (CCP Bank Transfer)	DZD (Chargily: CIB, Edahabia)
Payment Model	Pay-per-minute	Subscription	Fixed/Hourly	Direct Payment	Escrow with 10% Platform Fee
Live Medium	Telephone Call	Asynchronous Chat	Chat/Video Call	External Tools	Integrated WebRTC Video
Discovery Basis	Keyword search	Category routing	Job posts	Prior knowledge required	Vetted profile, Certs, Posts, Reviews
AI Chatbot	None	None	None	None	Gemini Profile Q&A
AI Transcription	None	None	None	None	AssemblyAI Speech-to-Text
AI Summaries	None	None	None	None	Gemini Session Summarization

2.4 Frontend and User Interfaces

2.4.1 React.js for Administrative Interfaces

React.js is a declarative, component-based JavaScript library developed by Meta for building user interfaces. Its virtual Document Object Model (DOM) implementation provides efficient rendering by minimizing direct manipulations of the browser DOM. Key advantages that make React.js suitable for the administrative dashboard include:

- **Component Reusability:** The shadcn/ui component library [1], built on Radix UI primitives and styled with Tailwind CSS [2], provides a comprehensive set of accessible, customizable components that accelerate development while maintaining design consistency.
- **State Management:** TanStack React Query [3] (formerly React Query) provides powerful server state management capabilities including automatic caching, background refetching, optimistic updates, and pagination support. This is particularly valuable for data-intensive administrative interfaces.

- **Type Safety:** TypeScript [4] integration provides static type checking that reduces runtime errors and improves code maintainability across the extensive component hierarchy.
- **Build Performance:** Vite, the build tool used in this project, leverages native ES modules and SWC-based compilation to achieve significantly faster development server startup and hot module replacement compared to traditional bundlers like Webpack.

2.4.2 React Native and Expo for Cross-Platform Mobile Development

React Native is an open-source framework developed by Meta that enables the development of native mobile applications using JavaScript and React. Expo [5] is a managed workflow built on top of React Native that provides additional tooling, libraries, and services to simplify mobile app development. The choice of React Native with Expo for the customer mobile application was driven by several factors:

- **Cross-Platform Deployment:** A single codebase targets both iOS and Android platforms, significantly reducing development and maintenance costs compared to native development.
- **Expo Ecosystem:** Expo provides managed services including push notifications, over-the-air updates, build automation through Expo Application Services (EAS), and a comprehensive library of pre-built native modules.
- **Native Performance:** React Native components compile to native views, providing performance characteristics close to native applications. The new architecture (Fabric renderer and TurboModules) further improves performance and interoperability.
- **Code Sharing:** Knowledge and patterns from the web React codebase can be leveraged, including shared utilities for API communication, state management patterns, and TypeScript type definitions.
- **NativeWind Integration:** Tailwind CSS utility classes are available in React Native through NativeWind, enabling consistent styling approaches across web and mobile platforms.

2.4.3 Performance Optimization for Mobile Commerce

Mobile application performance is critical for user retention and conversion rates. The Istachir mobile app implements several optimization strategies:

- **Efficient Data Fetching:** TanStack React Query provides intelligent caching with stale-while-revalidate strategies, reducing network requests and improving perceived performance.
- **Image Optimization:** Supabase Storage provides image transformation capabilities including resizing, compression, and format conversion. Images are loaded with progressive resolution and proper caching headers.
- **Reanimated Animations:** React Native Reanimated provides performant animations that run on the UI thread rather than the JavaScript thread, ensuring smooth transitions and interactions.

- **Minimized Bundle Size:** Expo’s tree-shaking capabilities and proper import optimization reduce the application bundle size, improving initial load times.

2.5 Backend Development

2.5.1 Supabase Backend Platform

Supabase [6] is an open-source Backend as a Service (BaaS) that provides a comprehensive backend infrastructure including a PostgreSQL database, authentication, real-time subscriptions, storage, and edge functions. It serves as the sole backend for Istachir, replacing traditional custom backend development.

2.5.1.1 PostgreSQL Database

PostgreSQL is a powerful, open-source object-relational database system with over 30 years of active development. Its feature set includes:

- **Advanced Indexing:** Support for B-tree, Hash, GiST, GIN, and BRIN indexes enables efficient querying across diverse data access patterns.
- **RLS:** Row-level security policies provide granular access control at the database level, ensuring users can only access data they are authorized to see.
- **Foreign Data Wrappers:** Enable integration with external data sources and services.
- **Full-Text Search:** Built-in full-text search capabilities support efficient text-based queries across expert profiles, posts, and services.

2.5.1.2 Authentication System

Supabase Auth provides a complete authentication system supporting multiple authentication methods:

- **Email and Password:** Traditional authentication with built-in password hashing, session management, and email verification.
- **Google OAuth:** Single sign-on integration enabling users to authenticate with their Google accounts.
- **Session Management:** Automatic token refresh and session persistence through HTTP-only cookies and local storage.
- **Role-Based Access:** Custom user metadata and database-level roles enable granular permission management for administrators, experts, and clients.

2.5.2 RESTful API Design

While Supabase provides direct database access through its client library, the application implements a service-oriented architecture within the frontend codebase. API interactions follow consistent patterns:

- **Query Operations:** Data retrieval uses Supabase’s query builder with filtering, sorting, pagination, and relationship joins.
- **Mutation Operations:** Data modifications follow optimistic update patterns where the UI updates immediately while the server operation completes in the background.
- **Real-Time Subscriptions:** Supabase Realtime provides WebSocket-based subscriptions for live data changes, enabling real-time messaging and signaling for video calls.
- **Remote Procedure Call (RPC) Calls:** Complex operations such as conversation creation are encapsulated in PostgreSQL functions and invoked through Supabase RPC.

2.5.3 Justification of Technical Stack Choices

Selecting the technical stack for a multi-role, real-time consultation marketplace requires evaluating factors such as development velocity, data model complexity, operational security, and runtime performance. The architectural decisions for Istachir are justified as follows:

2.5.3.1 Supabase versus Firebase

Supabase was selected as the Backend-as-a-Service (BaaS) provider over Firebase due to fundamental differences in their database architectures, security models, and ecosystem hosting:

- **Relational Database Structure (PostgreSQL) vs. NoSQL (Firestore):** A consultation platform is highly relational; it involves complex linkages between users, profiles, bookings, services, transactions, and reviews. Firebase’s NoSQL Firestore relies on denormalized document structures, making queries like “find all pending bookings for a specific expert where payment is completed” extremely complex, requiring duplicate data entry and manual indexing. PostgreSQL’s native schema validation, foreign keys, and relational integrity ensure database-level consistency.
- **Row-Level Security (RLS) vs. Firebase Security Rules:** Firebase uses a proprietary JSON-like configuration for security rules, which becomes hard to maintain for complex relational checks. Supabase leverages PostgreSQL’s native Row-Level Security, allowing developers to write access control rules directly in SQL using the user’s JWT details (e.g., verifying if the requesting user’s ID matches the client ID in a booking row). This guarantees that security policies are enforced at the database query engine level.
- **SQL Aggregation and Full-Text Search:** Firebase lacks server-side SQL aggregation and advanced text indexing, requiring third-party search integrations (like Algolia) or client-side filtering. Supabase provides native PostgreSQL full-text search over expert profiles, expertise arrays, and post content out of the box, as well as complex SQL triggers and stored procedures (Remote Procedure Calls).

- **Open-Source and Portability:** Firebase is a proprietary, closed-source Google service that creates strong vendor lock-in. Supabase is open-source, allowing the entire backend infrastructure to be self-hosted on a private Docker-based server if required, reducing external dependencies and hosting costs.

2.5.3.2 React Native vs. Native Development (Swift/Kotlin)

For the customer mobile application, cross-platform development via React Native and Expo was selected over native development for iOS and Android:

- **Single Codebase for Dual Platforms:** Native development requires separate teams writing Swift for iOS and Kotlin for Android, doubling development time and resource overhead. React Native targets both platforms from a single codebase, assuring 100% feature parity.
- **Shared Code and Schema Ecosystem:** Using TypeScript across the entire platform allows the React Native mobile app and the React web dashboard to share type definitions, validation schemas, API hooks (via TanStack React Query), and translation assets, accelerating development and minimizing architectural divergence.
- **Managed Build and Deploy Pipelines (Expo EAS):** Native compilation requires complex manual build, signing, and provisioning setups. Expo Application Services (EAS) abstracts these tasks, providing automated cloud builds and over-the-air (OTA) updates, bypassing long app store approval processes for minor fixes.

2.5.3.3 React.js for the Web Admin Dashboard

The administrative dashboard was implemented using React.js instead of traditional server-side rendering (such as PHP or Django) for several reasons:

- **Single-Page Application (SPA) Performance:** Administrative tasks require managing dynamic tables, charts, and moderation requests. React's virtual DOM allows quick updates without full page reloads, providing a desktop-like user experience.
- **Rich Component Ecosystem:** Admin dashboards are heavily reliant on structured tables, dialog boxes, and interactive forms. Utilizing the shadcn/ui library (based on Radix UI and Tailwind CSS) allowed rapid construction of accessible, responsive layout blocks.

2.6 Real-Time Capabilities

2.6.1 WebRTC for Video Consultations

WebRTC [7] is a free, open-source project that provides real-time communication capabilities for web and mobile applications through simple Application Programming Interface (API)s. The Istachir platform leverages WebRTC for peer-to-peer video consultations between clients and experts.

2.6.1.1 WebRTC Architecture

The WebRTC implementation follows a client-server-signaling architecture:

1. **Signaling:** Supabase Realtime broadcast channels handle signaling message exchange (offer, answer, ICE candidates) between peers. This eliminates the need for a dedicated signaling server.
2. **NAT Traversal:** Google Session Traversal Utilities for NAT (STUN) servers discover public IP addresses and port mappings. Metered Traversal Using Relays around NAT (TURN) servers provide relay fallback when direct peer-to-peer connections fail due to symmetric NAT configurations.
3. **Media Streams:** Local audio and video streams are captured using browser or native device APIs, encoded using codecs like VP8, H.264, or Opus, and transmitted over secure Secure Real-Time Transport Protocol (SRTP) channels.
4. **ICE Framework:** The Interactive Connectivity Establishment framework manages the connection establishment process, attempting direct connections before falling back to relay servers.

2.6.1.2 Multi-Platform WebRTC

The web dashboard uses browser WebRTC APIs while the mobile application uses react-native-webrtc, which provides a JavaScript interface to native WebRTC implementations. Cross-platform consistency is maintained through:

- Abstracted media stream management that handles platform-specific stream tracks
- Unified signaling protocol using JSON messages over Supabase Realtime
- Consistent Interactive Connectivity Establishment (ICE) server configuration across platforms
- Platform-adaptive User Interface (UI) components for camera switching and media controls

2.6.2 Real-Time Messaging

The messaging system uses Supabase Realtime subscriptions to provide instant message delivery:

- Conversations are stored in a PostgreSQL table with participant references.
- Messages are inserted into the database and immediately broadcast to subscribed clients.
- Unread message counts are tracked and displayed in the tab bar and conversation list.
- Typing indicators and read receipts are implemented through real-time presence features.

2.7 Security Frameworks and Authentication

2.7.1 Multi-Layer Security Architecture

The platform implements security at multiple layers:

- **Database Layer:** Supabase RLS policies ensure users can only access their own data. Policies are defined per table and enforce constraints based on user authentication status and role.
- **Application Layer:** Route guards (ProtectedRoute and AdminRoute components) prevent unauthorized access to protected pages. API keys are stored in environment variables and never exposed to the client.
- **Authentication Layer:** JSON Web Token (JWT)-based authentication with automatic token refresh. Session persistence is managed securely through Supabase’s built-in mechanisms.
- **Storage Layer:** Supabase Storage buckets have RLS policies controlling upload and download access. File uploads are validated for type and size restrictions.

2.7.2 Authentication and Authorization Strategies

The platform implements a comprehensive authentication and authorization system:

- **Multi-Method Authentication:** Users can authenticate via email/password or Google OAuth, providing flexibility and reducing registration friction.
- **Role-Based Access Control:** Three primary roles are defined: administrators (full platform access), experts (service management, consultation delivery), and clients (booking, consultation attendance). Roles are stored in user profile metadata and enforced through RLS policies and application-level guards.
- **Session Management:** Supabase manages session lifecycle including automatic token refresh before expiration. Sessions can be terminated server-side for security incidents.
- **Password Security:** Passwords are hashed using bcrypt by Supabase Auth. Password reset flows use time-limited tokens sent via email.

2.8 Artificial Intelligence Integration

2.8.1 Google Gemini AI

Google Gemini [8] is a family of multimodal large language models developed by Google DeepMind. The Istachir platform integrates Gemini through the Google Generative AI SDK for several features:

- **Expert Profile Q&A:** A floating chat widget on expert profiles allows users to ask questions about an expert’s background, services, and reviews. The AI is prompted with the expert’s profile data to provide contextual responses.
- **Session Summary Generation:** After consultations, Gemini generates structured summaries including overview, key insights, action items, resources, and next session focus areas.
- **Post-Session Transcript Q&A:** Users can ask questions about their consultation transcript, enabling them to extract specific information from recorded sessions.

A model fallback chain ensures reliability: gemini-2.5-flash → gemini-2.0-flash → gemini-1.5-flash → gemini-1.5-flash-latest → gemini-pro. This provides resilience against quota limits and service disruptions.

2.8.2 AssemblyAI Platform

AssemblyAI [9] is a leading Voice AI company that provides an API enabling developers to transform any audio file or live stream into structured, actionable data. Built as a complete enterprise infrastructure, its platform powers voice agents, meeting note-taking tools, clinical documentation systems, and large-scale call centers.

2.8.2.1 The Principle of AssemblyAI

AssemblyAI goes far beyond simple transcription. Its main objective is to extract semantic meaning from speech by automatically identifying speaker identities, overall sentiment, named entities, and producing structured summaries. The platform is built on a suite of specialized AI models accessible via a single unified API. This "all-in-one" approach allows developers to submit an audio file and receive a complete analysis (transcription, summary, chapters, sentiment, entities) in under 60 seconds.

2.8.2.2 Core Product Offerings

AssemblyAI provides several solutions to cover different voice processing needs:

- **Speech-to-Text API:** The core transcription engine, supporting 99 languages.
- **Speech Understanding API:** An analysis layer built on top of transcription to extract structured information such as sentiment, entities, topics, and summaries.
- **LLM Gateway:** A gateway that analyzes transcripts using Large Language Models (LLMs) to generate custom summaries, thematic chapters, and action items.
- **Voice Agent API:** A solution for building real-time, low-latency voice agents billed at a flat-rate price.

2.8.2.3 Key Features for Summarization

AssemblyAI includes several features that automate the creation of session summaries:

- **Summarization:** Generates a concise summary of an entire audio file. Developers can choose the model and output format (e.g., paragraph, bullet points).
- **Auto Chapters:** Automatically divides the audio into thematic chapters, each with its own summary.
- **Action Items:** Extracts tasks to be completed directly from the conversation.
- **Speaker Diarization:** Identifies and labels each speaker so the system knows "who said what" (e.g., distinguishing between Expert and Client).

2.8.2.4 How It Works in Practice

Using the API is designed to be simple and fast. A typical workflow for transcribing and summarizing a meeting can be done in just a few lines of Python code using the official SDK, as shown in Figure 2.1.

```

import assemblyai as aai

# 1. Configure the API key and transcription options
aai.settings.api_key = "YOUR_API_KEY"
config = aai.TranscriptionConfig(
    summarization=True,
    summary_model=aai.SummaryModel.informative,
    summary_type=aai.SummaryType.bullets,
    speaker_diarization=True
)

# 2. Transcribe the audio file (local or remote URL)
transcriber = aai.Transcriber()
transcript = transcriber.transcribe("https://example.com/session.mp3", config)

# 3. Access the results
print("Transcript:", transcript.text)
print("Summary:", transcript.summary)

```

Figure 2.1: *Python Implementation of AssemblyAI Summarization*

2.8.2.5 Pricing and Limitations

AssemblyAI operates on a pay-as-you-go model:

- **Free Tier:** Allows developers to test the API with hundreds of hours of transcription and summarization.
- **Pre-recorded Models:** The flagship Universal-3 Pro model is billed at \$0.21/hour of audio.
- **Real-time Models:** Specific pricing exists for streaming, such as Universal-3 Pro Streaming at \$0.45/hour.

However, developers must keep two points in mind. First, the most performant model, Universal-3 Pro, is optimized for 6 main languages (English, Spanish, French, Portuguese, German, Italian); other languages rely on different models. Second, the cost of add-ons (such as diarization and summarization) can increase the base price significantly.

2.8.3 Automated Session Summaries

The principle of automated session summaries relies on applying Artificial Intelligence (AI) and Natural Language Processing (NLP) to transform a raw stream of communication (audio, video, or text) into a structured, concise, and actionable report, without human intervention. This process is not merely a reduction of text, but a semantic reorganization of information.

2.8.3.1 Key Phases of the Summarization Process

The automation process is structured into four main phases:

1. **Capture and Transcription Phase (Listening):** The system captures the audio stream in real-time or from a recording. Using Automatic Speech Recognition (ASR) engines, it converts speech into raw text. A crucial technique used here is Diarization, which identifies "who said what" (e.g., distinguishing speakers A, B, and C).
2. **Cleaning and Preliminary Structuring (Sorting):** The raw transcription is often full of fillers ("uhms," "you know," silences, overlaps). The AI performs an initial cleanup. Simultaneously, it divides the session into thematic segments (e.g., "Introduction," "Problem Presentation," "Budget Discussion," "Conclusion") by analyzing topic shifts.
3. **Extraction and Generative Synthesis (Brain):** The cleaned transcription is fed into a Large Language Model (LLM) with a specific system prompt. The prompt instructs the AI to extract structured blocks of information, typically: a global summary (3 to 5 sentences), key decisions, blocking points/risks, and action items (To-Do List with assigned owners and deadlines).
4. **Contextualization (Business Intelligence):** Modern systems employ Retrieval-Augmented Generation (RAG). Before generating the final summary, the AI searches an internal database (CRM, past tickets, knowledge base) to enrich the output (e.g., "The client mentions problem X, which is similar to the one encountered last March").

2.8.3.2 Technical Viability and Context Windows

This principle works effectively today thanks to the ability of modern LLMs to handle very long context windows (up to millions of tokens). The AI can keep an entire 2-hour meeting in memory and understand the dialectical progression: it knows that what was said in the 5th minute might be contradicted or confirmed in the 55th minute, and it adjusts the final summary accordingly to maintain overall coherence.

2.8.3.3 Strategic Advantages

Implementing automated session summaries offers significant advantages:

- **Freeing Up Attention:** Participants no longer need to take notes during the meeting; they can focus 100% on the active exchange.
- **Objectivity:** The AI does not suffer from fatigue, memory biases, or emotional filters, providing a neutral record of the session.
- **Instant Traceability:** The structured summary is available just seconds after the session ends, accelerating follow-up actions.

2.8.3.4 Limitations and Precautions

Despite these advantages, several limitations must be addressed:

- **Hallucinations:** The LLM may invent a decision that was never made. Therefore, the workflow requires a human validation round (quick proofreading) before finalizing the summary.
- **Loss of Non-Verbal Cues:** Current systems do not capture ironic intonations, hesitant silences, or body language, which are critical in professional consultations (e.g., psychology or negotiation).

- **Confidentiality and Security:** Audio data and summaries are highly sensitive. The implementation must ensure secure cloud storage, anonymization of personal data, and compliance with data protection regulations (e.g., GDPR).

2.9 System Architecture Overview

The Istachir platform follows a layered architecture pattern with clear separation of concerns across presentation, application, data, and integration layers. Figure 2.2 illustrates the complete system architecture.

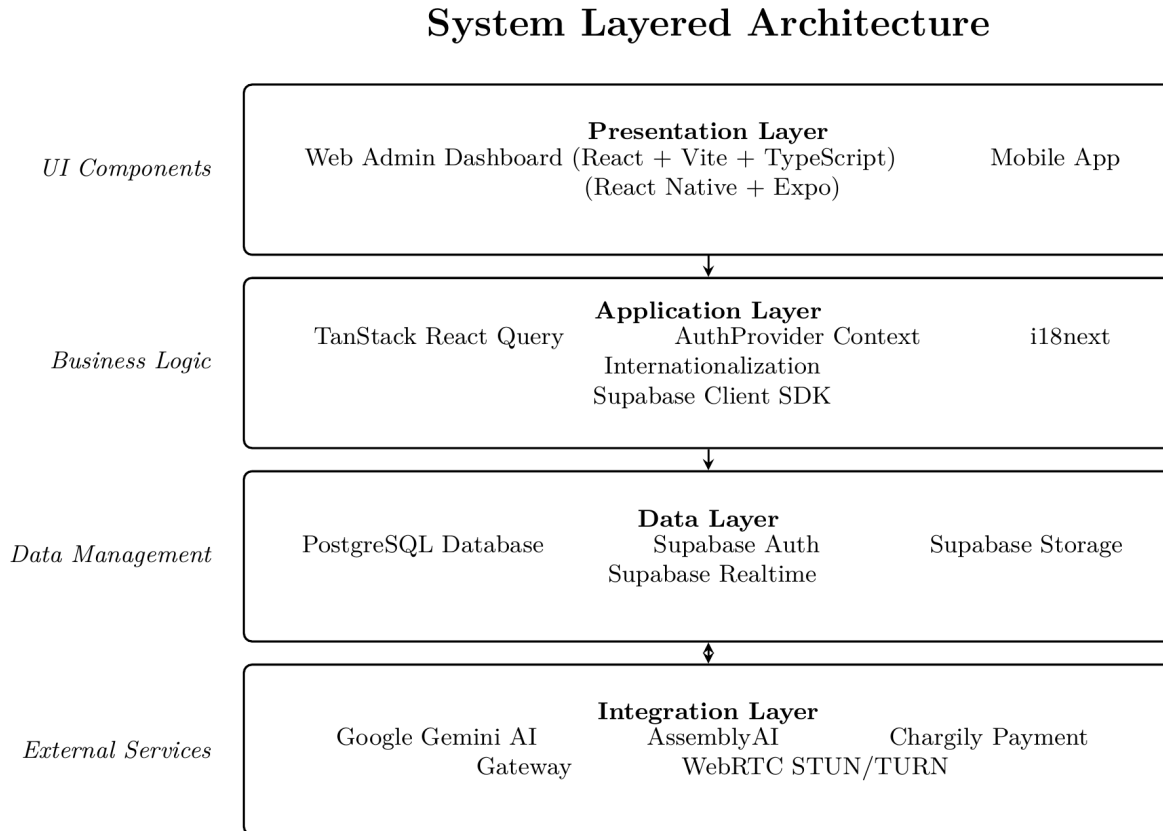


Figure 2.2: *System Layered Architecture*

2.10 Conclusion

This chapter has established the theoretical and technical foundations underlying the Istachir platform. We have examined the consultation marketplace business model, explored the frontend technologies including React.js and React Native, analyzed the Supabase backend platform, and reviewed real-time communication protocols, security frameworks, and AI integration capabilities. These foundations inform the architectural decisions and implementation strategies presented in subsequent chapters.

Part III

Requirements Analysis

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Introduction

This chapter presents a comprehensive analysis of the requirements for the Istachir consultation marketplace platform. We systematically identify and document functional requirements, non-functional requirements, system actors, and their interactions through use case, activity, and sequence diagrams. This analysis serves as the foundation for the system design and implementation phases.

3.2 Functional Requirements Specification

3.2.1 User Management and Authentication

Table 3.1 shows the user Management Functional Requirements.

Table 3.1: *User Management Functional Requirements*

ID	Requirement	Priority
FR01	The system shall allow users to register using email and password	High
FR02	The system shall support Google OAuth authentication	High
FR03	The system shall allow users to reset their password via email	Medium
FR04	The system shall support user profile management (name, photo, bio, title)	High
FR05	The system shall maintain session persistence across app restarts	High
FR06	The system shall implement role-based access control (admin, expert, client)	High
FR07	The system shall allow administrators to manage user roles	Medium

3.2.2 Expert and Service Management

Table 3.2 shows the expert Management Functional Requirements.

Table 3.2: *Expert Management Functional Requirements*

ID	Requirement	Priority
FR08	The system shall allow experts to manage their professional profile	High
FR09	The system shall enable experts to create, edit, and delete consultation services	High
FR10	The system shall allow experts to manage their certifications	Medium
FR11	The system shall display expert ratings and reviews	High
FR12	The system shall enable AI-powered profile Q&A for expert discovery	Medium
FR13	The system shall provide expert search by name, title, or expertise	High

3.2.3 Booking and Consultation Requirements

Table 3.3 shows the booking Functional Requirements.

Table 3.3: *Booking Functional Requirements*

ID	Requirement	Priority
FR14	The system shall allow clients to browse available consultation services	High
FR15	The system shall enable clients to schedule consultations using a calendar	High
FR16	The system shall support real-time video consultations via WebRTC	High
FR17	The system shall record consultation sessions for later review	Medium
FR18	The system shall generate AI-powered session summaries	Medium
FR19	The system shall provide post-session transcript viewing and Q&A	Low
FR20	The system shall allow clients to rate and review experts after sessions	High

3.2.4 Payment and Escrow Requirements

Table 3.4 shows the payment Functional Requirements.

Table 3.4: *Payment Functional Requirements*

ID	Requirement	Priority
FR21	The system shall integrate with the Chargily payment gateway	High
FR22	The system shall implement escrow-based payment processing	High
FR23	The system shall support payments in Algerian Dinar (DZD)	High
FR24	The system shall allow clients to release escrow funds after sessions	High
FR25	The system shall support dispute resolution for escrow transactions	Medium
FR26	The system shall allow experts to manage CCP/RIP payout accounts	Medium
FR27	The system shall display transaction history with filtering capabilities	Medium

3.2.5 Content and Communication Requirements

Table 3.5 shows the content and Communication Functional Requirements.

Table 3.5: *Content and Communication Functional Requirements*

ID	Requirement	Priority
FR28	The system shall support real-time messaging between clients and experts	High
FR29	The system shall allow image sharing in chat conversations	Medium
FR30	The system shall support a blog/posts system with likes and comments	Medium
FR31	The system shall allow reporting inappropriate content	Medium
FR32	The system shall support English and Arabic language interfaces	High
FR33	The system shall provide RTL layout support for Arabic	High

3.3 Non-Functional Requirements Specification

3.3.1 Usability Requirements

- **NR01 - Responsive Design:** The web dashboard shall be responsive and functional across desktop and tablet screen sizes. The mobile application shall provide an intuitive touch-optimized interface.
- **NR02 - Multi-Language Support:** The platform shall support seamless switching between English and Arabic without application restart.
- **NR03 - Accessibility:** The user interface shall follow accessibility best practices including proper contrast ratios, keyboard navigation, and screen reader support.
- **NR04 - Onboarding:** New users shall be guided through the platform's features with clear navigation and contextual help.

3.3.2 Security Requirements

- **NR05 - Authentication Security:** All authentication flows shall use secure protocols (HTTPS, JWT with automatic refresh). Passwords shall be hashed and never stored in plain text.
- **NR06 - Data Protection:** User data shall be protected through database-level row-level security policies. File uploads shall be validated for type and size.
- **NR07 - Payment Security:** Payment transactions shall be processed through PCI-compliant gateways. Escrow funds shall be protected through contractual and technical mechanisms.
- **NR08 - Session Security:** Video consultations shall be encrypted end-to-end where possible. Session recordings shall be stored with access control restrictions.

3.3.3 Performance Requirements

- **NR09 - Response Time:** API responses shall complete within 500ms for standard operations. Page loads shall complete within 2 seconds on standard broadband connections.
- **NR10 - Video Quality:** Video consultations shall maintain minimum 480p resolution with adaptive bitrate based on network conditions.
- **NR11 - Scalability:** The platform architecture shall support horizontal scaling to accommodate growing user numbers without significant performance degradation.
- **NR12 - Availability:** The platform shall target 99.9% availability during business hours, with planned maintenance windows for updates.

3.4 Identification of Actors

The Istachir platform involves three primary actor categories:

3.4.1 Administrators

Administrators have comprehensive access to platform management features. They are responsible for:

- User management including role assignment and account moderation.
- Content moderation including post, review, and report management.
- Financial oversight including transaction monitoring and payout management.
- Platform analytics and reporting.
- System configuration and policy enforcement.

3.4.2 Experts

Experts are qualified professionals who provide consultation services. Their capabilities include:

- Profile management including professional information and credentials.
- Service creation and pricing management.
- Certification documentation and verification.
- Consultation delivery through video calls.
- Financial management including payout account configuration.
- Content creation through posts and articles.

3.4.3 Clients

Clients are users seeking expert advice and consultation services. Their capabilities include:

- Expert discovery through search and browsing.
- Consultation booking and payment.
- Video call participation and session management.
- Post-session review and rating.
- Messaging with experts.
- Financial transaction history review.

3.5 Use Case Diagrams

To model the functional requirements of the Istachir platform, we divide the system into four primary subsystems: User Management, Consultation & Session, Payment & Escrow, and Content & Communication. The UML use case diagrams for these subsystems are presented in Figure 3.1, Figure 3.2, Figure 3.3, and Figure 3.4.

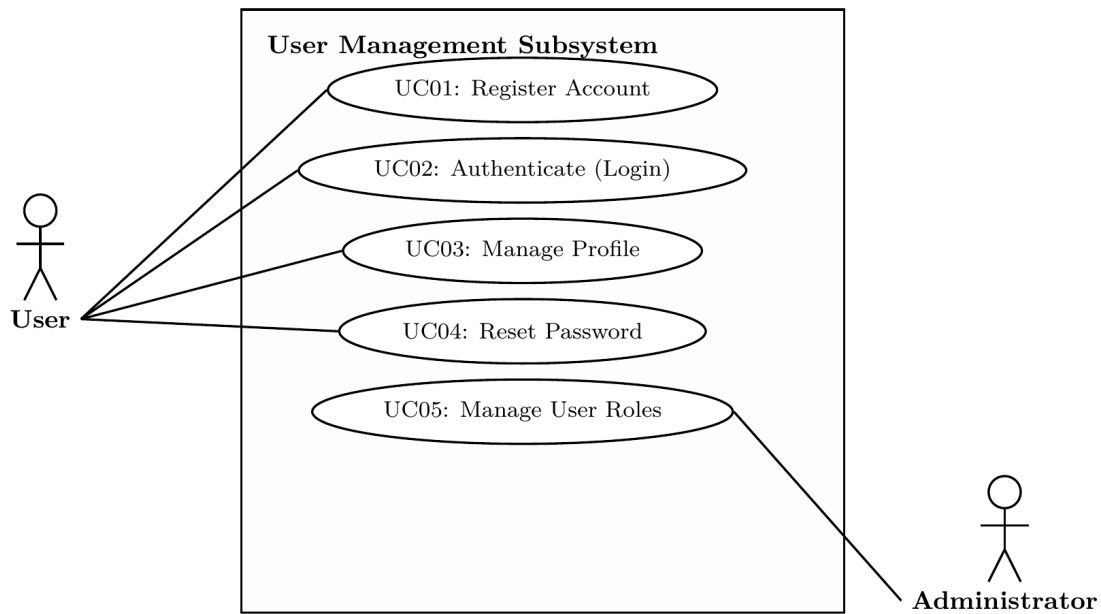


Figure 3.1: *User Management Subsystem Use Case Diagram*

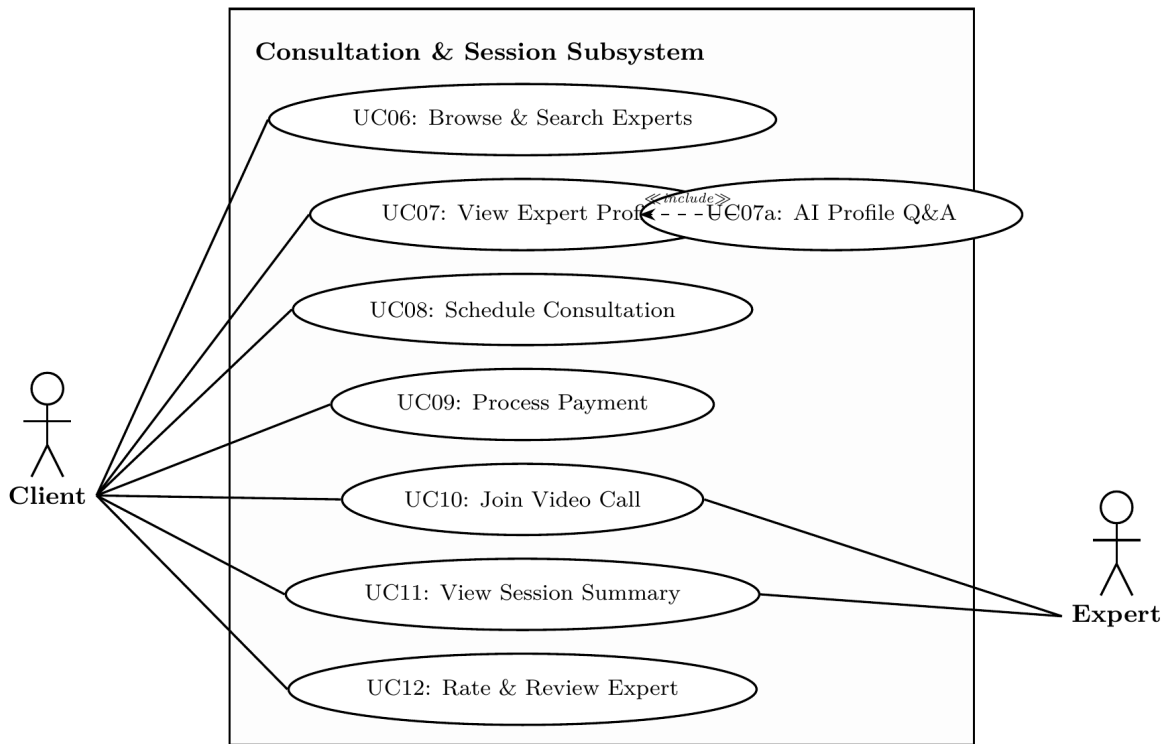


Figure 3.2: *Consultation & Session Subsystem Use Case Diagram*

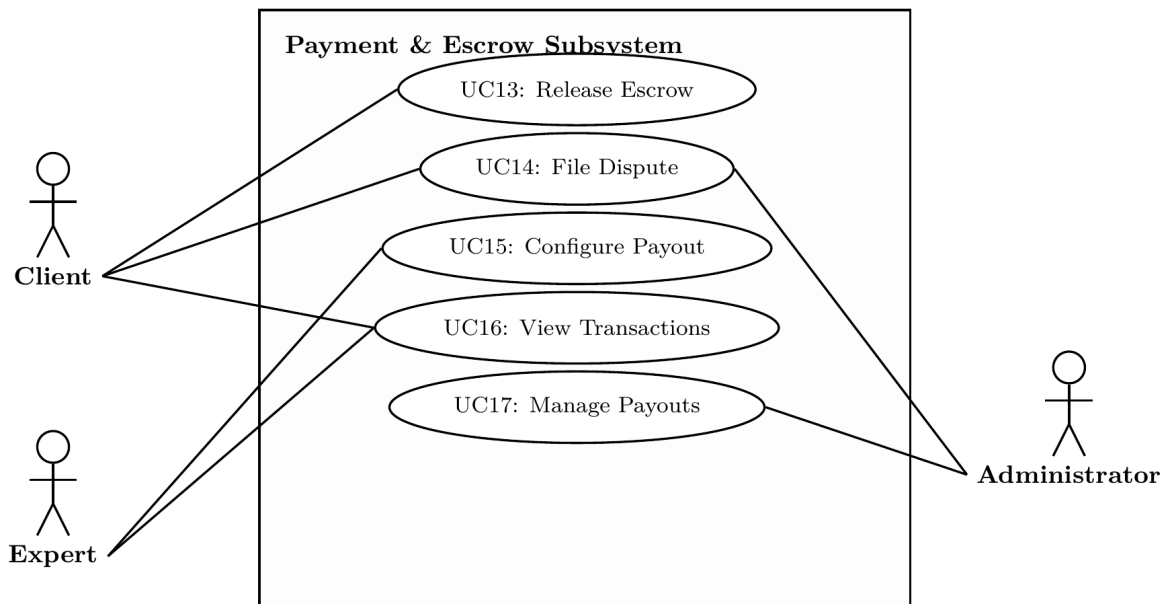


Figure 3.3: *Payment & Escrow Subsystem Use Case Diagram*

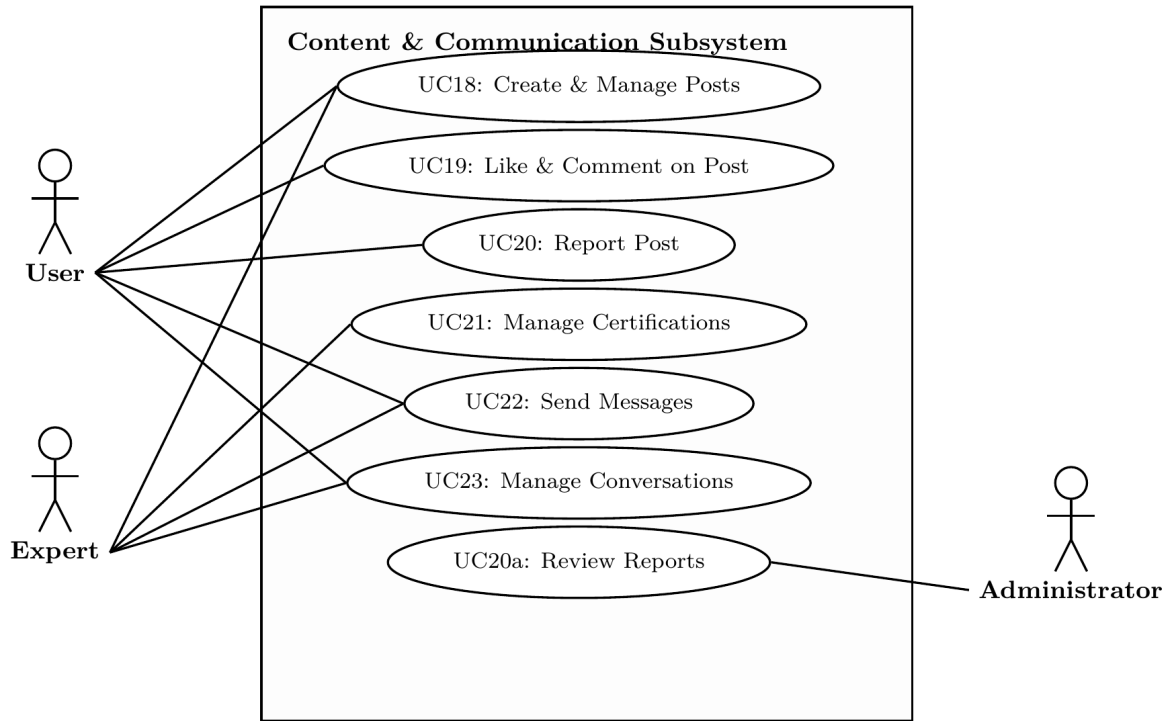


Figure 3.4: *Content & Communication Subsystem Use Case Diagram*

3.5.1 User Management Use Cases

The user management subsystem encompasses registration, authentication, profile management, and administrative user control. Key use cases include:

- **UC01 - User Registration:** New users create accounts using email/password or Google OAuth. The system validates input, creates authentication records, and initializes user profiles.
- **UC02 - User Authentication:** Registered users authenticate using their chosen method. The system validates credentials and establishes a secure session.
- **UC03 - Profile Management:** Users update their profile information including name, photo, professional details, and preferences.
- **UC04 - Password Reset:** Users request password reset via email. The system sends a time-limited reset link and processes the new password.
- **UC05 - Role Management (Admin):** Administrators view user roles and can modify them as needed for platform management.

3.5.2 Consultation Booking Use Cases

The consultation workflow encompasses browsing, scheduling, attending, and reviewing consultation sessions:

- **UC06 - Browse Experts:** Clients search and filter experts by name, expertise, rating, or other criteria. Results display expert cards with key information.
- **UC07 - View Expert Profile:** Clients access detailed expert profiles including biography, services, certifications, and reviews. AI-powered Q&A provides additional insights.
- **UC08 - Schedule Consultation:** Clients select a service, choose an available date from the calendar, and proceed to payment.
- **UC09 - Process Payment:** The system creates a Chargily checkout session and processes payment confirmation through webhook callbacks.
- **UC10 - Join Video Call:** Both parties join the WebRTC video consultation room at the scheduled time.
- **UC11 - View Session Summary:** After consultations, users access AI-generated summaries and session recordings.
- **UC12 - Rate and Review:** Clients submit ratings and written reviews for completed consultations.

3.5.3 Payment Management Use Cases

The financial subsystem handles payment processing, escrow management, and payout operations:

- **UC13 - Release Escrow:** Clients confirm satisfaction and release held funds to the expert.
- **UC14 - File Dispute:** Clients initiate dispute proceedings for unsatisfactory consultations.
- **UC15 - Configure Payout Account:** Experts configure their CCP/RIP payout account details.
- **UC16 - View Transaction History:** Users view their financial transactions with filtering by date range and status.
- **UC17 - Manage Payouts (Admin):** Administrators review and process expert payout requests.

3.5.4 Content and Communication Use Cases

The content and communication subsystem supports social engagement, direct messaging, and expert credential verification:

- **UC18 - Create and Manage Posts:** Users create, edit, and delete blog posts with rich text content, images, tags, and media attachments.
- **UC19 - Like and Comment on Posts:** Users interact with published posts through likes and comments, fostering community engagement.
- **UC20 - Report Post:** Users report inappropriate or policy-violating content for administrator review.

- **UC21 - Manage Certifications:** Experts upload, update, and manage their professional certifications and credentials displayed on their profile.
- **UC22 - Send Messages:** Users send text and media messages within conversations with real-time delivery via Supabase Realtime.
- **UC23 - Manage Conversations:** Users initiate, browse, and manage direct message conversations with other platform users.

3.5.5 Detailed Use Case Descriptions

To clarify the precise logic and interaction patterns of all system functions, detailed use case descriptions are presented in Table 3.6 through Table 3.28.

Table 3.6: *Detailed Description of UC01: User Registration*

Field	Description
Use Case ID	UC01
Use Case Name	User Registration
Primary Actor	Visitor (New User)
Preconditions	Visitor has access to the internet and has opened the application.
Postconditions	A new user authentication record is created in Supabase Auth, and a matching profile is initialized in the database.
Trigger	Visitor clicks the "Sign Up" button on the welcome or login screen.
Main Flow	1. Visitor selects their primary role (Client or Expert). 2. Visitor enters registration credentials: full name, email address, password. 3. Visitor submits the registration form. 4. System validates the input parameters (checks email format, verifies password is ≥ 6 characters). 5. System invokes Supabase Auth API to register the credentials. 6. System initializes profile record in database schema.
Alternative Flows	2a. Google OAuth: Visitor clicks "Sign Up with Google", system authenticates via Google OAuth API, retrieves details, and creates database profile. 4a. Invalid inputs: System highlights the faulty fields with specific validation error messages. 5a. Email already registered: System displays alert "Email is already taken".

Table 3.7 shows the detailed Description of UC02: User Authentication.

Table 3.7: *Detailed Description of UC02: User Authentication*

Field	Description
Use Case ID	UC02
Use Case Name	User Authentication
Primary Actor	Registered User (Client, Expert, or Admin)
Preconditions	User has a registered account.
Postconditions	User session JWT token is established in the client device, and the user is redirected to their specific dashboard.
Trigger	User opens the application and inputs login credentials.
Main Flow	1. User enters their email and password. 2. User clicks the "Sign In" button. 3. System sends authentication payload to Supabase Auth API. 4. Supabase validates details and returns a secure JWT session token. 5. System updates local session context. 6. System checks user role and redirects user to their appropriate dashboard screen.
Alternative Flows	1a. Google OAuth: User clicks "Login with Google", system handles authentication via Google OAuth API. 3a. Invalid credentials: System displays error message "Invalid email or password".

Table 3.8 shows the detailed Description of UC03: Profile Management.

Table 3.8: *Detailed Description of UC03: Profile Management*

Field	Description
Use Case ID	UC03
Use Case Name	Profile Management
Primary Actor	Client or Expert
Preconditions	User is authenticated.
Postconditions	Updated profile parameters are stored in the database.
Trigger	User navigates to Settings and clicks "Edit Profile".
Main Flow	1. User requests edit profile page. 2. System retrieves current profile attributes from database. 3. User modifies details (e.g. name, biography, tags, hourly rate for experts). 4. User clicks the "Save Changes" button. 5. System validates the updated data formats. 6. System updates the corresponding record in the database. 7. System displays a success alert.
Alternative Flows	3a. Avatar update: User uploads a image, system puts it in the Supabase Storage bucket, fetches public URL, and updates the database record.

Table 3.9 shows the detailed Description of UC04: Password Reset.

Table 3.9: *Detailed Description of UC04: Password Reset*

Field	Description
Use Case ID	UC04
Use Case Name	Password Reset
Primary Actor	User
Preconditions	User has a registered account.
Postconditions	Password reset link is sent to user email, and user successfully updates password.
Trigger	User clicks "Forgot Password" on the login screen.
Main Flow	1. User enters their registered email address. 2. User clicks "Send Reset Link" button. 3. System invokes Supabase Auth API to send a password reset mail. 4. User receives mail and clicks the secure reset link. 5. User is redirected to password update page inside the platform. 6. User enters and confirms the new password. 7. System validates and updates credentials.
Alternative Flows	1a. Email not found: System displays general message "If the email exists, a link will be sent" to protect database privacy.

Table 3.10 shows the detailed Description of UC05: Role Management (Admin).

Table 3.10: *Detailed Description of UC05: Role Management (Admin)*

Field	Description
Use Case ID	UC05
Use Case Name	Role Management
Primary Actor	Administrator
Preconditions	Administrator is authenticated.
Postconditions	Targeted user role is updated in database metadata.
Trigger	Admin decides to modify user role or verify credentials.
Main Flow	1. Admin opens the administrative web panel and navigates to User Management. 2. Admin selects a user profile from list. 3. Admin selects the new role category (Client, Expert, or Admin). 4. Admin clicks "Update Role" button. 5. System updates database user records. 6. System audits the update.
Alternative Flows	4a. Admin revokes expert verification: System updates status to "unverified", masking expert from client search results.

Table 3.11 shows the detailed Description of UC06: Browse Experts.

Table 3.11: *Detailed Description of UC06: Browse Experts*

Field	Description
Use Case ID	UC06
Use Case Name	Browse Experts
Primary Actor	Client
Preconditions	Client is authenticated.
Postconditions	A list of verified experts matching search criteria is rendered.
Trigger	Client opens the search and discovery tab.
Main Flow	1. Client accesses the search screen. 2. System fetches all verified experts by default. 3. Client inputs keyword query or filters by price range, rating, or expertise tags. 4. System executes query with filters on the database. 5. System displays matching expert cards.
Alternative Flows	4a. No matching experts: System displays message "No experts match your search".

Table 3.12 shows the detailed Description of UC07: View Expert Profile.

Table 3.12: *Detailed Description of UC07: View Expert Profile*

Field	Description
Use Case ID	UC07
Use Case Name	View Expert Profile
Primary Actor	Client
Preconditions	Client is authenticated.
Postconditions	Detailed expert profile data and services are rendered, and the profile Q&A chatbot is initialized.
Trigger	Client clicks on an expert's card in search results.
Main Flow	1. Client selects an expert card. 2. System fetches expert details (bio, credentials, ratings, reviews, services list). 3. System loads and initializes profile QA Chatbot context using Gemini model and expert profile text. 4. Client reviews details and can chat with the AI widget to ask questions about the expert.
Alternative Flows	3a. Chatbot initialization fails: Chatbot displays offline message, but profile details remain accessible.

Table 3.13 shows the detailed Description of UC08: Schedule Consultation.

Table 3.13: *Detailed Description of UC08: Schedule Consultation*

Field	Description
Use Case ID	UC08
Use Case Name	Schedule Consultation
Primary Actor	Client
Preconditions	Client is authenticated, and has found an expert profile with active services.
Postconditions	A pending booking is registered in the database, and the user is redirected to payment.
Trigger	Client clicks the "Book Session" button on the expert's service card.
Main Flow	1. Client selects a specific service (hourly rate, duration). 2. System displays a 30-day calendar indicating the expert's availability. 3. Client selects an open date/time slot. 4. System prompts the client to confirm a booking summary page showing fees. 5. Client clicks "Proceed to Payment". 6. System creates a pending booking record in Supabase and redirects to Chargily.
Alternative Flows	3a. Selected date/time slot becomes unavailable: System alerts client to select another slot. 5a. Client cancels scheduling: System returns client to the expert profile view.

Table 3.14 shows the detailed Description of UC09: Process Payment.

Table 3.14: *Detailed Description of UC09: Process Payment*

Field	Description
Use Case ID	UC09
Use Case Name	Process Payment
Primary Actor	Client
Preconditions	A pending booking record exists in the database.
Postconditions	Payment is processed, transaction is recorded, booking is marked as "confirmed", and funds are locked in escrow.
Trigger	Client clicks "Confirm and Pay" on booking checkout page.
Main Flow	1. System initializes Chargily checkout session and redirects user to Chargily gateway. 2. Client enters payment card details (Edahabia or CIB). 3. Card processor validates and processes transaction. 4. Chargily redirects client back to confirmation page. 5. Chargily webhook sends event to Supabase endpoint. 6. Webhook handler verifies event signature, updates booking status to "confirmed", and records payment transaction.
Alternative Flows	3a. Transaction declined: Client is redirected back with "failed" status. Booking remains "pending_payment". 5a. Webhook latency: Client redirect screen queries database until status updates.

Table 3.15 shows the detailed Description of UC10: Join Video Call.

Table 3.15: *Detailed Description of UC10: Join Video Call*

Field	Description
Use Case ID	UC10
Use Case Name	Join Video Call
Primary Actor	Client and Expert
Preconditions	A booking status is "confirmed" and the current time matches the scheduled slot.
Postconditions	A real-time peer-to-peer audio/video connection is established, and local recording begins.
Trigger	Both participants click the "Join Call" button in the booking details screen.
Main Flow	1. System checks camera and microphone permissions on participants' devices. 2. System initializes WebRTC peer connection object and local media streams. 3. Offerer client sends an ICE offer via Supabase Realtime broadcast channel. 4. Expert receiver gets the offer and responds with an ICE answer. 5. ICE candidates are exchanged and direct media connection is established. 6. System displays the remote video stream and starts a call timer. 7. Audio recording is initiated locally and buffered for cloud upload.
Alternative Flows	5a. Direct connection fails due to firewall/NAT: WebRTC falls back to TURN relay server. 7a. Session ends: Participant hangs up, system triggers local audio upload to Supabase storage.

Table 3.16 shows the detailed Description of UC11: View Session Summary.

Table 3.16: *Detailed Description of UC11: View Session Summary*

Field	Description
Use Case ID	UC11
Use Case Name	View Session Summary
Primary Actor	Client and Expert
Preconditions	The video session is completed, and the audio recording is successfully uploaded.
Postconditions	Users view AI-generated summary, action items, and can interact with the transcript Q&A bot.
Trigger	Client or Expert navigates to the completed appointment detail screen.
Main Flow	1. System triggers AssemblyAI API to transcribe the uploaded session recording. 2. Once transcription is complete, system calls Google Gemini API with the transcript. 3. Gemini generates a structured summary (Overview, key insights, action items). 4. Summary and transcript are stored in the database under the booking record. 5. User enters the session page and views the structured summary and transcription. 6. User can type questions in the "Ask AI" widget to query the session transcript.
Alternative Flows	1a. Transcription fails: System retries up to three times, then reports a fallback text. 2a. Gemini model rate limit: System falls back to the next model in the redundancy chain.

Table 3.17 shows the detailed Description of UC12: Rate and Review.

Table 3.17: *Detailed Description of UC12: Rate and Review*

Field	Description
Use Case ID	UC12
Use Case Name	Rate and Review
Primary Actor	Client
Preconditions	Session booking status is "completed".
Postconditions	Review is added to the database, and expert aggregate rating is updated.
Trigger	Client clicks "Rate Expert" on the session completion screen.
Main Flow	1. Client selects a rating score (1-5 stars). 2. Client writes optional comments in the text area. 3. Client clicks "Submit Review" button. 4. System validates inputs. 5. System inserts the review record. 6. Database trigger automatically recalculates and updates the expert average rating.
Alternative Flows	3a. Empty rating: System displays validation prompt.

Table 3.18 shows the detailed Description of UC13: Release Escrow.

Table 3.18: *Detailed Description of UC13: Release Escrow*

Field	Description
Use Case ID	UC13
Use Case Name	Release Escrow
Primary Actor	Client or Administrator
Preconditions	Session booking status is "completed" and payment status is "held_in_escrow".
Postconditions	Funds are released, booking payment status is updated, and funds are added to expert ledger balance.
Trigger	Client clicks "Confirm and Release Escrow" or automatic threshold timer expires.
Main Flow	1. Client confirms session was satisfactory and clicks "Release Payment". 2. System starts database transaction. 3. System updates booking payment status to "released". 4. System updates expert ledger balance with the session amount minus commission. 5. System logs financial payout transaction event.
Alternative Flows	1a. Auto-release: 48 hours post-session, if no dispute is opened, system cron-job automatically releases escrow funds.

Table 3.19 shows the detailed Description of UC14: File Dispute.

Table 3.19: *Detailed Description of UC14: File Dispute*

Field	Description
Use Case ID	UC14
Use Case Name	File Dispute
Primary Actor	Client or Expert
Preconditions	Session booking status is "completed" and payment status is "held_in_escrow".
Postconditions	Booking status is set to "disputed", escrow release is locked, and administrative audit is requested.
Trigger	User clicks "File Dispute" on session detail screen.
Main Flow	1. User clicks "File Dispute". 2. System displays dispute form. 3. User enters reasons and uploads supporting text or screenshots. 4. User submits the dispute. 5. System sets booking status to "disputed" and locks the escrow. 6. System generates admin notification task.
Alternative Flows	4a. Empty reasons: System prompts user to fill text areas.

Table 3.20 shows the detailed Description of UC15: Configure Payout Account.

Table 3.20: *Detailed Description of UC15: Configure Payout Account*

Field	Description
Use Case ID	UC15
Use Case Name	Configure Payout Account
Primary Actor	Expert
Preconditions	Expert is authenticated.
Postconditions	RIP/CCP payout credentials are verified and saved.
Trigger	Expert clicks "Configure Payout Details" in wallet settings.
Main Flow	1. Expert navigates to Wallet settings. 2. Expert enters full name, CCP/RIP number. 3. Expert clicks "Save Payout Details". 4. System validates RIP/CCP format. 5. System writes details to expert record. 6. System displays verification confirmation message.
Alternative Flows	4a. Invalid RIB length: System alerts user "RIB must be 20 digits".

Table 3.21 shows the detailed Description of UC16: View Transaction History.

Table 3.21: *Detailed Description of UC16: View Transaction History*

Field	Description
Use Case ID	UC16
Use Case Name	View Transaction History
Primary Actor	Client or Expert
Preconditions	User is authenticated.
Postconditions	Financial history log containing date, status, and amounts is displayed.
Trigger	User navigates to their profile wallet tab.
Main Flow	1. User clicks the Wallet button. 2. System queries database transaction table for rows matching user ID. 3. System displays lists of credit/debit logs. 4. User can inspect payment details, dates, invoice numbers.
Alternative Flows	2a. Offline mode: System displays cached wallet transactions.

Table 3.22 shows the detailed Description of UC17: Manage Payouts (Admin).

Table 3.22: *Detailed Description of UC17: Manage Payouts (Admin)*

Field	Description
Use Case ID	UC17
Use Case Name	Manage Payouts
Primary Actor	Administrator
Preconditions	Administrator is authenticated.
Postconditions	Payout is completed, and ledger balances are settled.
Trigger	Admin opens administrative payouts panel.
Main Flow	1. Admin reviews expert payout request list. 2. Admin processes bank bank transfer manually. 3. Admin clicks "Mark as Paid" on corresponding request. 4. System updates request status to "completed". 5. System deducts processed amount from expert platform ledger balance. 6. System logs payout audit event.
Alternative Flows	3a. Payout rejected: Admin inputs reason, system returns funds to expert balance, and notifies expert.

Table 3.23 shows the detailed Description of UC18: Create and Manage Posts.

Table 3.23: *Detailed Description of UC18: Create and Manage Posts*

Field	Description
Use Case ID	UC18
Use Case Name	Create and Manage Posts
Primary Actor	User (Expert or Admin)
Preconditions	User is authenticated.
Postconditions	Post is published and visible to all platform users.
Trigger	User clicks "Create Post" button.
Main Flow	1. User navigates to posts section. 2. User fills in title, content, excerpt, tag, and optional media. 3. System validates input fields (title and content are required). 4. System stores post in the <code>posts</code> table with <code>author_id</code>. 5. Post appears in the community feed.
Alternative Flows	2a. User edits an existing post: system updates the <code>updated_at</code> timestamp. 2b. User deletes a post: system removes the post and cascades deletion to related likes, comments, and reports.

Table 3.24 shows the detailed Description of UC19: Like and Comment on Posts.

Table 3.24: *Detailed Description of UC19: Like and Comment on Posts*

Field	Description
Use Case ID	UC19
Use Case Name	Like and Comment on Posts
Primary Actor	Authenticated User
Preconditions	User is authenticated and a post exists.
Postconditions	Like or comment is recorded in the database.
Trigger	User clicks the like button or submits a comment.
Main Flow	1. User views a post in the community feed. 2a. Like: User clicks heart icon; system inserts a record into <code>post_likes</code> and increments the post's <code>likes</code> counter. 2b. Comment: User types comment text and submits; system inserts a record into <code>post_comments</code> and increments <code>comments_count</code>. 3. UI updates in real-time to reflect the new interaction.
Alternative Flows	2a-alt. User unlikes: system deletes the <code>post_likes</code> record and decrements the counter. 2b-alt. User deletes their own comment: system removes the <code>post_comments</code> record.

Table 3.25 shows the detailed Description of UC20: Report Post.

Table 3.25: *Detailed Description of UC20: Report Post*

Field	Description
Use Case ID	UC20
Use Case Name	Report Post
Primary Actor	Authenticated User
Secondary Actor	Administrator
Preconditions	User is authenticated and a post exists.
Postconditions	Report is stored with status "pending" for admin review.
Trigger	User clicks "Report" on a post.
Main Flow	1. User selects a reason category for the report. 2. User optionally adds details text. 3. System creates a <code>post_reports</code> record with status "pending". 4. Administrator reviews the report in the admin panel. 5. Admin sets status to "reviewed", "dismissed", or "action_taken". 6. If action taken, admin may delete or hide the offending post.
Alternative Flows	4a. Report is dismissed: admin marks as "dismissed", no action on post.

Table 3.26 shows the detailed Description of UC21: Manage Certifications.

Table 3.26: *Detailed Description of UC21: Manage Certifications*

Field	Description
Use Case ID	UC21
Use Case Name	Manage Certifications
Primary Actor	Expert
Preconditions	Expert is authenticated with role "admin" or is the profile owner.
Postconditions	Certification record is created, updated, or deleted in the <code>certifications</code> table.
Trigger	Expert navigates to profile settings and clicks "Add Certification".
Main Flow	1. Expert opens the certifications management section. 2. Expert fills in certification name, issuer, issue date, expiry date, and verification URL. 3. System validates required fields (name, issuer). 4. System stores the certification with the expert's <code>expert_id</code>. 5. Certification appears on the expert's public profile.
Alternative Flows	2a. Expert edits an existing certification: system updates the record. 2b. Expert deletes a certification: system removes the record from the database.

Table 3.27 shows the detailed Description of UC22: Send Messages.

Table 3.27: *Detailed Description of UC22: Send Messages*

Field	Description
Use Case ID	UC22
Use Case Name	Send Messages
Primary Actor	Authenticated User
Preconditions	User is authenticated and a conversation exists between the two participants.
Postconditions	Message is stored in <code>messages</code> table and delivered in real-time.
Trigger	User types and sends a message in the chat interface.
Main Flow	1. User opens an existing conversation. 2. User types text content or attaches a media file. 3. System inserts message into the <code>messages</code> table with <code>conversation_id</code> and <code>sender_id</code>. 4. Supabase Realtime broadcasts the new message to the other participant. 5. System updates <code>last_message_at</code> on the conversation record. 6. Recipient's message list refreshes with the new message (<code>read = false</code>).
Alternative Flows	2a. Media attachment: user uploads image/document via Supabase Storage; <code>media_url</code> is stored with the message. 6a. Recipient opens message: system sets <code>read</code> to true.

Table 3.28: *Detailed Description of UC23: Manage Conversations*

Field	Description
Use Case ID	UC23
Use Case Name	Manage Conversations
Primary Actor	Authenticated User
Preconditions	User is authenticated.
Postconditions	A new or existing conversation is available for messaging.
Trigger	User clicks "Message" on another user's profile or opens the inbox.
Main Flow	1. User clicks "Message" on an expert or user profile. 2. System checks if a conversation already exists between the two participants via PostgreSQL RPC function. 3a. If exists: system opens the existing conversation thread. 3b. If not: system creates a new <code>conversations</code> record with <code>participant_1</code> and <code>participant_2</code>. 4. User views the conversation list sorted by <code>last_message_at</code>. 5. User selects a conversation to view message history.
Alternative Flows	4a. No conversations exist: system shows empty inbox state with prompt to start a conversation.

3.6 Activity Diagrams

3.6.1 Consultation Booking Process

The consultation booking activity begins when a client selects an expert and proceeds through service selection, date scheduling, payment processing, and confirmation. The process involves:

1. Client browses experts and selects a desired professional.
2. Client reviews expert profile, services, and ratings.
3. Client selects a consultation service (duration, price).
4. Client chooses an available date from the 30-day calendar.
5. System displays booking summary with total cost including platform fees.
6. Client proceeds to payment via Chargily gateway.
7. System confirms payment and creates booking record.
8. Both parties receive confirmation notifications.

3.6.2 Escrow Release Process

The escrow release process ensures fair compensation for experts while protecting clients:

1. Consultation session is completed.
2. System notifies client that funds are available for release.

3. Client reviews session quality and may optionally file a dispute.
4. If satisfied, client confirms escrow release.
5. System transfers funds from escrow to expert pending payout.
6. Expert receives notification of funds availability.
7. Platform commission (10%) is deducted and recorded.

3.6.3 Video Call Connection Process

Establishing a WebRTC video call involves signaling and media negotiation:

1. Both participants join the consultation room at scheduled time.
2. System initializes local media streams (audio/video).
3. Booking client (offerer) creates a WebRTC offer.
4. Offer is sent via Supabase Realtime broadcast channel.
5. Expert (answerer) receives offer and creates an answer.
6. ICE candidates are exchanged until connection is established.
7. Media streams flow directly between peers (or via TURN relay).
8. Call duration timer starts when connection is established.

Figure 3.5 illustrates the consultation Booking Process Activity Diagram.

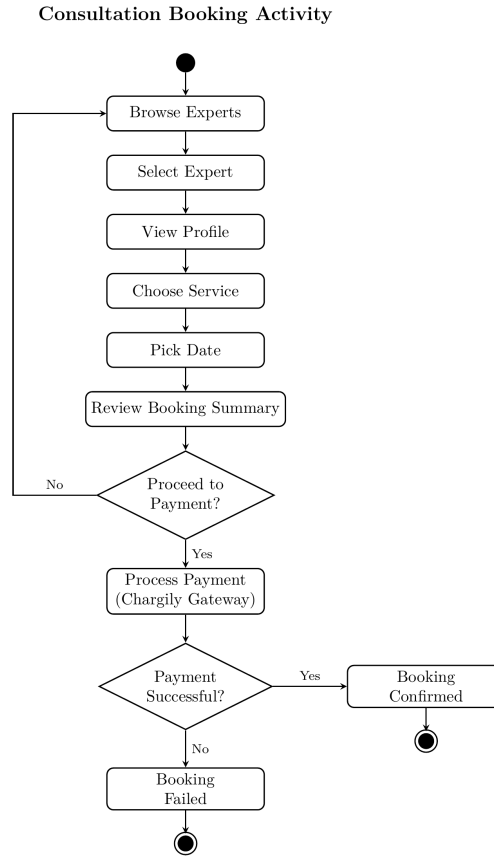


Figure 3.5: *Consultation Booking Process Activity Diagram*

Figure 3.6 illustrates the escrow Release Process Activity Diagram.

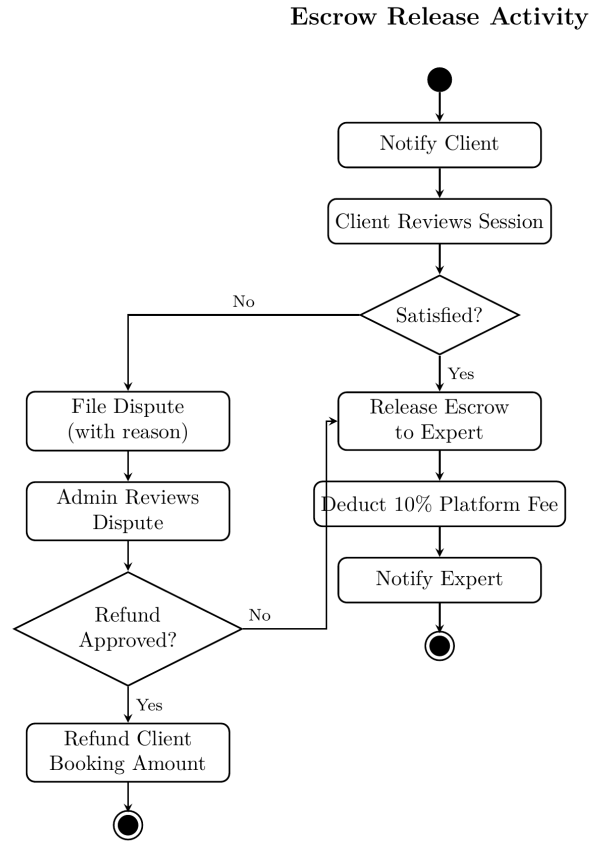


Figure 3.6: *Escrow Release Process Activity Diagram*

3.7 Sequence Diagrams

3.7.1 User Authentication Sequence

The authentication sequence involves the client application, Supabase Auth service, and the user database:

1. User submits credentials (email/password) through the login form.
2. Application sends sign-in request to Supabase Auth.
3. Supabase validates credentials against stored hash.
4. On success, Supabase returns a session object containing access and refresh tokens.
5. Application stores session and redirects to the main interface.
6. AuthProvider context updates to reflect authenticated state.
7. Protected routes become accessible to the user.

3.7.2 Booking Creation Sequence

The booking creation sequence involves multiple system components:

1. Client selects expert service and date.
2. Application creates a pending booking record in the database.
3. Application requests checkout session from Chargily API.
4. Chargily returns checkout URL.
5. Client is redirected to Chargily payment page.
6. After successful payment, Chargily redirects to confirmation URL.
7. Application updates booking status to confirmed.
8. Both parties receive real-time notification of the confirmed booking.

3.7.3 Messaging Sequence

Real-time messaging follows a publish-subscribe pattern:

1. Sender composes and sends a message.
2. Application inserts message record into the database.
3. Supabase Realtime broadcasts the new message to all subscribers of the conversation channel.
4. Recipient's application receives the broadcast and updates the UI.
5. Message is marked as read when recipient opens the conversation.
6. Unread message counter is updated for both parties.

Figure 3.7 illustrates the user Authentication Sequence Diagram.

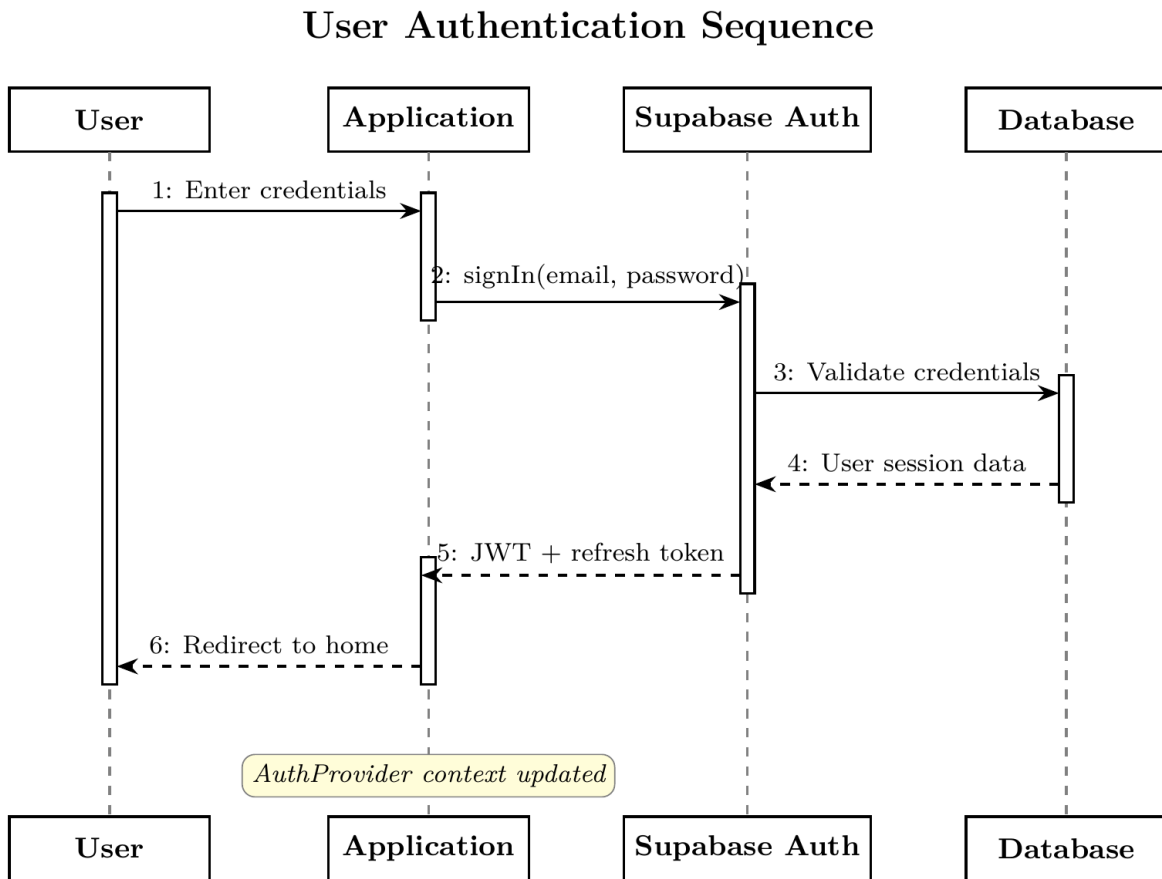


Figure 3.7: *User Authentication Sequence Diagram*

Figure 3.8 illustrates the booking Creation Sequence Diagram.

Booking Creation Sequence

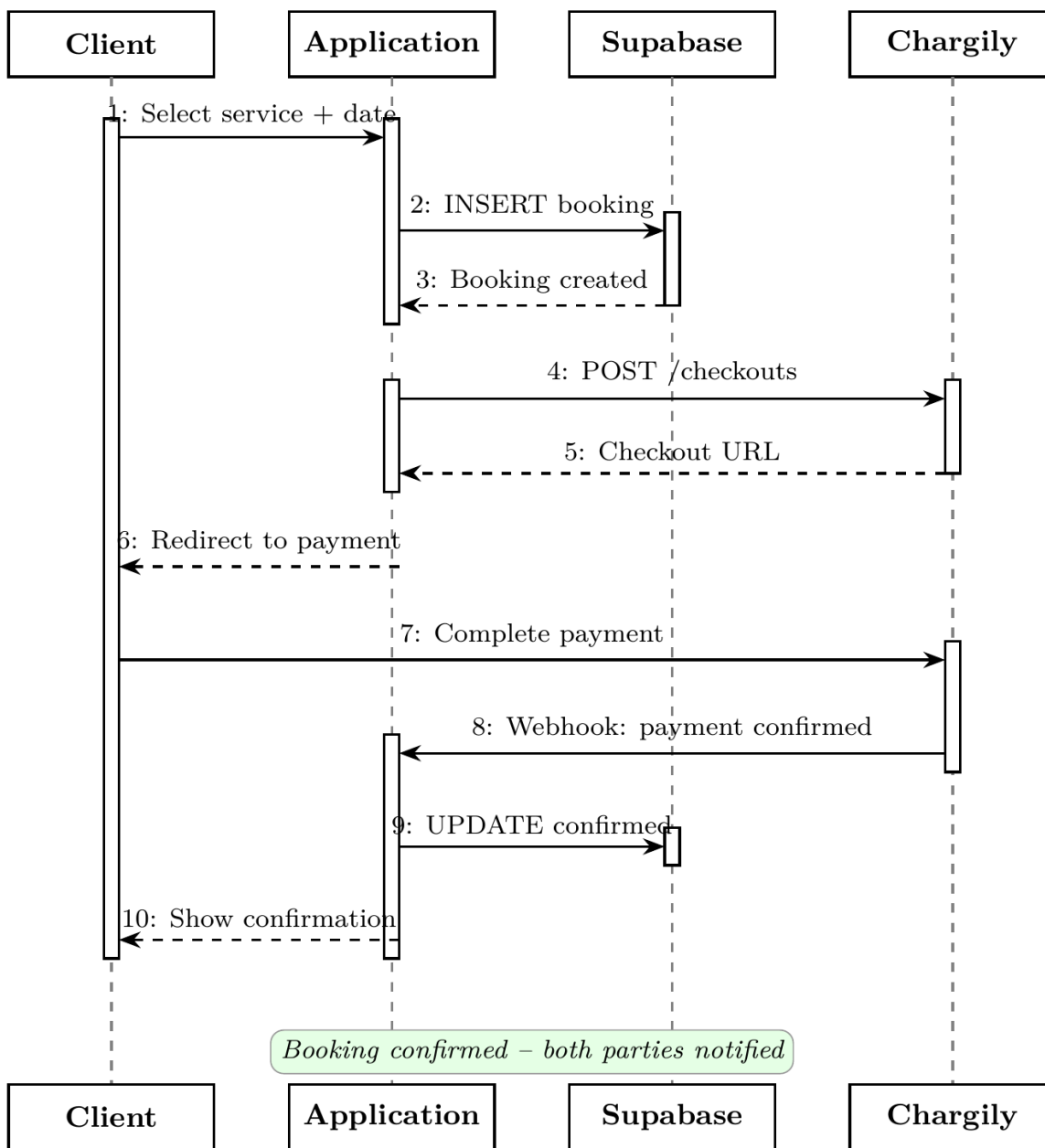


Figure 3.8: *Booking Creation Sequence Diagram*

Figure 3.9 illustrates the session Recording and Speech-to-Text Transcription Sequence Diagram.

Session Recording and Speech-to-Text Sequence

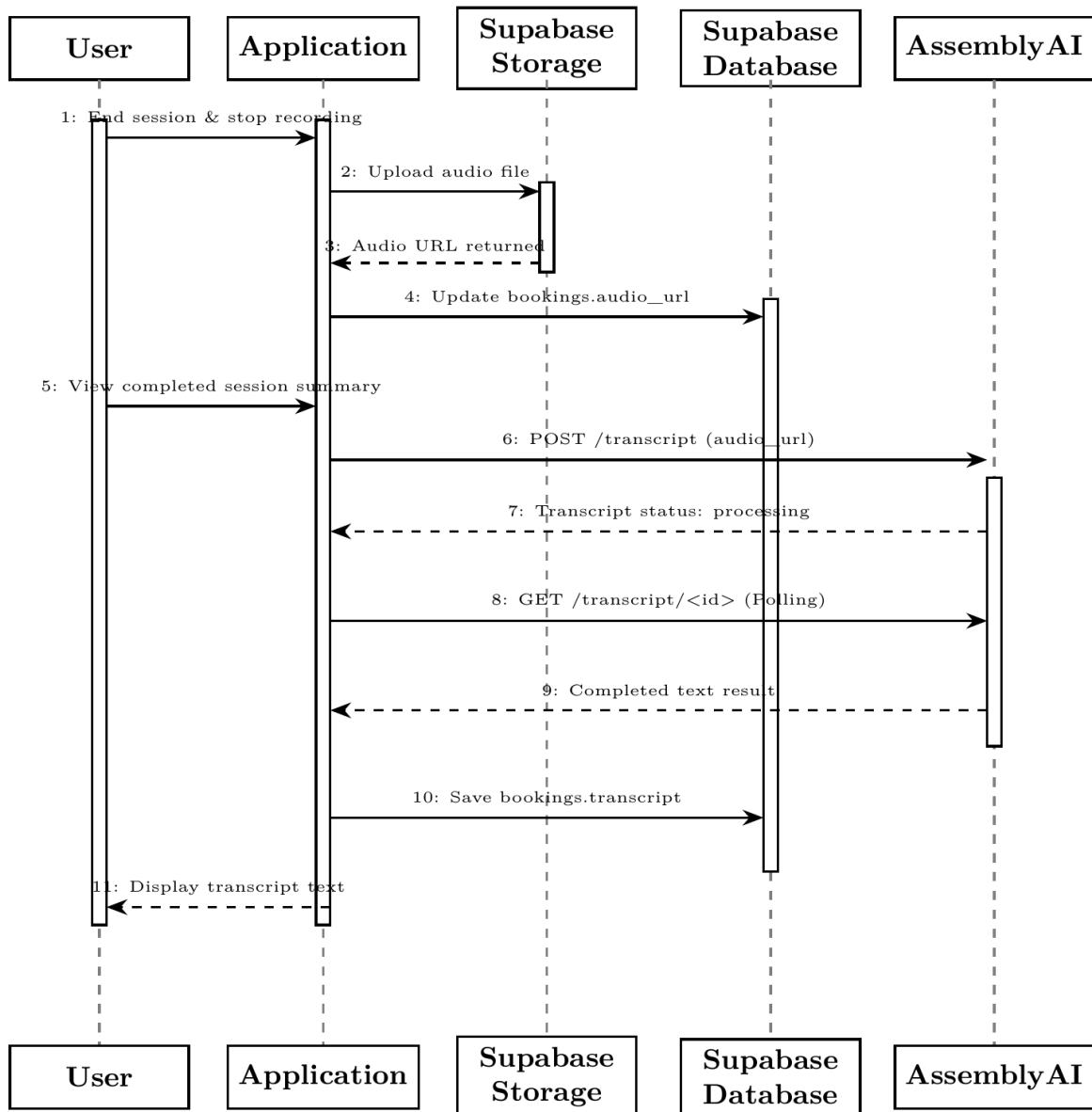


Figure 3.9: *Session Recording and Speech-to-Text Transcription Sequence Diagram*

3.8 Conclusion

This chapter has presented a comprehensive requirements analysis for the Istachir platform. We have identified and documented 33 functional requirements spanning user management, expert services, booking and consultation, payment processing, and content management. Non-functional requirements covering usability, security, and performance have been specified. System actors have been identified and their roles defined. Use case, activity, and sequence diagrams have been provided to illustrate system behavior and user interactions. This requirements specification serves as the foundation for the system design presented in the following chapter.

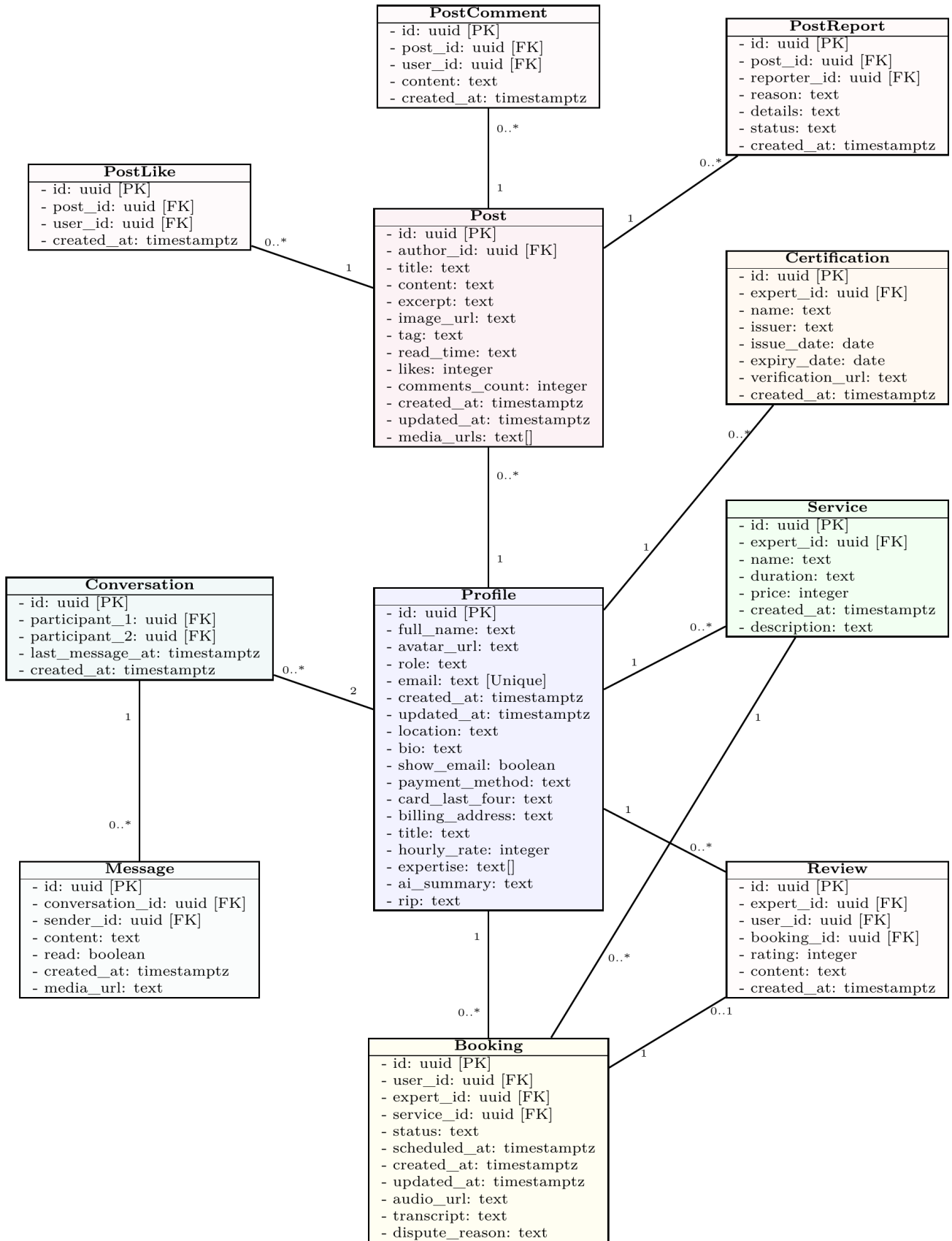
Part IV

System Design

4.1 Introduction

This chapter presents the architectural design of the Istachir consultation marketplace platform. We analyze the system's class structure, entity relationships, technical architecture layers, and deployment configuration. The design emphasizes scalability, maintainability, and alignment with the functional and non-functional requirements established in the previous chapter.

The class diagram defines the structural relationships, attributes, and operations (methods) of the core object-oriented models in the Istachir platform. Figure 4.1 illustrates the UML class hierarchy and their associations.



57
Figure 4.1: Core Entity UML Class Diagram

The system comprises five primary class domains:

4.1.1 Profile Management Domain

The user management domain is consolidated into a single central entity mapping directly to the physical database schema:

- **Profile Class:** Represents the unified user entity in the database (stored in the `profiles` table). It accommodates all platform roles (clients, experts, and administrators) without class inheritance. Common fields include `id`, `full_name`, `avatar_url`, `role` (client, expert, admin), `email`, and geographical `location`. Role-specific expert fields include `professional_title`, `bio`, `hourly_rate`, `expertise` tag arrays, `ai_summary`, and payout details (`rip`).

4.1.2 Consultation and Service Domain

The core consultation business logic is modeled by the following classes:

- **Service Class:** Models the specific consultation offerings created by an expert. It contains attributes like `name`, `description`, `duration`, and `price`. It supports standard CRUD operations: `create()`, `update()`, and `delete()`.
- **Booking Class:** Manages the lifecycle of scheduled appointments. Key attributes include `scheduledAt`, `status` (pending, confirmed, completed, etc.), `audioUrl` (recorded session file), session transcript text, and the AI-generated `summary`. Methods include `confirm()`, `complete()`, `releaseEscrow()`, and `fileDispute()`.
- **Review Class:** Represents client feedback submitted after a session. Attributes include `rating` (integer from 1 to 5) and textual `content`. It provides methods to `submit()` reviews or `flag()` inappropriate entries.

4.1.3 Communication Domain

Direct messaging and real-time exchanges are structured via:

- **Conversation Class:** Represents a logical chat channel between two users. It includes the `lastMessageAt` timestamp for inbox sorting, and operations like `create()` and `archive()`.
- **Message Class:** Models individual text or media messages sent within a conversation. Attributes include `content`, `mediaUrl`, `read` status, and `sentAt` timestamp. Methods include `send()` and `markRead()`.

4.1.4 Social Content Domain

To foster community and allow experts to showcase their insights, the content domain is modeled by:

- **Post Class:** Represents articles or posts published by experts or admins. Attributes include `title`, `content`, `tag`, and a list of `mediaUrls`. Methods include `create()` and `delete()`.
- **PostLike Class:** Represents user reactions to posts. Tracks which user liked which post, with a `createdAt` timestamp.

- **PostComment Class:** Represents user comments on posts. Contains the textual `content` and a `createdAt` timestamp, with methods for CRUD operations.
- **PostReport Class:** Represents content moderation reports filed by users. Contains attributes like `reason`, `details`, `status` (pending, reviewed, dismissed, `action_taken`), and the `createdAt` timestamp, with operational methods like `review()`, `dismiss()`, and `takeAction()`.

4.1.5 Credentials and Verification Domain

To verify the qualifications of professional experts on the platform:

- **Certification Class:** Models the academic or professional credentials uploaded by experts. Attributes include `name`, `issuer`, `issueDate`, `expiryDate`, and a public `verificationUrl`. It supports verification checks via the `verify()` method.

4.2 Relationship Patterns and Data Integrity

The database schema employs several relationship patterns to maintain data integrity:

- **Cascade Deletes:** Deleting a user profile cascades to associated services, certifications, posts, and reviews. Booking and conversation records are preserved for audit purposes.
- **Foreign Key Constraints:** All entity relationships are enforced through PostgreSQL foreign keys with appropriate ON DELETE actions.
- **Row-Level Security:** Every table has RLS policies ensuring users can only access authorized data. Policies are defined for SELECT, INSERT, UPDATE, and DELETE operations.
- **Indexing Strategy:** Indexes are created on frequently queried columns including foreign keys, status fields, and timestamp columns for efficient querying.

4.3 Technical Architecture Implementation

The platform follows a layered architecture pattern with clear separation of concerns:

4.3.1 Presentation Layer

The presentation layer comprises two main applications:

- **Web Administration Dashboard:** Built with React 18, TypeScript, and Vite. The UI leverages shadcn/ui components with Tailwind CSS styling. TanStack React Query manages server state with automatic caching and background updates.
- **Mobile Application:** Built with React Native and Expo, targeting iOS and Android platforms. NativeWind provides Tailwind CSS utility classes. The same TanStack React Query patterns ensure consistent data management.

Both applications share:

- Route protection mechanisms (ProtectedRoute and AdminRoute components).
- Internationalization infrastructure (i18next with English and Arabic locales).
- Authentication context (AuthProvider wrapping the component tree).
- Design systems with consistent color palettes, typography, and spacing.

4.3.2 Application Layer

The application layer encompasses business logic and state management:

- **Server State Management:** TanStack React Query handles all Supabase data operations including queries, mutations, and cache invalidation. Query keys are structured hierarchically for granular cache control.
- **Authentication State:** React Context (AuthProvider) manages authentication state throughout the application lifecycle. The provider integrates with Supabase Auth for session management.
- **Local State:** React's useState hook manages UI-specific state including modals, form inputs, and toggle states.

4.3.3 Data Layer

The data layer uses Supabase as a comprehensive backend platform:

- **PostgreSQL Database:** Relational data storage with advanced features including RLS, full-text search, and stored procedures.
- **Supabase Auth:** Authentication service with support for email/password, Google OAuth, and session management.
- **Supabase Storage:** File storage for avatars, post media, chat media, and session recordings. RLS policies control access at the bucket level.
- **Supabase Realtime:** WebSocket-based subscriptions for real-time messaging, chat, and WebRTC signaling.

4.3.4 Integration Layer

External service integrations are encapsulated in dedicated modules:

- **Google Gemini AI:** Integration through the Google Generative AI SDK for profile Q&A, session summaries, and transcript analysis.
- **AssemblyAI:** REST API integration for audio transcription with polling-based completion detection.
- **Chargily:** Payment gateway integration [10] with checkout session creation and webhook-based confirmation.

4.4 Deployment Architecture

The platform is deployed using a modern cloud infrastructure:

4.4.1 Client-Side Components

- **Web Dashboard:** Deployed on Vercel [11] as a static SPA with client-side routing. Vercel’s global CDN ensures low-latency content delivery.
- **Mobile App:** Distributed through respective app stores. EAS Build manages compilation and signing for both iOS and Android platforms.

4.4.2 Server Infrastructure

- **Supabase Cloud:** Managed database, authentication, and storage services. Auto-scaling handles variable load patterns.
- **Real-Time Infrastructure:** Supabase Realtime manages WebSocket connections for live features. The infrastructure scales based on concurrent connection counts.

4.4.3 Third-Party Services

- **Payment Processing:** Chargily handles payment gateway operations including PCI-compliant card processing.
- **AI Services:** Google Gemini and AssemblyAI APIs are consumed on-demand for AI features.
- **STUN/TURN Servers:** Google and Metered servers facilitate WebRTC NAT traversal.

4.5 AI Workflow Architecture

To elevate the Istachir platform beyond simple, standard API integrations, we designed a custom artificial intelligence orchestration framework. This system architecture represents a core contribution of this thesis, aligning with the requirements of the Artificial Intelligence and Data Science (IASD) specialization. The platform deploys structured generative AI pipelines to deliver two core intelligent services: Retrieval-Augmented Generation (RAG) for expert profile question-answering, and an automated Session Summary Generation Pipeline.

4.5.1 Retrieval-Augmented Generation (RAG) for Expert Profile Q&A

When a client visits an expert’s profile, they can interact with an automated chat interface to query the expert’s background, services, availability, and specific professional views. To prevent hallucinations (where the LLM invents services, qualifications, or prices) and ensure data security, we implement a local RAG-style pipeline.

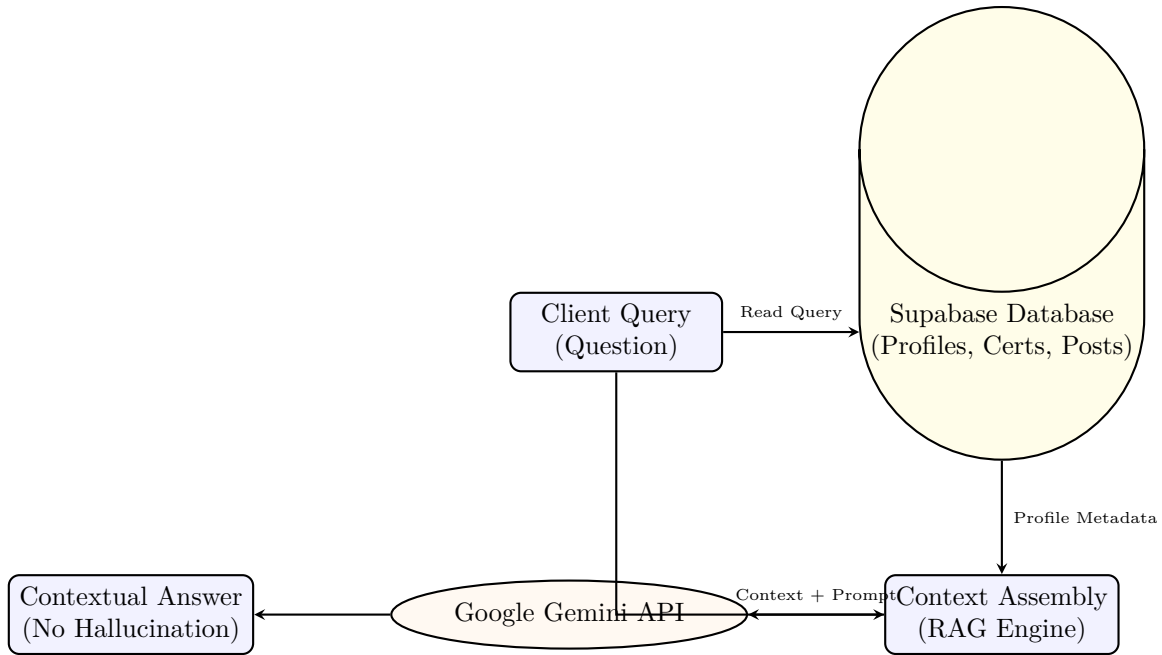


Figure 4.2: *Profile Q&A Chatbot RAG Workflow*

The workflow illustrated in Figure 4.2 proceeds as follows:

1. **Query Submission:** The user submits a natural language question about the expert through the mobile interface.
2. **Dynamic Profile Context Retrieval:** The system queries the Supabase database using the expert's ID, extracting:
 - The expert's textual biography and list of expertise areas.
 - A list of consultation services, including prices and durations.
 - Verified academic and professional certifications (names, issuing bodies, and dates).
 - Content snippets and tags from recent blog posts authored by the expert.
3. **Context Synthesis:** The retrieved data is serialized into a highly structured Markdown document, serving as the local knowledge base for the session.
4. **Prompt Engineering Wrapper:** The question and the structured context document are wrapped in a strict prompt template. The prompt specifies:
 - *Role Definition:* "You are a professional assistant representing the expert [Expert Name]. Speak in the first person on behalf of the expert's services."
 - *Grounding Constraint:* "Answer the user's question using only the provided context. If the answer cannot be found in the context, politely state that you do not have this information and recommend booking a direct session."
 - *Output Structure:* The response must be professional, brief, and translated to match the client's language (English or Arabic).
5. **LLM Execution:** The prompt is sent to Google's Gemini model (using a temperature of 0.2 to enforce high determinism and prevent creativity/hallucination). The output is rendered in real-time.

4.5.2 Session Summary Generation Pipeline

To provide significant value-added benefits to both experts and clients, the platform includes a pipeline that records WebRTC consultations, transcribes the audio, and uses an LLM to generate structured summaries.

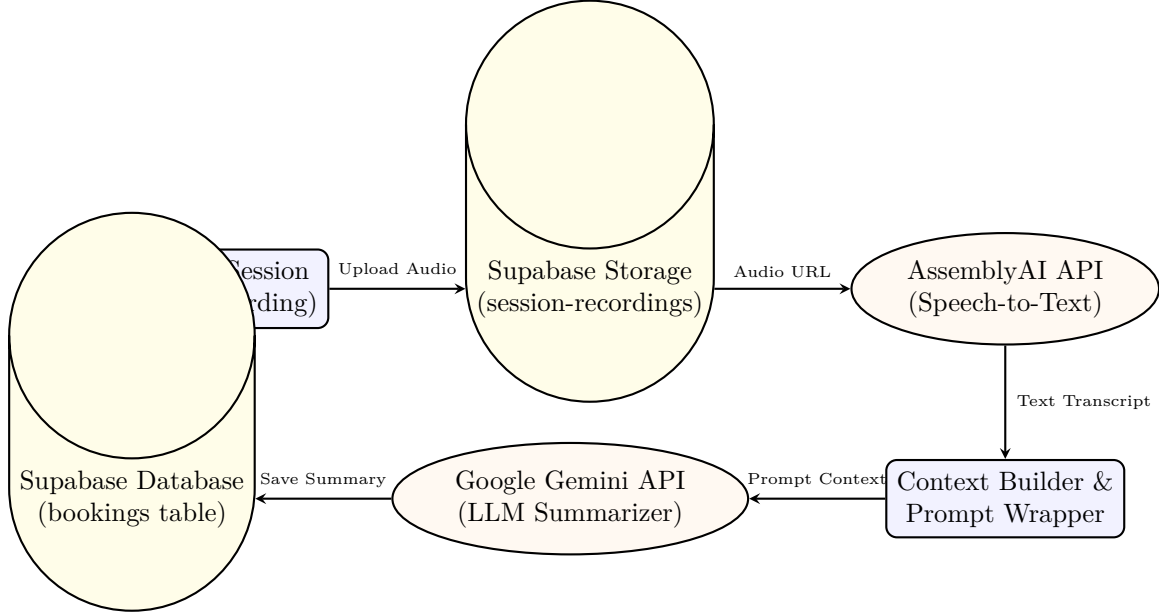


Figure 4.3: *AI Session Summary Generation Pipeline Architecture*

The session summary generation pipeline operates as illustrated in Figure 4.3:

1. **Audio Acquisition and Storage:** During a WebRTC consultation, the local audio tracks are captured. Upon session termination, the application uploads the recorded audio file to the secure `session-recordings` bucket in Supabase Storage.
2. **Asynchronous Transcription Request:** The client initiates an asynchronous transcription process by sending the audio file's public URL to the AssemblyAI API. The system uses a polling loop or a database trigger to wait for AssemblyAI to complete processing. Once finished, the speaker-diarized transcript text (distinguishing between Client and Expert) is retrieved.
3. **Prompt Engineering Scribe Wrapper:** The plain transcript is passed into the prompt wrapper, which includes:
 - *Role Definition:* "You are a professional scribe. Summarize the following consultation transcript between an expert and a client."
 - *Structure Enforcement:* The model is instructed to output the summary in a strict Markdown layout with five sections:
 - (a) **Executive Summary:** A 3-sentence high-level overview of the session.
 - (b) **Key Topics Discussed:** A bulleted list of topics.
 - (c) **Expert Recommendations:** Actionable advice and recommendations provided by the expert.

- (d) **Action Items for the Client:** Tasks the client must complete.
 - (e) **Next Steps:** Recommended timeline or follow-up sessions.
 - *Safety Guardrails:* "Do not invent any facts, dates, names, or values. If a topic is unclear, describe it as undetermined."
4. **Summary Generation & Saving:** The prompt is processed by the Google Gemini model. The resulting structured Markdown summary is saved back to the database in the `bookings` table under the column `ai_summary`.

This structured pipeline ensures that the AI component is not merely a wrapper around an API, but an engineered, secure data flow that maintains user privacy, enforces data grounding, and generates structured output for business-critical features.

4.6 Conclusion

This chapter has presented the system design of the Istachir platform, covering the class structure, entity relationships, layered architecture, and deployment configuration. The design follows industry best practices including clear separation of concerns, proper data integrity mechanisms, and scalable infrastructure choices. The architectural decisions support the functional and non-functional requirements identified in the previous chapter while providing a foundation for future enhancements.

Part V

**Implementation and Technical
Realization**

CHAPTER 5

PLATFORM IMPLEMENTATION

5.1 Introduction

This chapter details the implementation of the Istachir consultation marketplace platform. We describe the development environment, technology stack, and the implementation of both the web administration dashboard (Istachir Web Admin Portal) and the customer mobile application (Istachir Mobile Application). The chapter covers key implementation decisions, code architecture, and feature realization.

5.2 Development Environment and Technology Stack

5.2.1 Development Hardware Configuration

The development was conducted on a standard development workstation. The hardware specifications were sufficient to run multiple development servers simultaneously including the React development server, Expo development tools, and database management interfaces.

5.2.2 Software Environment

Table 5.1 shows the development Software Stack.

Table 5.1: *Development Software Stack*

Category	Technology	Version
Web Framework	React	18.3.1
Mobile Framework	React Native (Expo)	SDK 54
Language	TypeScript	5.3+
Build Tool (Web)	Vite	5.x
Backend Platform	Supabase	2.x
CSS Framework	Tailwind CSS	3.4.x
State Management	TanStack React Query	5.x
AI Integration	Google Gemini AI	0.24.x
Transcription	AssemblyAI	REST API
Payment Gateway	Chargily	REST API

5.2.3 Version Control and Collaboration

The project uses Git for version control with GitHub as the remote repository platform. The repository structure separates the web dashboard and mobile application into distinct project directories while maintaining shared documentation and configuration.

5.3 Web Admin Dashboard Implementation

The web administration dashboard, also referred to as the Istachir Web Admin Portal, provides comprehensive platform management capabilities. It is built with React 18, TypeScript, and Vite, utilizing the shadcn/ui component library for consistent design.

Figure 5.1 illustrates the web Dashboard Landing Page.

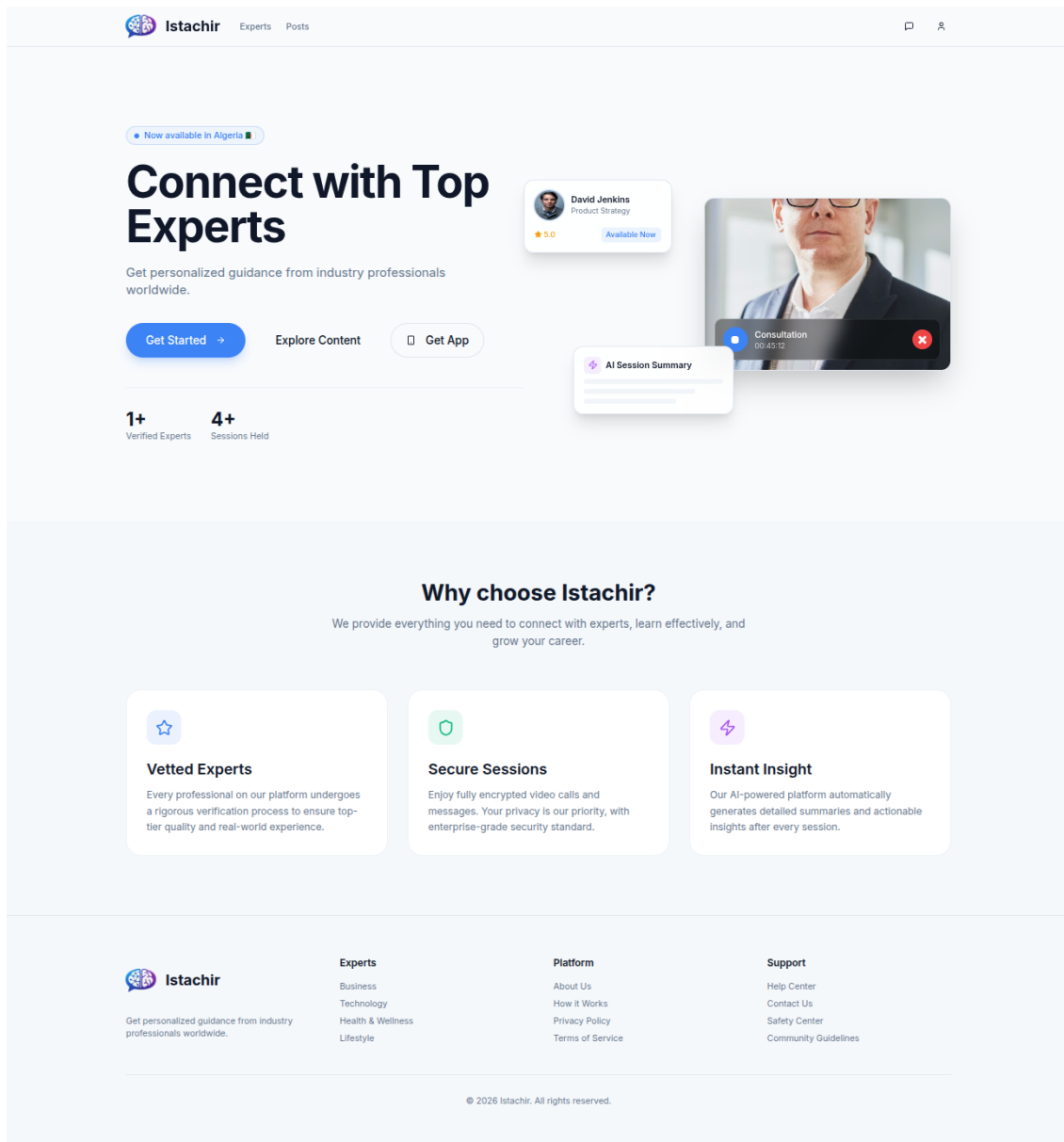


Figure 5.1: *Web Dashboard Landing Page*

Figure 5.2 illustrates the web Login Page.

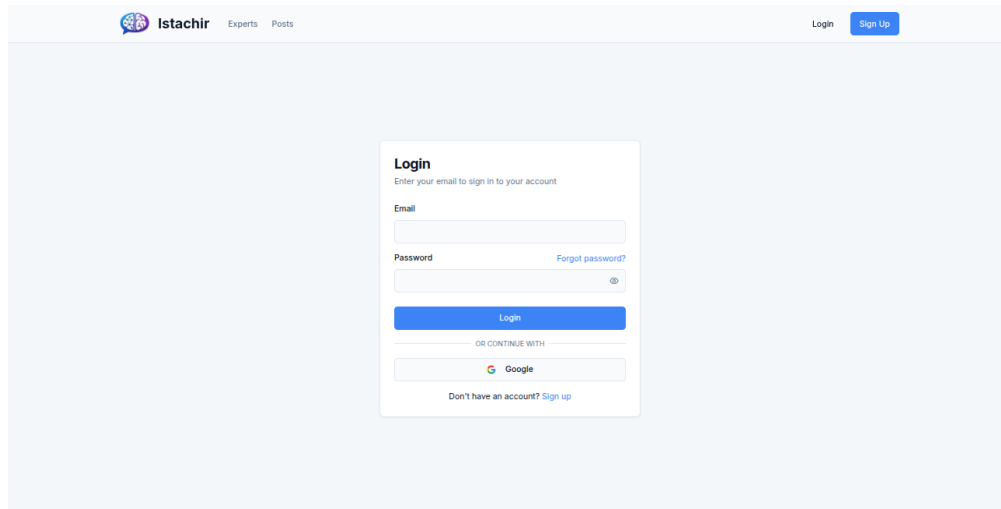


Figure 5.2: *Web Login Page*

Figure 5.3 illustrates the web Sign Up Page.

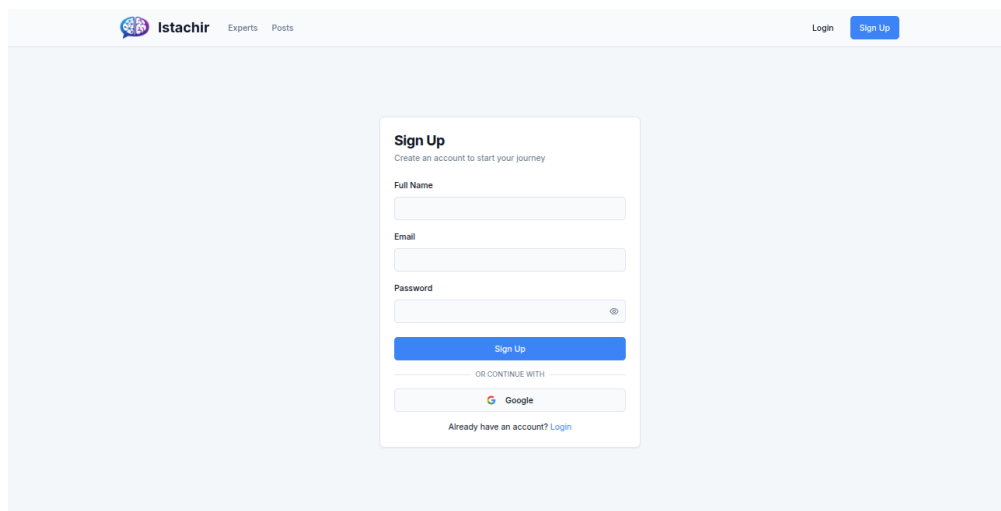


Figure 5.3: *Web Sign Up Page*

5.3.1 User Management Systems

The dashboard implements a complete user management interface allowing administrators to:

- **View All Users:** A searchable, paginated table displaying user profiles with key information including name, email, role, registration date, and account status.
- **Role Assignment:** Administrators can modify user roles through a dropdown interface. Role changes are immediately persisted to the database and reflected in access controls.
- **Account Moderation:** The system provides capabilities to review and moderate user accounts including profile content and reported violations.

The user management interface uses TanStack React Query with proper cache invalidation to ensure data consistency. The ‘useQuery’ hook fetches profiles with user information, while ‘useMutation’ handles role updates with optimistic UI updates.

Figure 5.4 illustrates the web Dashboard – Expert Browsing Interface.

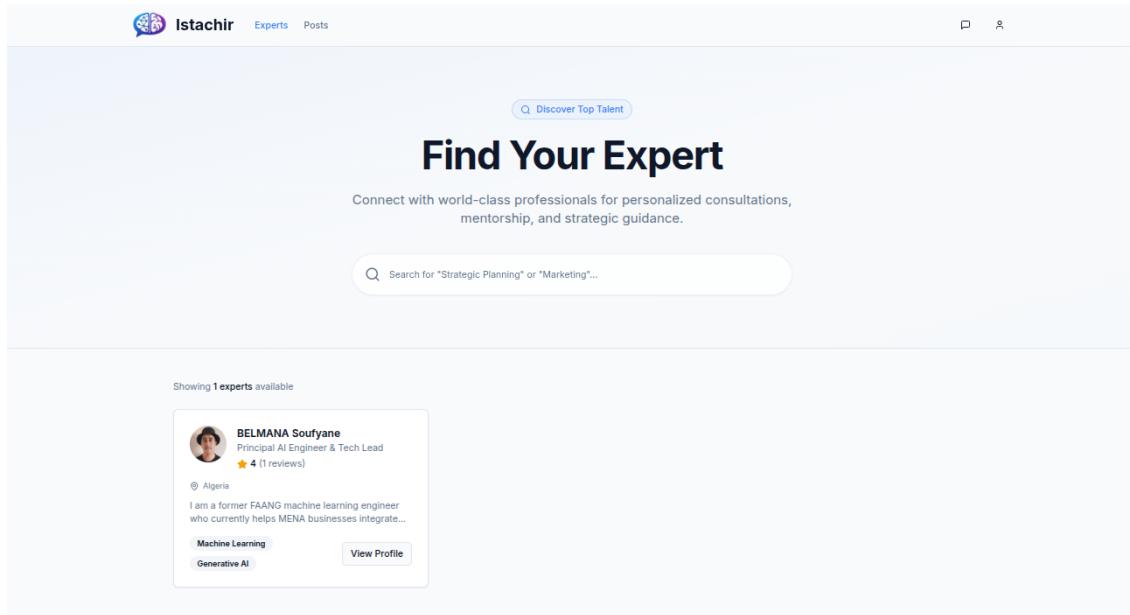


Figure 5.4: *Web Dashboard – Expert Browsing Interface*

Figure 5.5 illustrates the web – Expert Posts Directory Interface.

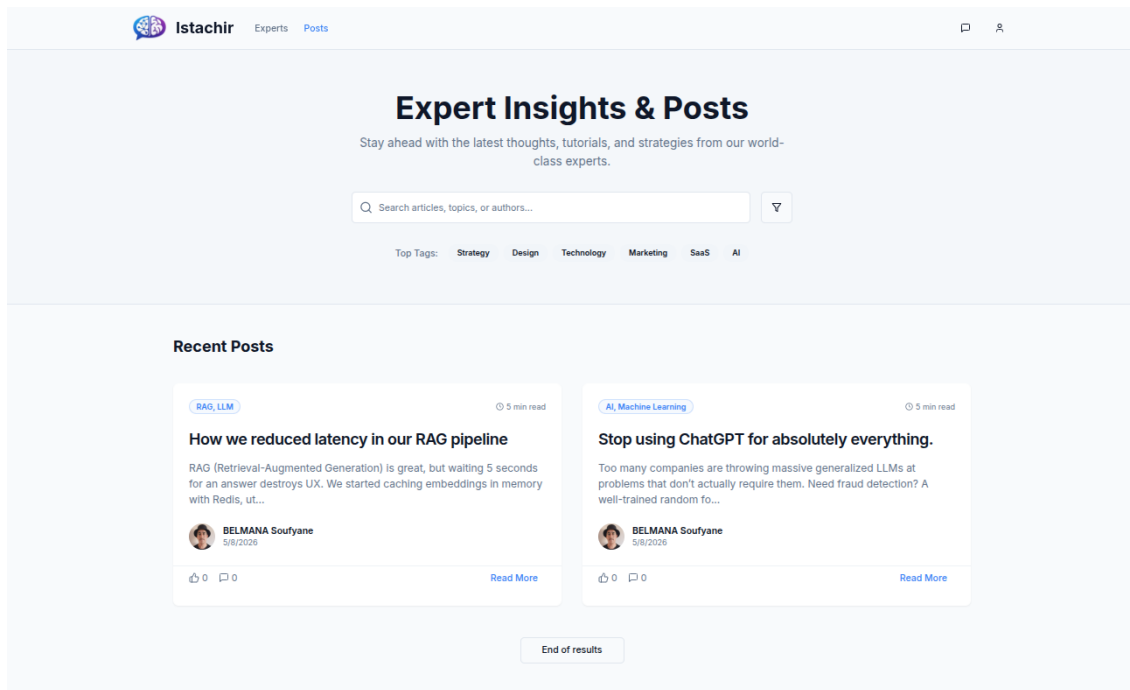


Figure 5.5: *Web – Expert Posts Directory Interface*

5.3.2 Expert and Service Management

The dashboard enables comprehensive oversight of expert services:

- **Service Listing:** Administrators can view all services offered by experts, including pricing, duration, and current status.
- **Certification Verification:** Expert-uploaded certifications are displayed for administrator review. The interface supports verification status tracking.
- **Review Moderation:** The system displays all expert reviews with the ability to delete inappropriate content.

Figure 5.6 illustrates the web Dashboard – Expert Profile View.

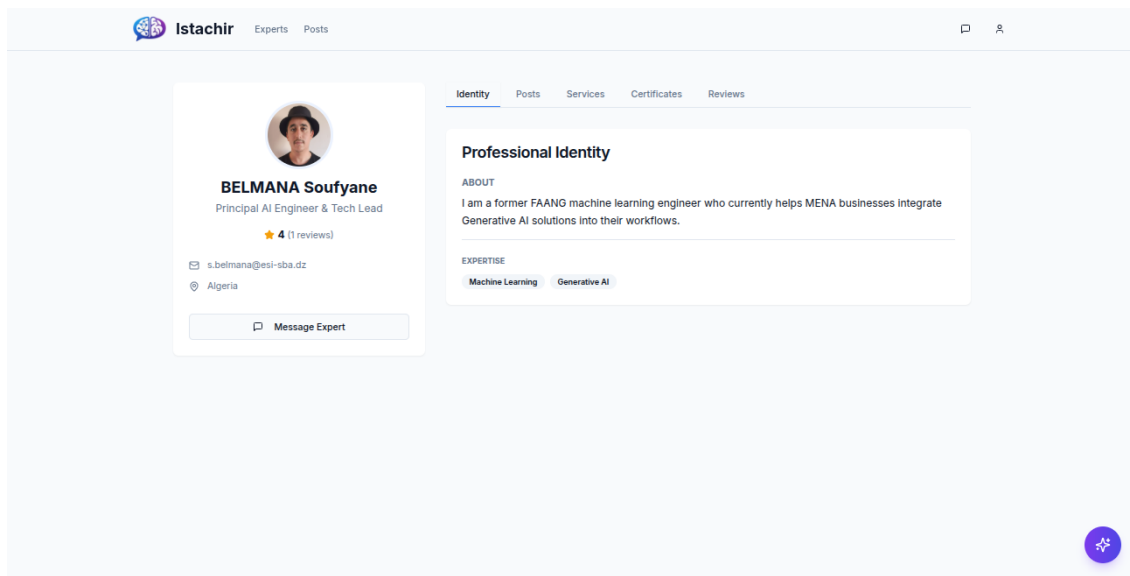


Figure 5.6: *Web Dashboard – Expert Profile View*

5.3.3 Booking and Schedule Management

Booking management provides a centralized view of all platform consultations:

- **Booking Overview:** A comprehensive table displays all bookings with expert and client names, scheduled dates, current status, and payment information.
- **Status Management:** Administrators can view booking status progression and intervene when necessary, particularly for dispute resolution.
- **Session Records:** Completed sessions display metadata including duration, recording URLs, and AI-generated summaries.

Figure 5.7 illustrates the web Dashboard – Booking Schedule Calendar.

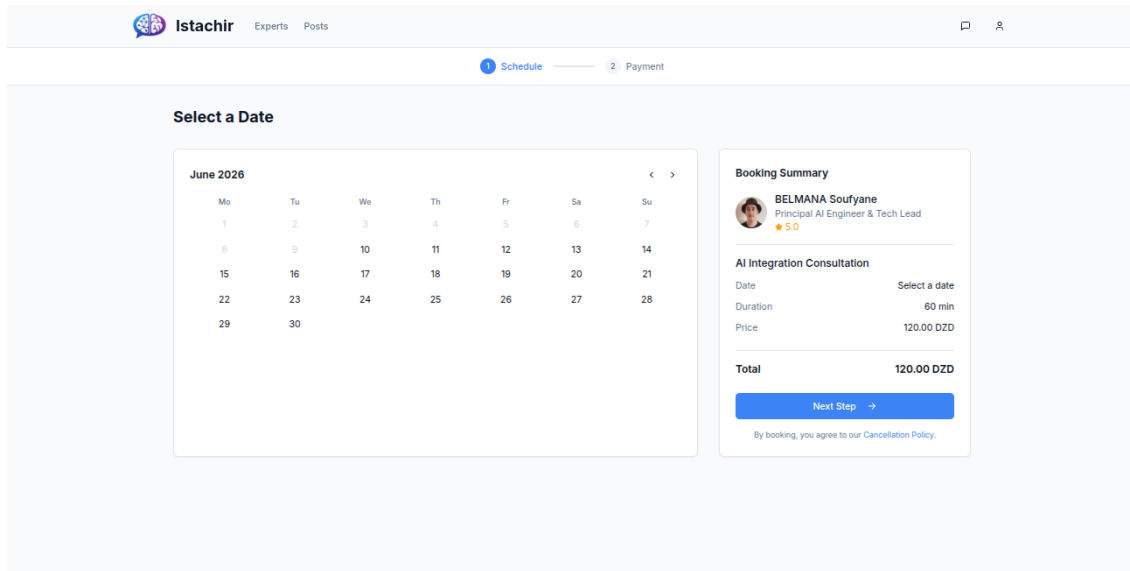


Figure 5.7: *Web Dashboard – Booking Schedule Calendar*

Figure 5.8 illustrates the web Dashboard – Administration Management Interface.

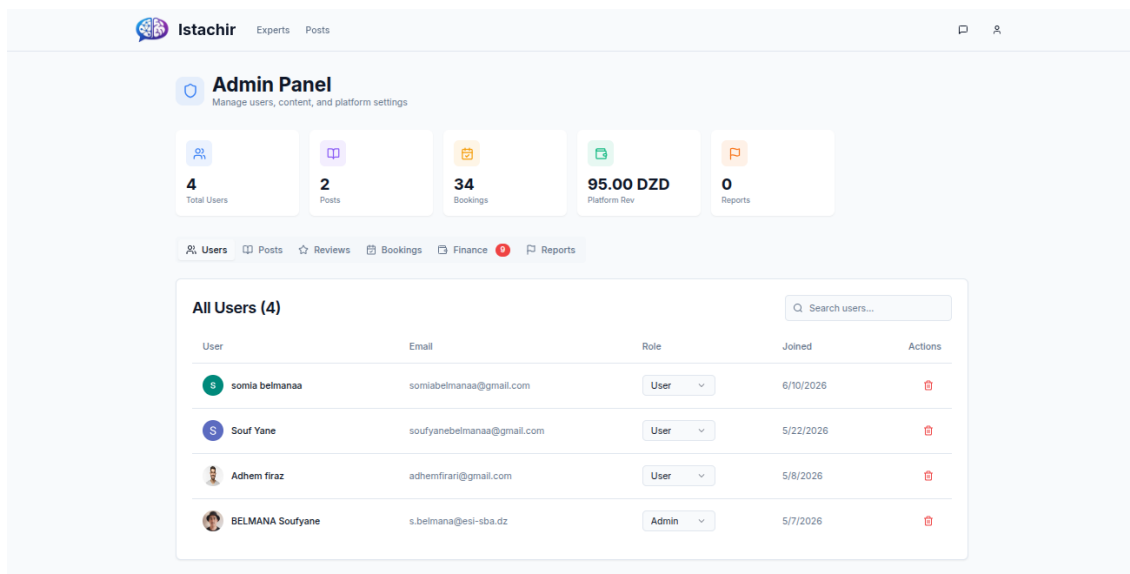


Figure 5.8: *Web Dashboard – Administration Management Interface*

Figure 5.9 illustrates the web Dashboard – Booking Detail View.

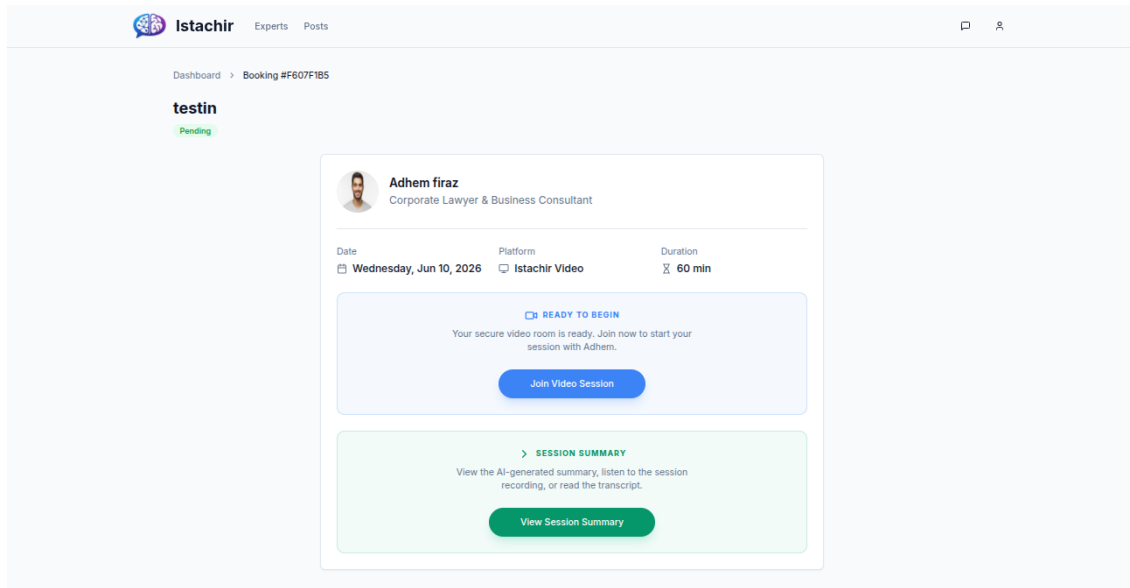


Figure 5.9: *Web Dashboard – Booking Detail View*

Figure 5.10 illustrates the web Dashboard – Post-Session Summary View.

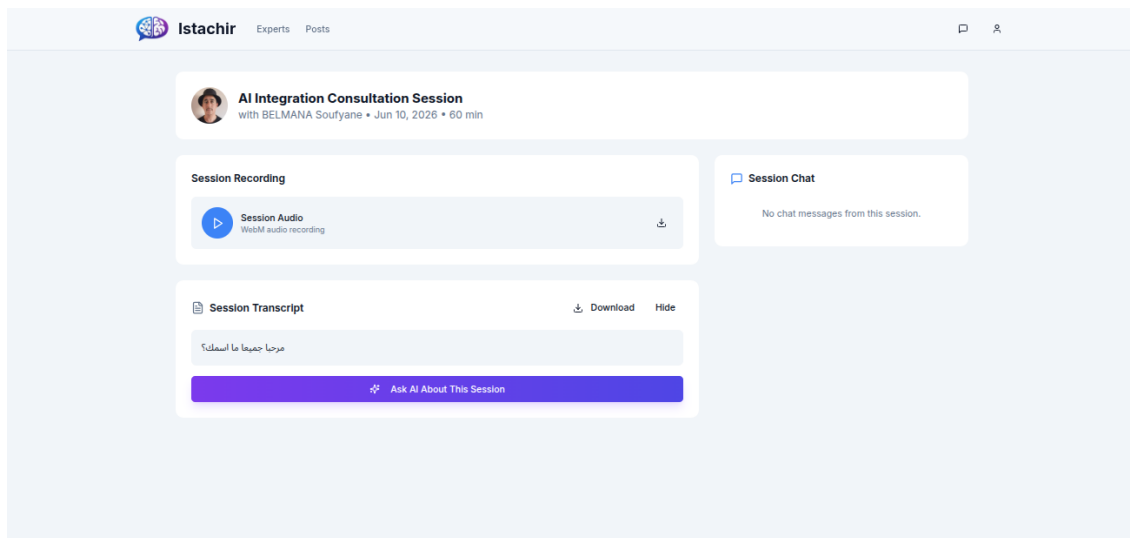


Figure 5.10: *Web Dashboard – Post-Session Summary View*

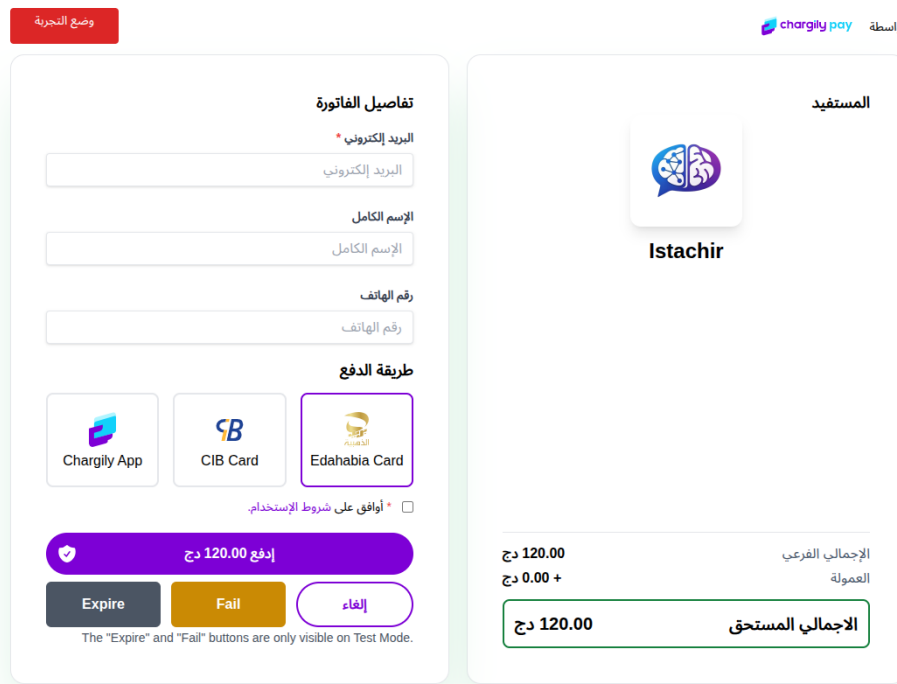
5.3.4 Payment and Financial Management

The financial management system provides tools for platform revenue oversight:

- **Revenue Dashboard:** Key financial metrics including total platform revenue, pending escrow amounts, and completed payouts are displayed with visual charts using Recharts.
- **Transaction List:** All platform transactions are displayed with filtering by date range, status, and involved parties.

- **Payout Management:** Administrators can review expert payout requests, verify CCP/RIP account details, and mark payouts as processed.

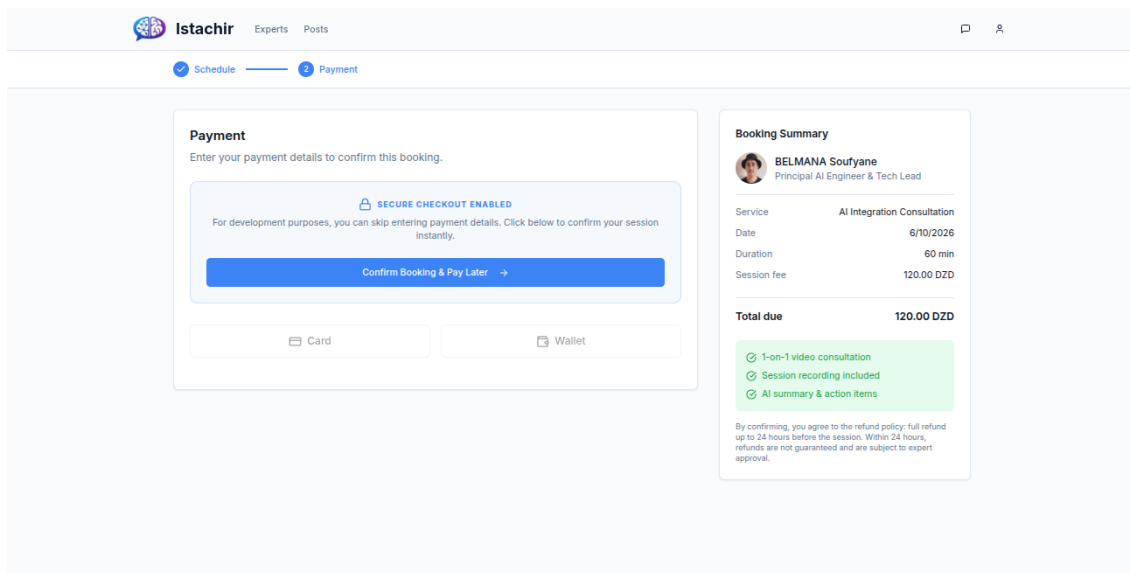
Figure 5.11 illustrates the web – Chargily Payment Checkout Page.



The image shows a web interface for a payment checkout. On the left, there's a form titled 'تفاصيل الفاتورة' (Invoice Details) with fields for 'البريد الإلكتروني' (Email), 'الإسم الكامل' (Full Name), and 'رقم الهاتف' (Phone Number). Below this is a section for 'طريقة الدفع' (Payment Method) with three options: 'Chargily App', 'CIB Card', and 'Edahabia Card'. A checkbox for 'أوافق على شروط الاستخدام' (I agree to the terms of use) is present. At the bottom of the form are buttons for 'Expire', 'Fail', and 'إلغاء' (Cancel). A note states: 'The "Expire" and "Fail" buttons are only visible on Test Mode.' On the right, there's a section titled 'المستفيد' (Beneficiary) showing a profile for 'Istachir' with a brain icon. Below this, it shows the 'الإجمالي الفرعي' (Subtotal) as '120.00 دج' and 'العمولة' (Commission) as '0.00 دج'. The 'الإجمالي المستحق' (Total Due) is '120.00 دج'.

Figure 5.11: Web – Chargily Payment Checkout Page

Figure 5.12 illustrates the web Dashboard – Payment and Webhook Confirmation.



The image shows a web dashboard for 'Istachir' with tabs for 'Experts' and 'Posts'. The 'Payment' tab is active. The main section is titled 'Payment' and says 'Enter your payment details to confirm this booking.' It features a 'SECURE CHECKOUT ENABLED' banner and a 'Confirm Booking & Pay Later' button. Below this are buttons for 'Card' and 'Wallet'. On the right, there's a 'Booking Summary' section for 'BELMANA Soufyane', Principal AI Engineer & Tech Lead. It lists the service as 'AI Integration Consultation', the date as '6/10/2026', the duration as '60 min', and the session fee as '120.00 DZD'. The 'Total due' is '120.00 DZD'. A green box lists the included items: '1-on-1 video consultation', 'Session recording included', and 'AI summary & action items'. A disclaimer at the bottom states: 'By confirming, you agree to the refund policy: full refund up to 24 hours before the session. Within 24 hours, refunds are not guaranteed and are subject to expert approval.'

Figure 5.12: Web Dashboard – Payment and Webhook Confirmation

5.3.5 Real-Time Communication

The web dashboard integrates real-time communication tools that enable both pre-consultation messaging and live video consultations directly from the browser:

- **Messaging Interface:** Administrators and users can view all platform conversations through a unified chat panel. The interface displays conversation threads with participant avatars, message history, timestamps, and message status indicators. Messages are delivered in real time via Supabase Realtime subscriptions, ensuring instant communication between clients and experts before and after consultations.
- **Video Consultation Room:** The web-based video call interface leverages browser-native WebRTC APIs to provide real-time audio and video communication. The call room displays both local and remote video streams with controls for mute/unmute, video toggle, screen sharing, and an in-call text chat panel. A session timer tracks consultation duration, and upon session termination the audio recording is uploaded to Supabase Storage for AI-powered transcription and summary generation.

Figure 5.13 illustrates the web Dashboard – Messaging Interface.

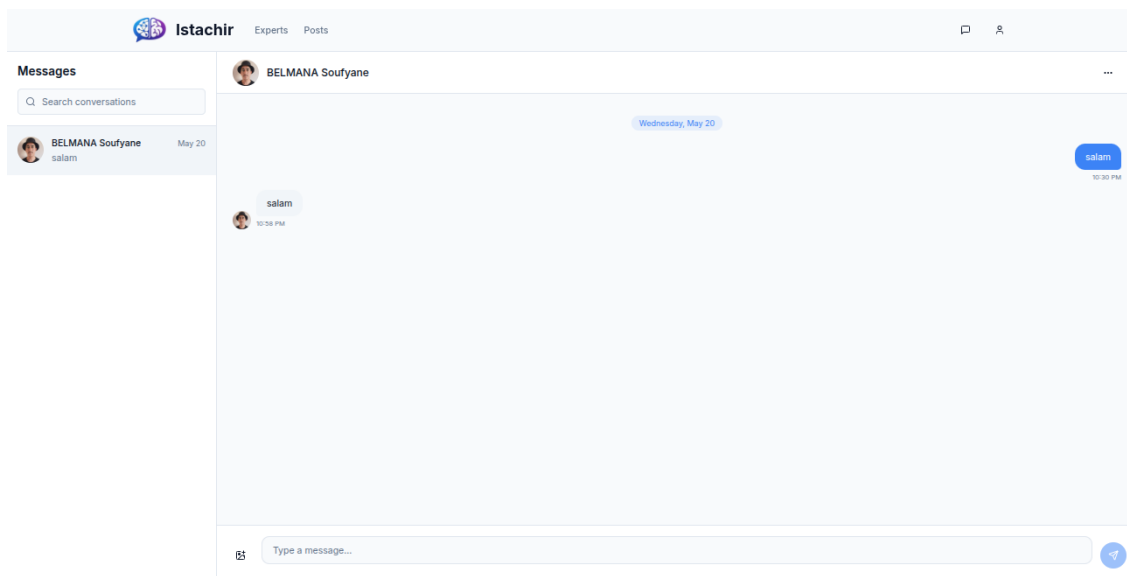


Figure 5.13: *Web Dashboard – Messaging Interface*

Figure 5.14 illustrates the web Dashboard – Video Consultation Room.

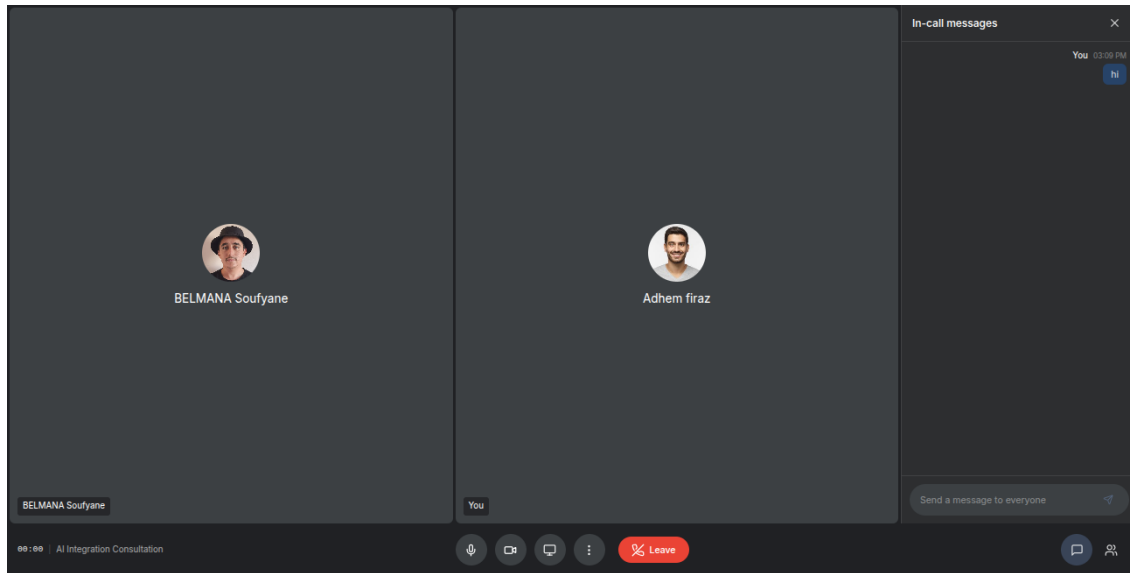


Figure 5.14: *Web Dashboard – Video Consultation Room*

5.4 Customer Mobile Application Implementation

The customer mobile application, referred to as the Istachir Mobile Application, provides the client-facing experience for discovering experts, booking consultations, and managing sessions. It is built with React Native and Expo, targeting both iOS and Android platforms.

5.4.1 Authentication and Onboarding

The mobile authentication flow provides a seamless entry point:

- **Welcome Screen:** Branded landing page with platform name and key value propositions. Users can proceed to login or create a new account.
- **Multi-Method Authentication:** Email/password registration integrates with Supabase Auth. Google Sign-In is implemented using @react-native-google-signin for native platforms.
- **Deep Link Handling:** The app handles OAuth callbacks and password reset deep links through Expo's linking configuration.
- **Profile Creation:** Upon first login, user profiles are automatically created from authentication metadata. Users can complete their profiles through the settings interface.

Figure 5.15 illustrates the mobile App – Authentication Flow (Splash, Login, Sign Up).

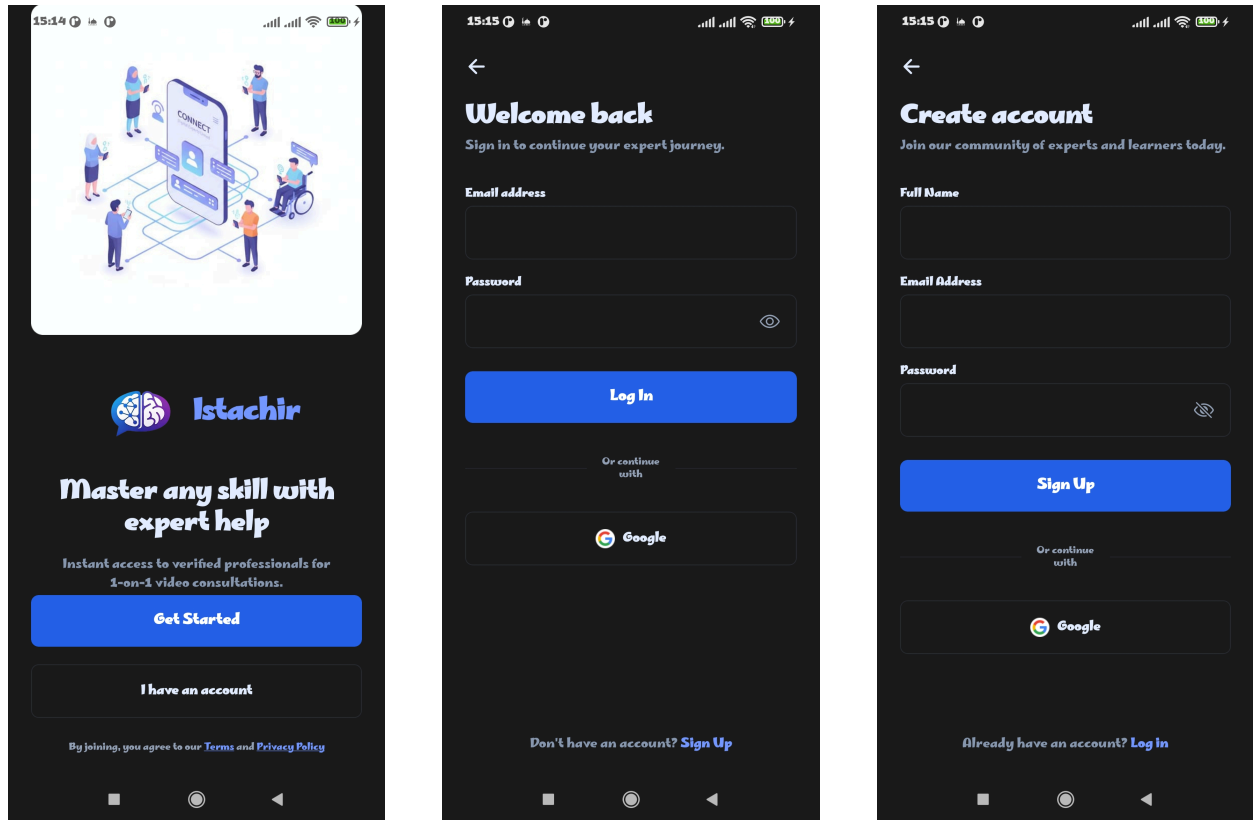


Figure 5.15: Mobile App – Authentication Flow (Splash, Login, Sign Up)

5.4.2 Expert Discovery and Profiles

The expert discovery system helps clients find the right professional:

- **Home Screen:** A personalized dashboard displays active sessions, top-rated experts, and recommended posts. Pull-to-refresh keeps content current.
- **Search Interface:** Real-time search filters experts by name, title, or bio. Results display compact expert cards with key information including rating and expertise tags.
- **Expert Profile:** A comprehensive profile view with tabbed navigation through Identity, Posts, Services, Certifications, and Reviews sections.
- **AI Profile Assistant:** A floating action button opens a modal where users can ask Gemini AI questions about the expert. The AI is prompted with the expert's profile data for contextual responses.
- **Review System:** Reviews display with rating distribution charts and filtering by star rating. Users can mark reviews as helpful.

Figure 5.16 illustrates the mobile App – Expert Discovery and Community (Home, Search, Posts).

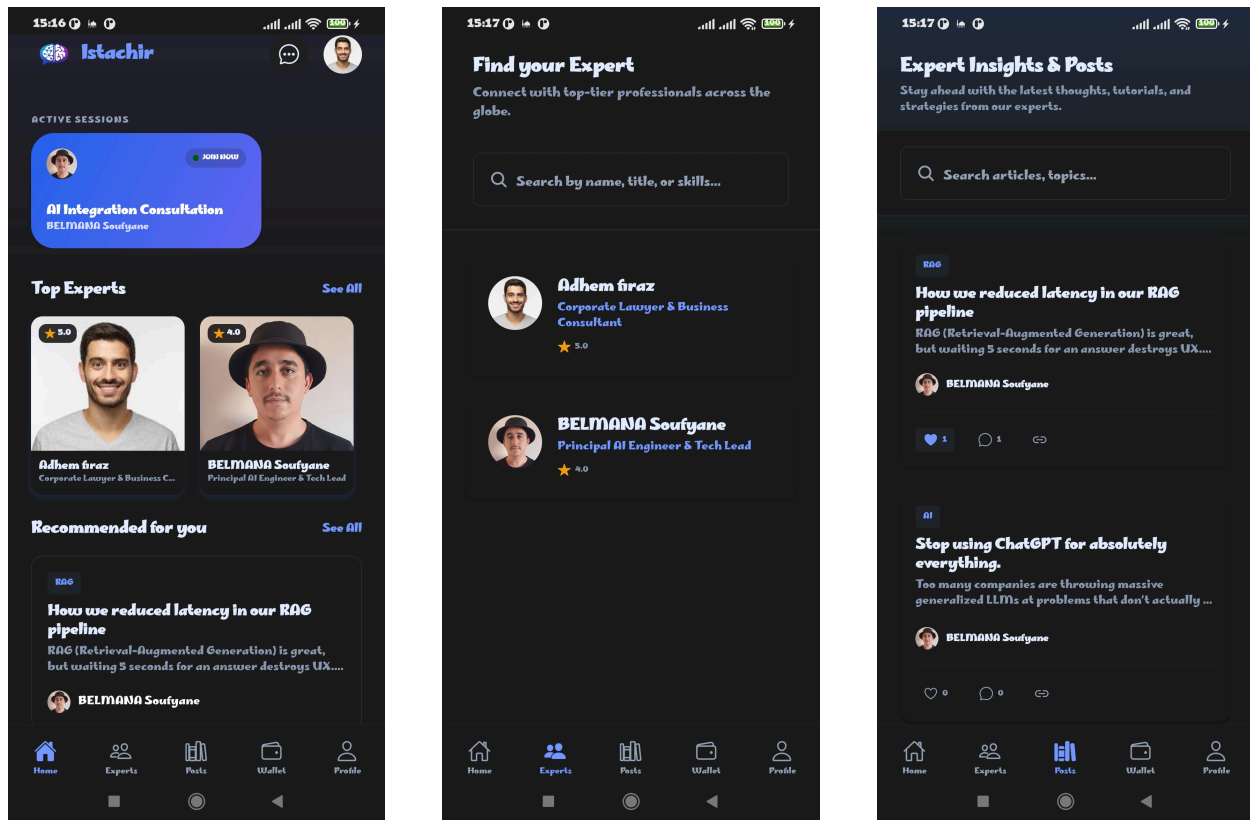


Figure 5.16: *Mobile App – Expert Discovery and Community (Home, Search, Posts)*

Figure 5.17 illustrates the mobile App – Expert Profile and Client Dashboard.

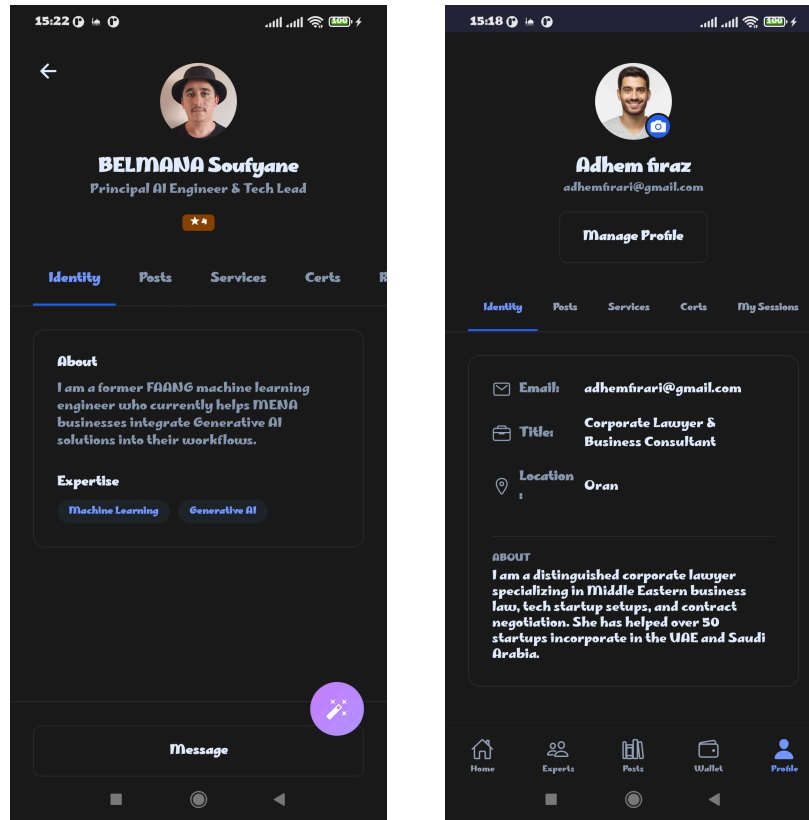


Figure 5.17: Mobile App – Expert Profile and Client Dashboard

5.4.3 Booking and Scheduling

The booking flow guides users through consultation scheduling:

- **Service Selection:** Users select from available consultation services, each displaying duration and price (DZD).
- **Calendar Picker:** A 30-day date selection interface prevents past date selection. Selected dates are highlighted with visual feedback.
- **Checkout Summary:** Booking summary displays expert details, selected service, scheduled date, and total cost including platform fee breakdown.
- **Payment Processing:** The app redirects to Chargily payment page through the device browser. Webhook confirmation updates booking status.
- **Confirmation:** Successful bookings display confirmation details with options to add to calendar or share booking information.

Figure 5.18 illustrates the mobile App – Booking Interface and Checkout Details.

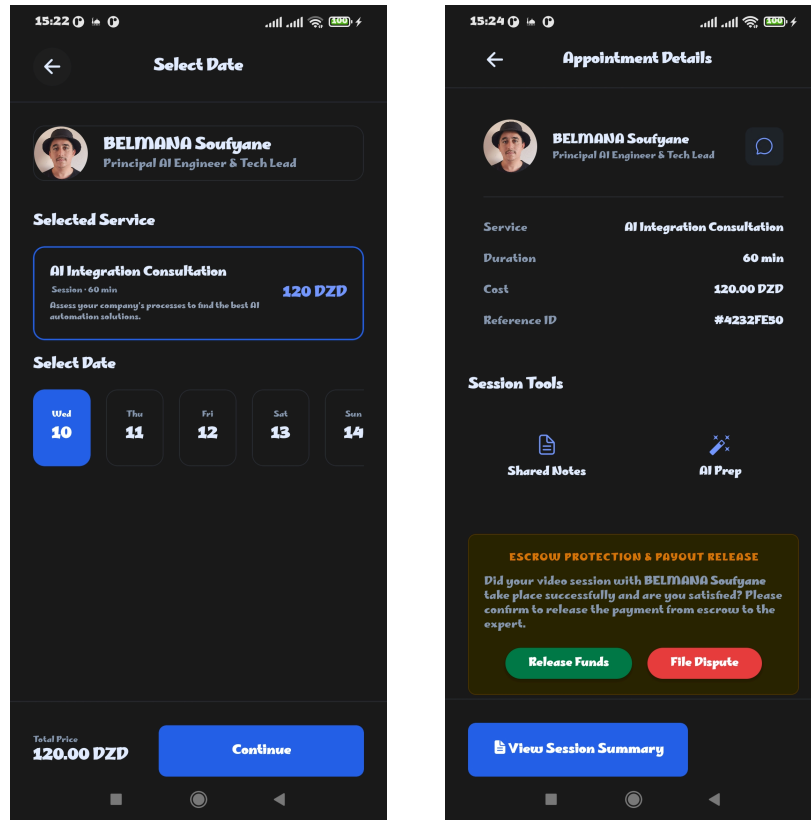


Figure 5.18: *Mobile App – Booking Interface and Checkout Details*

Figure 5.19 illustrates the mobile App – Completed Session and AI-Generated Summary.

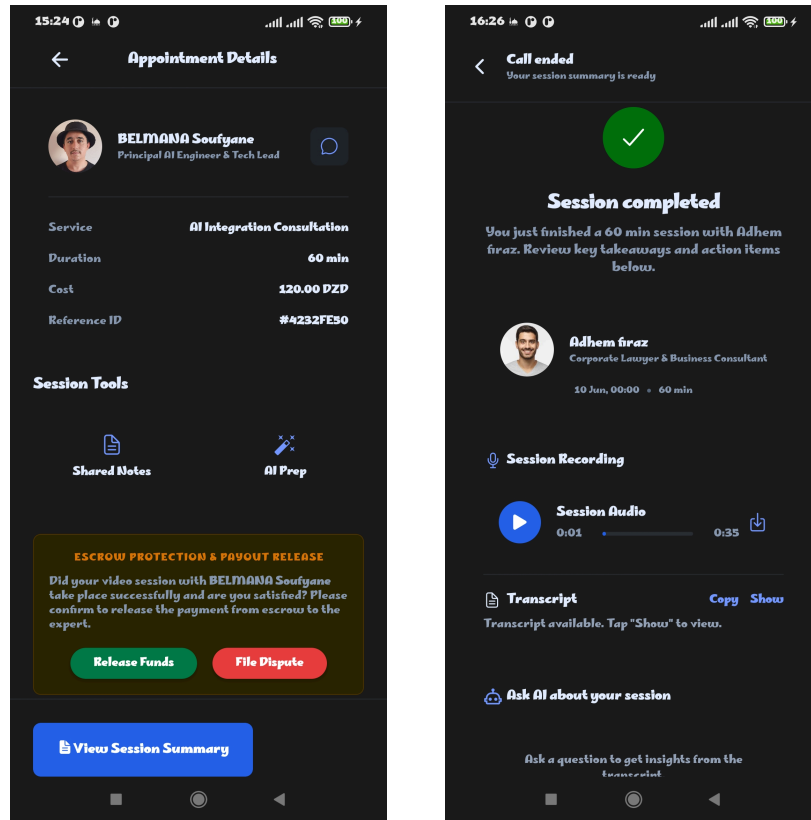


Figure 5.19: Mobile App – Completed Session and AI-Generated Summary

5.4.4 Video Consultation (WebRTC)

The WebRTC video call implementation provides real-time consultation capabilities:

- **Cross-Platform WebRTC:** The web dashboard uses browser WebRTC APIs while the mobile app uses react-native-webrtc. A platform-adaptive abstraction layer handles differences in media stream management.
- **Signaling Protocol:** Supabase Realtime broadcast channels exchange offer, answer, and ICE candidate messages. The booking client initiates as the offerer, with retry logic for connection resilience.
- **Call Controls:** User interface includes mute/unmute, video toggle, camera flip (mobile), speaker toggle, and screen sharing (web). The red end-call button terminates the session.
- **Call Timer:** Duration tracking starts when the connection is established and displays in MM:SS format.
- **Session Recording:** Audio is recorded using the MediaRecorder API (web) and sent to Supabase Storage for later processing.
- **In-Call Chat:** Participants can exchange text messages during the call through a slide-out panel using Supabase Realtime broadcast.

Figure 5.20 illustrates the mobile App – Video Consultation Room and Appointment Details.

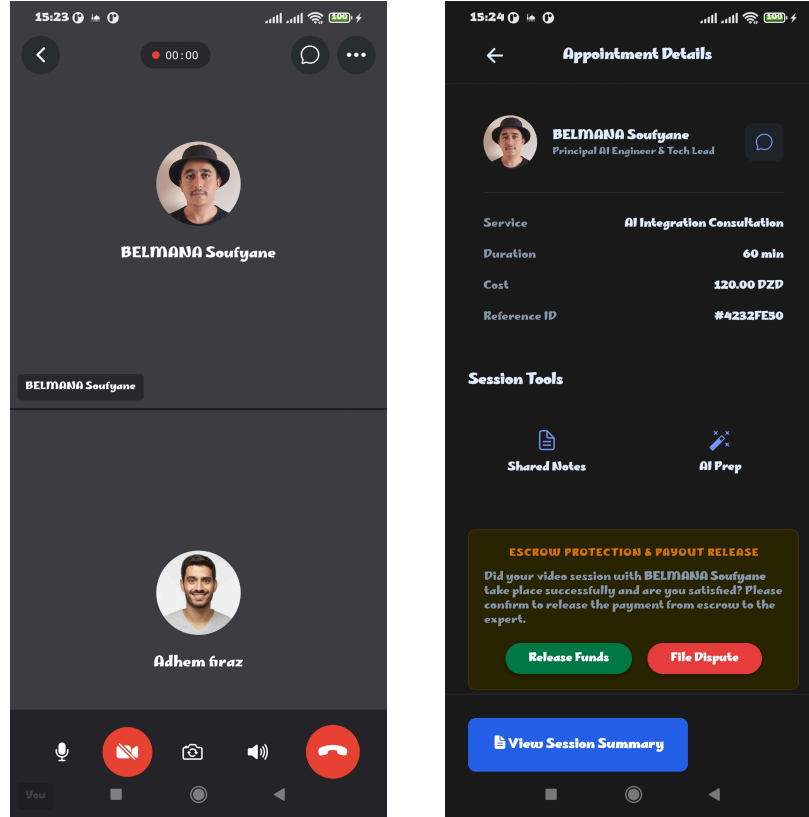


Figure 5.20: Mobile App – Video Consultation Room and Appointment Details

5.4.5 AI-Powered Session Features

Artificial intelligence enhances the consultation experience at multiple touchpoints:

- **Session Summary Generation:** After consultation completion, Gemini AI generates a structured summary including:
 - Overview of the consultation discussion.
 - Key insights and takeaways.
 - Action items for the client.
 - Recommended resources.
 - Focus areas for the next session.
- **Transcription Service:** AssemblyAI transcribes session audio recordings with multi-language support and speaker diarization. The system polls the API for completion status.
- **Transcript Q&A:** Users can ask questions about their session transcript through a chat interface. Gemini AI uses the transcript as context to provide relevant answers.
- **Model Resilience:** A fallback chain of Gemini models ensures service availability despite rate limits or temporary outages.

Figure 5.21 illustrates the mobile App – Post-Session AI Summary and Transcription View.

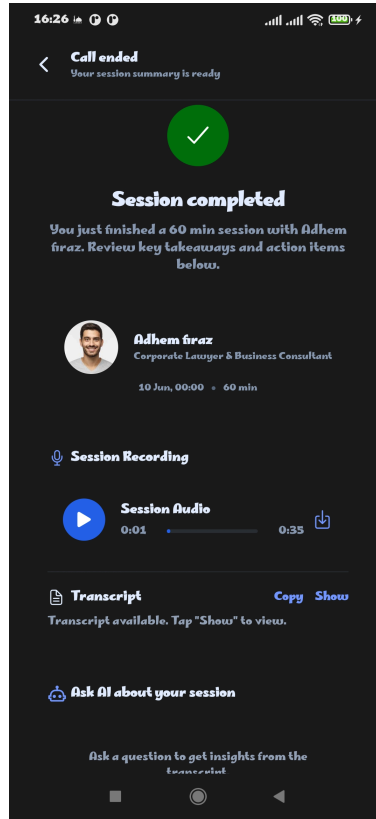


Figure 5.21: *Mobile App – Post-Session AI Summary and Transcription View*

5.4.6 Messaging System

The real-time messaging system facilitates pre and post-consultation communication:

- **Conversation List:** All user conversations display with participant avatars, last message preview, timestamps, and unread message badges.
- **Chat Interface:** Individual conversation screens support text messages and image uploads. Messages display date separators (Today, Yesterday, full date).
- **Real-Time Updates:** Supabase Realtime subscriptions deliver new messages instantly. The UI scrolls to display new messages with proper scroll position management.
- **Read Receipts:** Messages are automatically marked as read when the recipient opens the conversation. Unread counts are updated accordingly.

Figure 5.22 illustrates the mobile App – Real-Time Chat Interface.

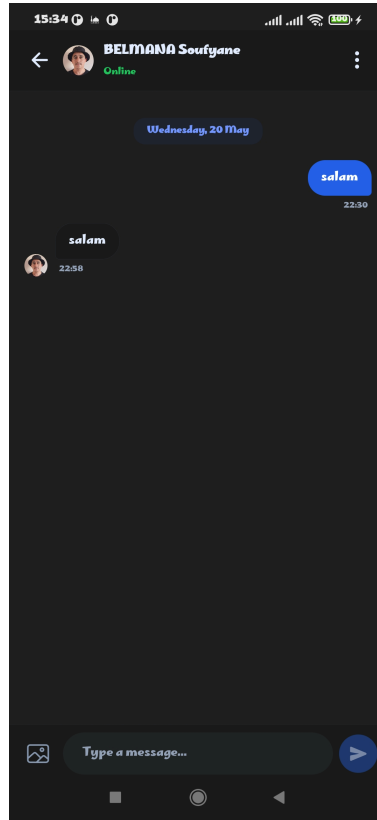


Figure 5.22: *Mobile App – Real-Time Chat Interface*

5.4.7 Financial Dashboard

The financial management interface provides transparency over monetary transactions:

- **Wallet Overview:** Summary cards display total received, total sent, and amount held in escrow. Metrics are calculated from transaction history.
- **Transaction History:** All financial transactions display in a paginated list with color-coded status badges. Year/month filtering enables historical review.
- **Transaction Details:** Individual transaction views show the breakdown including platform fee (10%), net amount, and current escrow status.
- **Payout Configuration:** Experts can save their CCP/RIP account details for payout processing. The interface validates input format.

Figure 5.23 illustrates the mobile App – Booking Details and Wallet/Finance View.

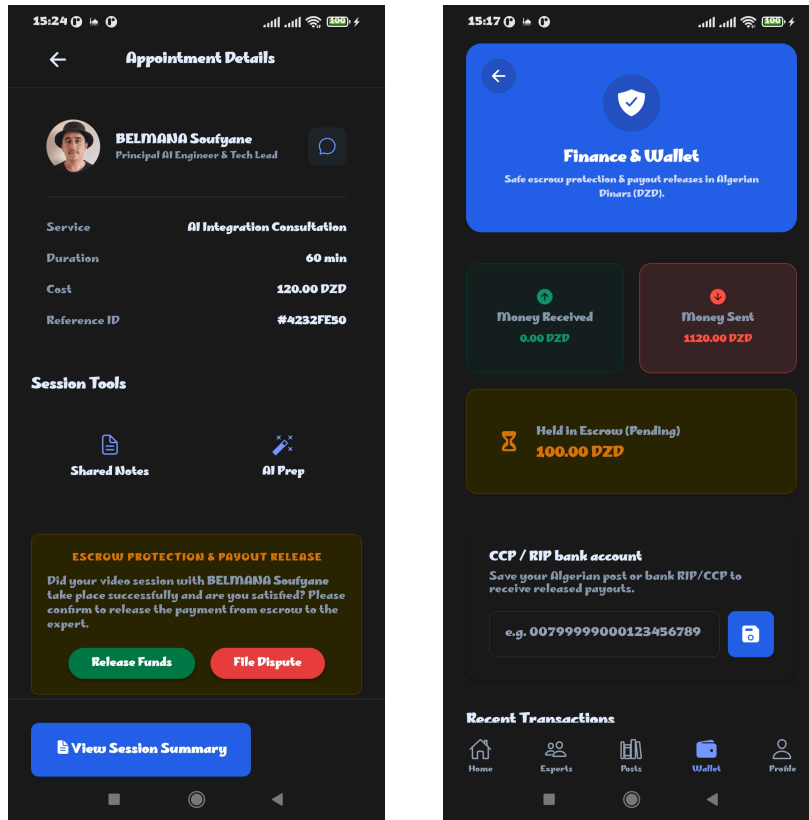


Figure 5.23: Mobile App – Booking Details and Wallet/Finance View

5.5 Backend and Database Implementation

The backend infrastructure is entirely managed through Supabase, eliminating the need for custom server development.

5.5.1 Database Schema

The backend database is implemented in PostgreSQL through Supabase. The relational database schema is structured into 11 main tables. The detailed column-level configurations, data types, check constraints, default values, and relational references are detailed in Table 5.2 through Table 5.12.

Table 5.2: *Supabase Database Schema: profiles Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Foreign Key referencing <code>auth.users(id)</code>
full_name	text	Nullable, display name of the user
avatar_url	text	Nullable, storage path to user's profile image
role	text	Default: 'user', Check constraint: 'user' or 'admin'
email	text	Nullable, Unique email address
created_at	timestamptz	Default: <code>now()</code>
updated_at	timestamptz	Default: <code>now()</code>
location	text	Nullable, geographical location
bio	text	Nullable, biography/description profile
show_email	bool	Default: <code>true</code> , privacy preference check
payment_method	text	Default: '', type of checkout card
card_last_four	text	Default: '', last four digits of user's payment card
billing_address	text	Default: '', invoice address information
title	text	Nullable, professional title headline
hourly_rate	int4	Nullable, hourly consultation rate in DZD
expertise	_text	Nullable, array of tags indicating specialties
ai_summary	text	Nullable, AI-generated summary of profile
rip	text	Nullable, bank/CCP RIP identifier for payouts

Table 5.3 shows the supabase Database Schema: services Table.

Table 5.3: *Supabase Database Schema: services Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
expert_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
name	text	Not Null, title of the service offered
duration	text	Nullable, consultation session duration
price	int4	Not Null, price of the service
created_at	timestamptz	Default: <code>now()</code>
description	text	Nullable, detailed description of service scope

Table 5.4 shows the supabase Database Schema: posts Table.

Table 5.4: *Supabase Database Schema: posts Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
author_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
title	text	Not Null, post title
content	text	Not Null, post content body
excerpt	text	Nullable, short introductory teaser
image_url	text	Nullable, post header image URL
tag	text	Nullable, category tag
read_time	text	Nullable, estimated read time
likes	int4	Default: 0, total likes count
comments_count	int4	Default: 0, total comments count
created_at	timestamptz	Default: <code>now()</code>
updated_at	timestamptz	Default: <code>now()</code>
media_urls	_text	Default: <code>'{}'::text[]</code> , array of supplemental media asset paths

Table 5.5 shows the supabase Database Schema: bookings Table.

Table 5.5: *Supabase Database Schema: bookings Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
user_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
expert_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
service_id	uuid	Nullable, Foreign Key referencing <code>public.services(id)</code>
status	text	Default: <code>'pending'</code> , Check constraint: <code>'pending', 'confirmed', 'completed', 'cancelled', 'approved_for_payout', 'disputed', or 'paid_out'</code>
scheduled_at	timestamptz	Not Null, scheduled start time
created_at	timestamptz	Default: <code>now()</code>
updated_at	timestamptz	Default: <code>now()</code>
audio_url	text	Nullable, session recording audio file path
transcript	text	Nullable, raw AI text transcription
dispute_reason	text	Nullable, reason text provided by the client for refund request

Table 5.6 shows the supabase Database Schema: reviews Table.

Table 5.6: *Supabase Database Schema: reviews Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
expert_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
user_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
booking_id	uuid	Nullable, Foreign Key referencing <code>public.bookings(id)</code>
rating	int4	Nullable, Check constraint: ≥ 1 and ≤ 5 stars
content	text	Nullable, user review text feedback
created_at	timestamptz	Default: <code>now()</code>

Table 5.7 shows the supabase Database Schema: conversations Table.

Table 5.7: *Supabase Database Schema: conversations Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
participant_1	uuid	Not Null, Foreign Key referencing <code>public.profiles(id)</code>
participant_2	uuid	Not Null, Foreign Key referencing <code>public.profiles(id)</code>
last_message_at	timestamptz	Default: <code>now()</code>
created_at	timestamptz	Default: <code>now()</code>

Table 5.8 shows the supabase Database Schema: messages Table.

Table 5.8: *Supabase Database Schema: messages Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
conversation_id	uuid	Not Null, Foreign Key referencing <code>public.conversations(id)</code>
sender_id	uuid	Not Null, Foreign Key referencing <code>public.profiles(id)</code>
content	text	Nullable, text message body
read	bool	Default: <code>false</code> , read notification toggle
created_at	timestamptz	Default: <code>now()</code>
media_url	text	Nullable, media file attachment path

Table 5.9 shows the supabase Database Schema: certifications Table.

Table 5.9: *Supabase Database Schema: certifications Table*

Column Name	Data Type	Constraints & Description
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>
expert_id	uuid	Nullable, Foreign Key referencing <code>public.profiles(id)</code>
name	text	Not Null, certification/degree title
issuer	text	Not Null, issuing organization
issue_date	date	Nullable, issue date
expiry_date	date	Nullable, expiry date
verification_url	text	Nullable, online certification verification path
created_at	timestamptz	Default: <code>now()</code>

Table 5.10 shows the supabase Database Schema: `post_likes` Table.

Table 5.10: *Supabase Database Schema: `post_likes` Table*

Column Name	Data Type	Constraints & Description
<code>id</code>	<code>uuid</code>	Primary Key, Default: <code>gen_random_uuid()</code>
<code>post_id</code>	<code>uuid</code>	Not Null, Foreign Key referencing <code>public.posts(id)</code>
<code>user_id</code>	<code>uuid</code>	Not Null, Foreign Key referencing <code>auth.users(id)</code>
<code>created_at</code>	<code>timestamptz</code>	Default: <code>now()</code>

Table 5.11 shows the supabase Database Schema: `post_comments` Table.

Table 5.11: *Supabase Database Schema: `post_comments` Table*

Column Name	Data Type	Constraints & Description
<code>id</code>	<code>uuid</code>	Primary Key, Default: <code>gen_random_uuid()</code>
<code>post_id</code>	<code>uuid</code>	Not Null, Foreign Key referencing <code>public.posts(id)</code>
<code>user_id</code>	<code>uuid</code>	Not Null, Foreign Key referencing <code>auth.users(id)</code>
<code>content</code>	<code>text</code>	Not Null, comment text body
<code>created_at</code>	<code>timestamptz</code>	Default: <code>now()</code>

Table 5.12: *Supabase Database Schema: `post_reports` Table*

Column Name	Data Type	Constraints & Description
<code>id</code>	<code>uuid</code>	Primary Key, Default: <code>gen_random_uuid()</code>
<code>post_id</code>	<code>uuid</code>	Not Null, Foreign Key referencing <code>public.posts(id)</code>
<code>reporter_id</code>	<code>uuid</code>	Not Null, Foreign Key referencing <code>public.profiles(id)</code>
<code>reason</code>	<code>text</code>	Not Null, reason choice category
<code>details</code>	<code>text</code>	Default: <code>'</code> , descriptive details
<code>status</code>	<code>text</code>	Default: <code>'pending'</code> , Check constraint: <code>'pending'</code> , <code>'reviewed'</code> , <code>'dismissed'</code> , or <code>'action_taken'</code>
<code>created_at</code>	<code>timestamptz</code>	Default: <code>now()</code>

5.5.2 API Implementation

Data operations are performed through the Supabase JavaScript client library:

- **Query Builder:** Supabase's query builder provides type-safe database operations with filtering, sorting, pagination, and relationship joins.
- **Real-Time Subscriptions:** Channels are created for conversations, messages, and video call signaling with appropriate filter conditions.
- **Remote Procedure Calls:** Complex operations like conversation creation/retrieval are encapsulated in PostgreSQL functions and invoked via `supabase.rpc()`.
- **File Storage:** Upload and download operations are managed through Supabase Storage buckets with RLS policy enforcement.

5.5.3 Row-Level Security Policies

Data access control is implemented through PostgreSQL RLS policies:

- **Public Read Access:** Expert profiles, services, certifications, and reviews are publicly readable to enable discovery.
- **Authenticated Write Access:** Users can only create, update, and delete their own data. Profile updates are restricted to the profile owner.
- **Admin Override:** Administrators have read/write access across all tables for platform management purposes.
- **Booking Isolation:** Users can only access bookings where they are the client or the expert. Administrators can view all bookings.

5.6 Internationalization Implementation

The platform supports English and Arabic through a comprehensive i18n infrastructure:

- **Translation Files:** JSON files define translation keys for both languages covering navigation, forms, notifications, and content labels.
- **Language Detection:** Device locale is auto-detected on first launch. Language preference is persisted in local storage.
- **RTL Support:** Arabic language selection triggers RTL layout mode. The web dashboard adjusts CSS direction, while the mobile app uses `I18nManager.forceRTL()` with app restart.
- **Translation Coverage:** Key interface elements including navigation, authentication forms, booking flow, and financial terms are fully translated.

5.7 System Testing, Evaluation, and Validation

To verify the correctness, reliability, and performance of the Istachir platform, a comprehensive testing and evaluation framework was executed. The testing process comprised functional validation of the core workflows, quantitative performance benchmarking of critical subsystems (WebRTC and AI pipelines), and security policy verification.

5.7.1 Functional Testing

Functional testing was conducted using black-box testing methodologies to validate the system's compliance with functional requirements. Test cases were structured to simulate standard user interactions across the client mobile application, the web admin dashboard, and the Supabase backend. Table 5.13 summarizes the core functional test cases, their expected outcomes, and verification results.

Table 5.13: *Functional Test Suite Results*

Test ID	Component	Test Description	Expected Result	Status
TC-01	Supabase Auth	Register a new client using email and password.	Authentication record created; profile table initialized.	Pass
TC-02	Web Admin	Administrator views uploaded expert certifications and verifies account.	Expert status updated to "verified"; badge shown on mobile app.	Pass
TC-03	Mobile Client	Client books a consultation session for a specific expert slot.	Pending booking record created; slot locked in database.	Pass
TC-04	Payment System	Client checkout session via Chargily card sandbox.	Escrow payment marked successful; booking confirmed via webhook.	Pass
TC-05	WebRTC Video	Client and Expert join the video consultation room.	Signaling succeeds via Supabase Realtime; P2P media streams load.	Pass
TC-06	AI Transcription	Upload session recording to storage and invoke AssemblyAI API.	Speech-to-Text returns speaker-labeled transcript.	Pass
TC-07	AI Summarizer	Dispatch session transcript text to Google Gemini API.	Returns structured Markdown summary; saved to bookings table.	Pass
TC-08	RAG Chatbot	Client questions expert chatbot on pricing and services.	Chatbot responds using expert data; refuses out-of-context info.	Pass

5.7.2 Performance Evaluation

Performance benchmarking was conducted on the communication and artificial intelligence subsystems to measure latency, processing overhead, and responsiveness.

5.7.2.1 WebRTC Connection Latency

The time taken to establish a peer-to-peer video connection was measured under two network configurations: direct connection (STUN discovery) and relay connection (TURN server routing). Table 5.14 presents the average connection latency over 50 test trials.

Table 5.14: *WebRTC Connection Establishment Latency*

Configuration	Min Latency (ms)	Max Latency (ms)	Average Latency (ms)
Direct P2P (STUN)	420	880	560
Relayed (TURN)	910	2100	1340

While TURN connections exhibit higher latency due to intermediary packet routing, the average of 1.34 seconds remains well within the acceptable threshold for real-time signaling, ensuring a reliable fallback for restrictive firewall environments.

5.7.2.2 Database Query Optimization

The efficiency of search queries on the `profiles` table was evaluated to measure the impact of PostgreSQL database indexes. Testing measured the execution time of full-text expert search queries with and without a GIN index on the search vector columns (`bio`, `title`, `full_name`, `expertise`). Table 5.15 illustrates the query execution time across varying dataset sizes.

Table 5.15: *Expert Search Query Execution Time (ms)*

Dataset Size	Without Index (Sequential Scan)	With GIN Index	Performance Gain
1,000 profiles	12 ms	1.5 ms	87.5%
10,000 profiles	98 ms	4.2 ms	95.7%
50,000 profiles	412 ms	12.1 ms	97.1%

The GIN index reduces query execution times to sub-millisecond levels for standard database sizes, ensuring sub-second response times for clients searching for experts.

5.7.2.3 AI Pipeline Processing Latency

The latency of the session summarization pipeline was evaluated across three different audio lengths. Latency was measured in three stages: uploading the recording to Supabase Storage, transcribing via AssemblyAI, and generating the summary via Google Gemini. Table 5.16 shows the results.

Table 5.16: *AI Summarization Pipeline Latency Breakdown*

Audio Duration	Upload Latency	AssemblyAI Transcription	Gemini Summarization	Total Time
10 Minutes	3.2 seconds	14.5 seconds	1.8 seconds	19.5 seconds
30 Minutes	8.5 seconds	38.2 seconds	2.6 seconds	49.3 seconds
60 Minutes	17.1 seconds	64.8 seconds	3.5 seconds	85.4 seconds

The transcription phase represents the primary bottleneck of the pipeline, which is standard for speech-to-text APIs. Because this pipeline executes asynchronously in the background and notifies users via push notifications upon completion, the 85-second duration for a full 1-hour session is highly acceptable and does not degrade the immediate user experience.

5.7.3 Security Policy Validation

To validate user data isolation, security testing was conducted on Supabase Row-Level Security (RLS) policies and external integrations.

- **RLS Policy Verification:** Automated test scripts attempted unauthorized database actions. When client account A attempted to insert or delete data in the chat messages table belonging to client account B, the PostgreSQL database engine rejected the operation with a security violation code, verifying that RLS policies prevent cross-tenant data leaks.
- **Webhook Integrity:** Payment confirmation webhooks from Chargily were tested. The endpoint successfully rejected webhook payloads that lacked valid cryptographic signatures matching the private key, preventing malicious actors from forging payment success events.

The execution of the testing suite demonstrates that Istachir is a functionally complete, secure, and optimized platform capable of facilitating on-demand consultation sessions.

5.8 Conclusion

This chapter has detailed the implementation of the Istachir consultation marketplace platform. We have described the development environment, the web administration dashboard implementation, the customer mobile application implementation, and the backend and database infrastructure. The implementation demonstrates the successful integration of modern web and mobile technologies including React, React Native, Supabase, WebRTC, and AI services to create a production-ready consultation marketplace. The modular architecture and comprehensive feature set provide a solid foundation for the platform's continued evolution.

Part VI

Conclusion and Perspectives

6.1 General Conclusion

This thesis presented the design, development, and implementation of Istachir, a comprehensive consultation marketplace platform that connects clients with verified experts through web and mobile interfaces. The project successfully demonstrated the integration of modern software technologies to address real-world challenges in the professional services market.

The platform delivers a complete ecosystem spanning expert discovery, consultation booking, secure payment processing, real-time video sessions, AI-powered analysis, and comprehensive administrative controls. By leveraging Supabase as a backend platform, we eliminated the need for custom server infrastructure while maintaining robust security through row-level policies and authentication workflows.

The web administration dashboard, built with React and TypeScript, provides powerful tools for platform management including user administration, financial oversight, and content moderation. The cross-platform mobile application, developed with React Native and Expo, delivers an intuitive client experience optimized for on-the-go usage across iOS and Android devices.

6.2 Key Achievements and Contributions

6.2.1 Technical Innovation

The project introduces several technical innovations:

- **Unified Backend Architecture:** The use of Supabase as a comprehensive backend platform demonstrates a modern approach to application development that reduces infrastructure complexity while maintaining scalability and security.
- **Real-Time Video Integration:** The WebRTC implementation with Supabase Realtime signaling provides a cost-effective, serverless video consultation solution without dedicated media servers.

- **AI-Enhanced Consultation:** Integration of Google Gemini AI for session summaries, profile Q&A, and transcript analysis demonstrates practical applications of large language models in professional services.
- **Cross-Platform Consistency:** The shared state management patterns, authentication infrastructure, and API communication between web and mobile platforms demonstrate effective code sharing strategies.

6.2.2 Practical Contributions

- A production-ready consultation marketplace platform tailored to the Algerian market with localized payment integration and bilingual support.
- An escrow-based payment system that protects both clients and experts while maintaining transparent fee structures.
- A comprehensive administrative dashboard enabling effective platform governance and financial management.
- Documentation of architectural decisions and implementation patterns that can inform similar platform development efforts.

6.3 Limitations and Challenges

6.3.1 Technical Limitations

- **Offline Support:** The current implementation requires active internet connectivity for most features. Offline support for browsing expert profiles and cached content is not implemented.
- **Push Notifications:** While the infrastructure supports push notifications, comprehensive notification integration for booking reminders, message alerts, and session reminders is not fully realized.
- **Advanced Analytics:** The analytics capabilities are limited to basic metrics. Advanced business intelligence features including user behavior tracking, conversion funnel analysis, and predictive analytics are not implemented.

6.3.2 Implementation Challenges

- **WebRTC Connection Reliability:** Peer-to-peer video connections can be unreliable in restrictive network environments. TURN relay servers introduce latency and bandwidth costs.
- **Arabic Language Support:** Ensuring consistent RTL layout behavior across all screens, particularly with dynamic content and mixed language text, required significant testing and refinement.
- **Payment Gateway Integration:** The Chargily payment integration required careful handling of redirect-based payment flows and webhook confirmation processing to ensure transactional integrity.

6.4 Future Perspectives

6.4.1 Short-Term Enhancements (6-12 Months)

- **Push Notification System:** Implement comprehensive push notifications for booking confirmations, session reminders, new messages, and promotional updates using Expo Push Notifications and Firebase Cloud Messaging.
- **Offline Mode:** Enable offline browsing of expert profiles and cached content using local storage and background data synchronization.
- **Advanced Search:** Implement full-text search across expert profiles, services, and posts with advanced filtering and sorting capabilities.

6.4.2 Medium-Term Developments (1-2 Years)

- **Dedicated Expert Application:** Develop a separate mobile application for experts featuring session management, availability scheduling, financial analytics, and professional development tools.
- **Comprehensive Loyalty Program:** Implement a tiered loyalty program with rewards, discounts, and exclusive benefits for frequent users. Integration with the platform's financial infrastructure for seamless reward redemption.
- **AI-Driven Recommendations:** Develop machine learning models for personalized expert recommendations based on user behavior, consultation history, and preferences.

6.4.3 Long-Term Vision (2-5 Years)

- **Group Consultation Support:** Enable multi-participant consultation sessions for team coaching, family therapy, and group training scenarios.
- **Marketplace Expansion:** Extend the platform to support additional service categories beyond consultations, including workshops, courses, and digital products.
- **Internationalization:** Expand language support to include French, Spanish, and other languages to serve broader markets beyond Algeria.

6.5 Final Remarks

The Istachir platform represents a significant step toward digitizing professional services in the Algerian market. By combining modern web and mobile technologies with AI capabilities and localized payment infrastructure, the platform addresses genuine market needs while demonstrating technical excellence.

The project's architecture prioritizes scalability, maintainability, and user experience, providing a solid foundation for future enhancements. The successful integration of diverse technologies – from React and React Native for frontend development to WebRTC for real-time communication and Google Gemini for AI features – demonstrates the feasibility of building complex digital platforms using modern, open-source technologies.

We believe that Istachir has the potential to make a meaningful impact on how professional services are accessed and delivered in Algeria, contributing to the broader digital transformation of the service economy. The lessons learned from this project, particularly around cross-platform development, real-time communication, and AI integration, provide valuable insights for similar platform development initiatives.

REFERENCES

- [1] shadcn. *shadcn/ui Documentation*. Accessed: 2025-12-01. 2024. URL: <https://ui.shadcn.com/>.
- [2] Tailwind Labs. *Tailwind CSS Documentation*. Accessed: 2025-12-01. 2024. URL: <https://tailwindcss.com/docs>.
- [3] Tanner Linsley. *TanStack React Query Documentation*. Accessed: 2025-12-01. 2024. URL: <https://tanstack.com/query/latest>.
- [4] Microsoft. *TypeScript Documentation*. Accessed: 2025-12-01. 2024. URL: <https://www.typescriptlang.org/docs/>.
- [5] Expo. *Expo Documentation*. Accessed: 2025-12-01. 2024. URL: <https://docs.expo.dev/>.
- [6] Supabase. *Supabase Documentation*. Accessed: 2025-12-01. 2024. URL: <https://supabase.com/docs>.
- [7] W3C. *WebRTC: Real-Time Communication in Browsers*. W3C Recommendation. 2023. URL: <https://www.w3.org/TR/webrtc/>.
- [8] Google. *Gemini API Documentation*. Accessed: 2025-12-01. 2024. URL: <https://ai.google.dev/docs>.
- [9] AssemblyAI. *AssemblyAI API Documentation*. Accessed: 2025-12-01. 2024. URL: <https://www.assemblyai.com/docs>.
- [10] Chargily. *Chargily Payment Gateway Documentation*. Accessed: 2025-12-01. 2024. URL: <https://dev.chargily.net/>.
- [11] Vercel. *Vercel Documentation*. Accessed: 2025-12-01. 2024. URL: <https://vercel.com/docs>.